ФГБОУ ВО «МИРЭА - Российский технологический университет»
Институт кибербезопасности и цифровых технологий

Кафедра КБ-1 «Защита информации»

Дисциплина: «Разработка и эксплуатация автоматизированных систем в защищенном исполнении»

Отчет по практической работе №2

Тема: “Разработка клиентской части АСЗИ”

Вариант задания №10

Выполнил:
Студент группы БАСО-04-22
Кузнецов Алексей Александрович

Проверил:
доцент Федин Ф.О.

Москва
2026

Учебная цель занятия: Углубить теоретические знания и выработать практические умения в области разработки клиентской части защищенной автоматизированной информационной системы.

Место проведения занятия: компьютерная аудитория.

Учебно-материальное обеспечение:

1) Методическая разработка.

2) Компьютерный класс с ПЭВМ.

3) ОС семейства Windows.

4) ERWin Data Modeler.

5) MS SQL Server 2019 Developer.

6) Среда разработки Microsoft Visual Studio.

СОДЕРЖАНИЕ

Задание №1. Создать базу данных своего варианта. 4

Задание №2. Выполнить анализ процесса разработки объектов базы данных АСЗИ (хранимых процедур) рассмотренного примера. 11

Задание №3. Выполнить и описать процессы создания проекта MS VS и подключения (программно!!!) к ранее созданной базе данных АСЗИ своего варианта. 19

Задание №4. Выполнить и описать процесс создания главной формы клиентского приложения АСЗИ своего варианта. 24

Задание №5. Выполнить и описать процесс создания диалоговых окон клиентского приложения АСЗИ своего варианта. 30

Задание №6. Выполнить и описать процесс написания программного кода клиентского приложения АСЗИ своего варианта. 39

Задание №7. Выполнить и описать процесс вызова хранимых процедур из клиентского приложения АСЗИ своего варианта. 45

Задание №8. Выполнить и описать процесс создания инсталлятора клиентского приложения АСЗИ своего варианта. 49

Задание №1. Создать базу данных своего варианта.

Ход выполнения задания

Предметной областью является учет кадров маркетинговой компании ООО «Маркетинг с умом». Рассматривается этап трудоустройства утвержденного кандидата, который начинается с момента принятия положительного решения руководством компании и завершается оформлением сотрудника в первый рабочий день. Участниками процесса являются менеджер по подбору персонала, специалист по кадровому делопроизводству, начальник отдела кадров, а также смежные подразделения (ИТ, АХУ, бухгалтерия).

Требования к базе данных, сформулированные на предыдущих занятиях, остаются актуальными. Функциональные требования включают ввод, редактирование, удаление и поиск данных о кандидатах, сотрудниках отдела кадров, согласиях на обработку персональных данных, трудовых договорах, заявках в смежные отделы, ресурсах, пропусках и отчетах; время отклика на запрос не более 1,5 с; многопользовательский доступ к данным; автоматизированное формирование заявок и выгрузку персональных данных из защищенной формы; проверку ввода и обеспечение ссылочной целостности; разграничение доступа на основе ролей; журналирование действий пользователей.

Аутентификация пользователей выполняется средствами базы данных АСЗИ, пароли хранятся только в виде хеш-значений. После трех неудачных попыток ввода пароля учетная запись блокируется автоматически, сведения о попытках сохраняются в базе данных. События безопасности фиксируются в отдельной таблице базы данных АСЗИ, что обеспечивает их сбор, запись и хранение в течение заданного срока. Права пользователей назначаются на уровне роли базы данных, прямой доступ к таблицам закрывается, работа с данными выполняется только через хранимые процедуры.

Проектирование выполнялось по ранее построенной трансформационной модели, поэтому структура таблиц, состав полей и связи между ними не изменялись. Для построения физической модели определены домены атрибутов, области допустимых значений и типы данных. Состав доменов приведен в таблице 1.

Таблица 1 – Домены атрибутов сущностей базы данных АСЗИ

Шифр домена	**Наименование домена**	**Определение домена**	**Тип данных**	**Пример значения**
D1	Порядковый номер	Целое число, принимает уникальные значения в пределах таблицы	int	123
D2	Дата	Дата в формате ЧЧ-ММ-ГГГГ, где ЧЧ от 01 до 31, ММ от 01 до 12, ГГГГ от 0000 до 9999	datetime	27-03-2026
D3	Строка символов длиной до 40 знаков	Множество символьных значений переменной длины не более 40 знаков	varchar(40)	Кузнецов Алексей Александрович
D4	Строка символов длиной до 150 знаков	Множество символьных значений переменной длины не более 150 знаков	varchar(150)	Россия, город Москва, улица Текстильщиков, дом 10, квартира 10
D5	Строка символов длиной до 20 знаков	Множество символьных значений переменной длины не более 20 знаков	varchar(20)	В работе
D6	Вещественное число	Числовое значение с плавающей точкой	float	120000,00
D7	Числовое значение	Длинное целое число	bigint	79994951100

Состав таблиц базы данных и их назначение приведены в таблице 2. Девять таблиц относятся к предметной области, две таблицы являются служебными и обеспечивают выполнение требований по защите информации.

Таблица 2 – Состав таблиц базы данных АСЗИ отдела кадров

Имя таблицы	**Назначение**	**Ключевые поля**
Kandidat	Персональные данные кандидатов	Nomer_kandidata (PK, IDENTITY)
Sotrudnik	Сотрудники отдела кадров, работающие в АСЗИ	Kod_sotrudnika (PK)

Продолжение таблицы 2

Имя таблицы	Назначение	Ключевые поля
Soglasie_na_obrabotku_PDn	Согласия кандидатов на обработку персональных данных	Nomer_soglasiya (PK), Nomer_kandidata (FK), Kod_sotrudnika (FK)
Trudovoy_dogovor	Трудовые договоры, подготовленные по кандидатам	Nomer_dogovora (PK), Nomer_kandidata (FK), Kod_sotrudnika (FK)
Zayavka	Заявки в смежные подразделения (ИТ, АХУ, бухгалтерия)	Nomer_zayavki (PK, IDENTITY), Nomer_kandidata (FK), Nomer_dogovora (FK), Kod_sotrudnika (FK)
Pozitsii_zayavki	Позиции заявок, то есть заказываемые ресурсы	Nomer_zayavki (PK, FK), Kod_resursa (PK, FK)
Resurs	Ресурсы и доступы, заказываемые для будущего сотрудника	Kod_resursa (PK)
Propusk	Пропуска будущих сотрудников и уровни доступа	Nomer_propuska (PK), Nomer_kandidata (FK), Kod_sotrudnika (FK)
Otchet	Ежемесячные отчеты для начальника отдела кадров	Nomer_otcheta (PK), Kod_sotrudnika (FK)
Polzovatel_ASZI	Служебная таблица. Учетные записи пользователей АСЗИ, роли, состояние учетной записи, число неудачных попыток входа	Kod_polzovatelya (PK), Login (UNIQUE), Kod_sotrudnika (FK)
Zhurnal_sobitiy_bezopasnosti	Служебная таблица. События безопасности с датой, временем, учетной записью, объектом, результатом и рабочей станцией	Nomer_zapisi (PK, IDENTITY)

Номера кандидатов и заявок не вводятся пользователем, а присваиваются автоматически. Для полей Nomer_kandidata таблицы Kandidat и Nomer_zayavki таблицы Zayavka задано свойство IDENTITY, поэтому очередное значение формирует сама СУБД. Начальные значения выбраны так, чтобы продолжалась нумерация ранее подготовленных данных, для кандидатов с числа 101 с шагом 1, для заявок с числа 401 с шагом 1. Такой порядок исключает появление двух записей с одинаковым номером, избавляет пользователя от ручного подбора номера и делает нумерацию непрерывной даже при одновременной работе нескольких пользователей.

Связи между таблицами реализованы ограничениями внешних ключей с именами R_4-R_20. Для всех связей заданы правила NO ACTION по удалению и обновлению, что исключает автоматическое каскадное изменение данных и заставляет приложение выполнять операции осознанно, в одной транзакции. Перечень ограничений приведен в таблице 3.

Таблица 3 – Ограничения ссылочной целостности базы данных АСЗИ

Ограничение	**Дочерняя таблица**	**Родительская таблица**	**Поле связи**
R_4	Pozitsii_zayavki	Resurs	Kod_resursa
R_5	Pozitsii_zayavki	Zayavka	Nomer_zayavki
R_8	Zayavka	Kandidat	Nomer_kandidata
R_10	Soglasie_na_obrabotku_PDn	Kandidat	Nomer_kandidata
R_11	Propusk	Kandidat	Nomer_kandidata
R_13	Trudovoy_dogovor	Sotrudnik	Kod_sotrudnika
R_14	Propusk	Sotrudnik	Kod_sotrudnika
R_15	Soglasie_na_obrabotku_PDn	Sotrudnik	Kod_sotrudnika
R_16	Trudovoy_dogovor	Kandidat	Nomer_kandidata
R_17	Otchet	Sotrudnik	Kod_sotrudnika
R_18	Zayavka	Trudovoy_dogovor	Nomer_dogovora
R_19	Zayavka	Sotrudnik	Kod_sotrudnika
R_20	Polzovatel_ASZI	Sotrudnik	Kod_sotrudnika

Т-модель базы данных для АСЗИ, построенная в инструментальном CASE-средстве моделирования данных, приведена на рисунке 1.

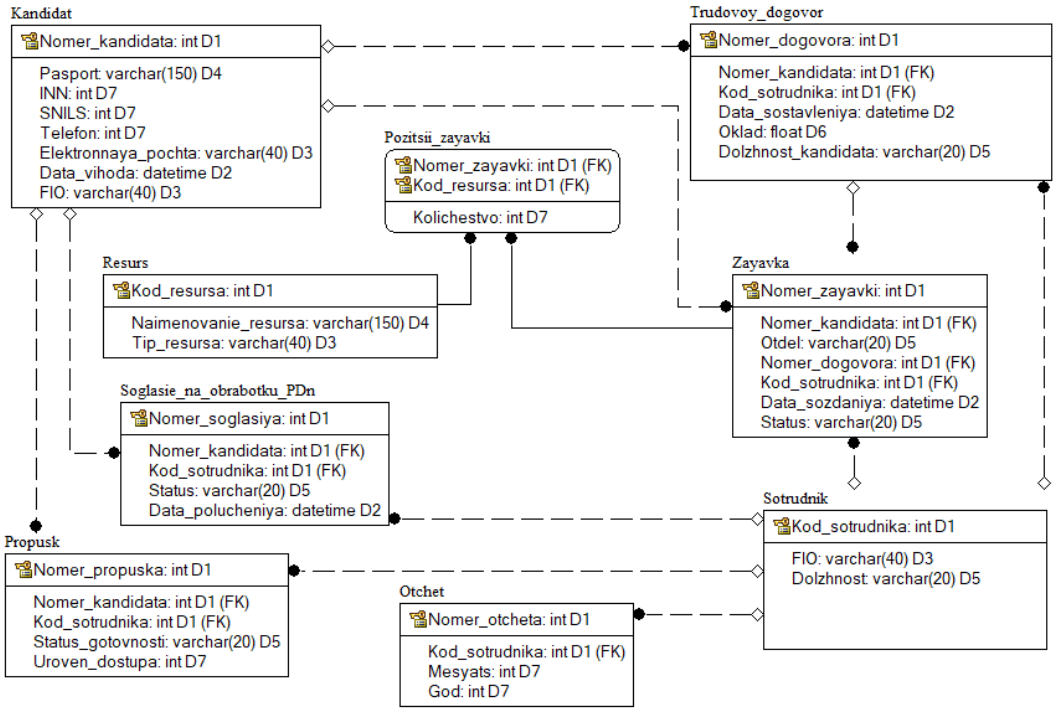


Рисунок 1 – Т-модель базы данных АСЗИ отдела кадров

Схема базы данных АСЗИ с указанием таблиц, полей и связей приведена на рисунке 2.

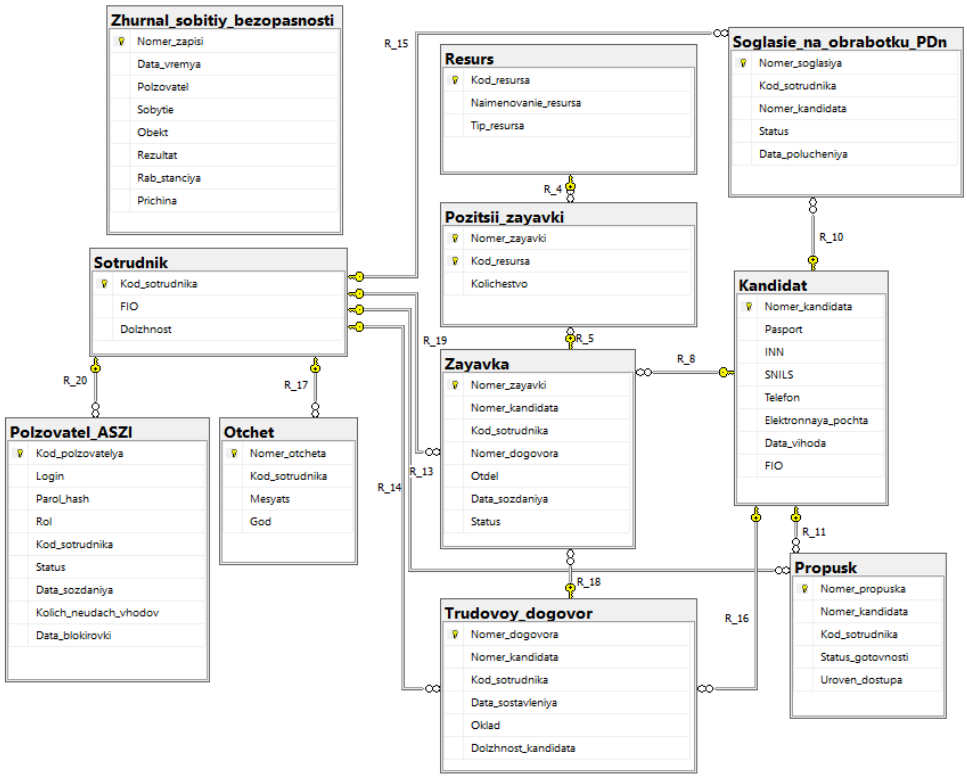


Рисунок 2 – Схема базы данных АСЗИ отдела кадров ООО «Маркетинг с умом», реализованная в СУБД

Создание базы данных в среде SQL Server Management Studio (SSMS) выполнялось скриптом, что обеспечивает повторяемость операции и позволяет применить его на любой рабочей станции. Скрипт создания базы данных сохранен в файле PZ9_01_BD_marketing_create.sql, его листинг приведен в приложении А. Работа выполнялась в следующем порядке.

1. В среде SQL Server Management Studio установлено соединение с экземпляром СУБД.
2. Открыт файл скрипта PZ9_01_BD_marketing_create.sql и выполнен запуск на исполнение командой Execute (клавиша F5).
3. Скрипт создал базу данных Kuznetsov_marketing с файлом данных Kuznetsov_marketing.mdf и файлом журнала транзакций Kuznetsov_marketing_log.ldf, для базы данных установлена модель восстановления FULL.
4. В базе данных созданы семь доменов D1-D7, одиннадцать таблиц, первичные ключи, ограничение уникальности на поле Login и тринадцать внешних ключей.
5. Выполнена проверка результата в узле Databases обозревателя объектов, при необходимости список обновлен командой Refresh.

Заполнение базы данных тестовыми данными выполнено скриптом PZ9_02_BD_marketing_data.sql. Скрипт загружает сведения о пяти сотрудниках отдела кадров, пяти кандидатах, ресурсах, трудовых договорах, согласиях на обработку персональных данных, заявках, позициях заявок, пропусках, отчетах и четырех учетных записях пользователей АСЗИ. Пароли пользователей внесены в таблицу не в открытом виде, а в виде результата хеширования функцией HASHBYTES по алгоритму SHA2_256, при этом к паролю добавлен логин в качестве простейшей соли. Открытые значения паролей в базе данных не хранятся.

Контроль загрузки выполнен запросом к таблице Kandidat и итоговой выборкой по количеству строк в каждой таблице. Результат на рисунке 3.

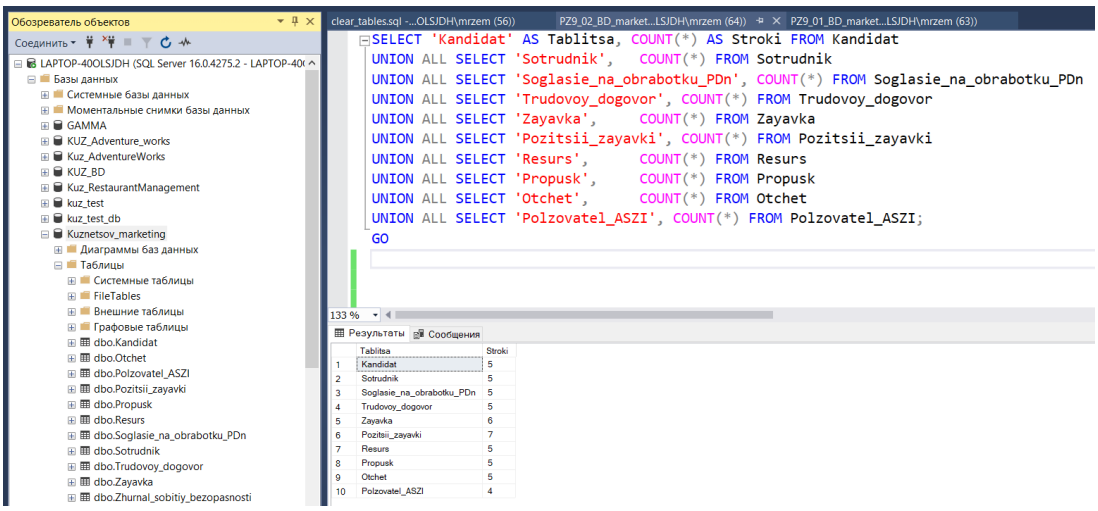


Рисунок 3 – Результат выполнения оператора SELECT из таблицы Kandidat

На уровне базы данных реализованы следующие механизмы защиты информации.

Ограничения целостности. Первичные и внешние ключи исключают появление записей, не связанных с данными предметной области, а ограничение уникальности не позволяет создать две учетные записи с одинаковым логином.

Хеширование аутентификационных данных. В таблице Polzovatel_ASZI хранится только хеш пароля, что снижает ущерб от компрометации базы данных.

Учет неудачных попыток входа. Значение поля Kolich_neudach_vhodov и состояние учетной записи позволяют реализовать автоматическую блокировку.

Регистрация событий безопасности. Таблица Zhurnal_sobitiy_bezopasnosti хранит дату, время, учетную запись, событие, объект, результат, рабочую станцию и причину отказа.

Разграничение доступа. Роли базы данных и запрет прямого доступа к таблицам описаны далее в отчете.

Резервное копирование. Модель восстановления FULL позволяет создавать полные и дифференциальные резервные копии базы данных штатными средствами СУБД по расписанию.

Задание №2. Выполнить анализ процесса разработки объектов базы данных АСЗИ (хранимых процедур) рассмотренного примера.

Выполнить и описать процесс разработки объектов базы данных АСЗИ своего варианта:

- не менее двух хранимых процедур без параметров;
- не менее двух хранимых процедур с параметрами.

Задачи, решаемые хранимыми процедурами, выбрать самостоятельно. Одна из процедур обязательно должна решать задачу, связанную с защитой информации.

Ход выполнения задания

В методической разработке объекты базы данных рассматриваются на примере защищенной информационной системы коммерческого банка, работающей с таблицами сведений о клиентах и их имуществе. Процесс включает в себя следующие шаги:

1. Открыть узел Programmability для БД в SSMS,
2. Выбрать Stored Procedures,
3. Создать процедуру New Stored Procedure,
4. Записать тело процедуры на T-SQL,
5. Выполнить скрипт,
6. Проверить вызов EXEC имя_процедуры,
7. Вызвать процедуру из приложения.

В базе данных АСЗИ отдела кадров разработаны шесть хранимых процедур без параметров. Они решают задачи вывода справочной и сводной информации, которая используется как в главной форме приложения, так и в отдельных диалоговых окнах. Две процедуры формируют справочники для выпадающих списков формы создания заявки. Перечень процедур приведен в таблице 4.

Таблица 4 – Хранимые процедуры без параметров

Имя процедуры	Решаемая задача	Возвращаемый результат	Применение
usp_Kandidaty_Spisok	Формирование списка кандидатов с состоянием согласия на обработку персональных данных, готовности пропуска и наличия трудового договора	Одна таблица	Таблица dataGridView1 главной формы
usp_Svodka_Zayavok	Сводка по заявкам в разрезе подразделений с подсчетом количества новых, находящихся в работе и исполненных заявок и числа заказанных ресурсов	Одна таблица	Форма сводки по заявкам
usp_Propuski_Gotovnost	Статистика готовности пропусков с минимальным, максимальным и средним уровнем доступа	Одна таблица	Форма готовности пропусков
usp_Zhurnal_Bezopasnosti	Вывод последних зарегистрированных событий безопасности	Одна таблица	Форма журнала событий безопасности
usp_Kandidaty_Vybor	Список кандидатов в виде пар «номер и фамилия» для выбора значения из выпадающего списка	Одна таблица	Выпадающий список кандидатов формы добавления заявки
usp_Resursy_Spisok	Список ресурсов с кодом, наименованием и типом ресурса для выбора значения из выпадающего списка	Одна таблица	Выпадающий список ресурсов формы добавления заявки

Процедура usp_Zhurnal_Bezopasnosti относится к процедурам, решающим задачи защиты информации. Она предназначена для контроля действий пользователей АСЗИ и позволяет просматривать сведения о входах в систему, отказах в доступе, изменениях персональных данных и попытках выполнения запрещенных операций.

Также разработаны пятнадцать хранимых процедур с параметрами. Они решают задачи выборки данных по условию, изменения записей и защиты

информации. Перечень процедур приведен в таблице 5.

Таблица 5 – Хранимые процедуры с параметрами

Имя процедуры	Параметры	Решаемая задача
usp_Zapis_v_zhurnal	Входные @Polzovatel, @Sobytie, @Obekt, @Rezultat, @Prichina	Служебная процедура регистрации события безопасности. Вызывается другими процедурами и приложением
usp_Vhod_v_sistemu	Входные @Login, @Parol; выходные @Rol, @Soobshchenie	Защита информации. Проверка подлинности пользователя, блокировка учетной записи после трех неудачных попыток, регистрация результата входа
usp_Kandidat_Dobavit	Входные @Nomer_kandidata, @FIO, @Pasport, @INN, @SNILS, @Telefon, @Elektronnaya_pochta, @Data_vihoda, @Polzovatel	Добавление кандидата с контролем вводимых значений и регистрацией изменения персональных данных
usp_Kandidat_Redaktirovat	Входные @Nomer_kandidata, @FIO, @Pasport, @INN, @SNILS, @Telefon, @Elektronnaya_pochta, @Data_vihoda, @Polzovatel	Изменение данных кандидата с регистрацией изменения персональных данных
usp_Kandidat_Udalit	Входные @Nomer_kandidata, @Polzovatel	Защита информации. Удаление кандидата с запретом удаления записей, по которым заключен трудовой договор или созданы заявки
usp_Zayavka_Sozdat	Входные @Nomer_zayavki, @Otdel, @Data_sozdaniya, @Status, @Nomer_kandidata, @Kod_sotrudnika, @Nomer_dogovora, @Polzovatel	Создание заявки в смежное подразделение с контролем допустимых значений и ссылочной целостности
usp_Zayavka_Redaktirovat	Входные @Nomer_zayavki, @Otdel, @Status, @Polzovatel	Изменение подразделения и состояния заявки
usp_Zayavka_Udalit	Входные @Nomer_zayavki, @Polzovatel	Удаление заявки вместе с позициями в одной транзакции
usp_Zhurnal_za_period	Входные @Data_ot, @Data_do	Выборка событий безопасности за указанный период
usp_Poisk_Kandidata	Входные @Fragment_FIO, @Rol, @Polzovatel	Защита информации. Поиск кандидатов по фрагменту фамилии с маскированием персональных данных для роли специалиста
usp_Zayavki_Kandidata	Входной @Nomer_kandidata	Вывод заявок и заказанных ресурсов по выбранному кандидату
usp_Statistika_PDn	Входной @God	Контроль объема обрабатываемых персональных данных и состояния согласий на обработку
usp_Dogovory_Spisok	Входной @Nomer_kandidata (необязательный)	Список трудовых договоров с должностью и датой составления. При указании кандидата выводятся только его договоры, что используется выпадающим списком формы добавления заявки
usp_Procedura_Parametry	Входной @Imya_procedury	Сведения о параметрах хранимой процедуры: имя, тип данных, размер, признаки выходного и необязательного параметра. Используется окном тестирования процедур для автоматического создания полей ввода

Основной процедурой защиты информации является

usp_Vhod_v_sistemu. Процедура принимает логин и пароль, вычисляет хеш введенного пароля по алгоритму SHA2_256 и сравнивает полученное значение с эталонным хешем из таблицы Polzovatel_ASZI. Дополнительно проверяются состояние учетной записи и допустимый интервал времени доступа. При неверном пароле счетчик неудачных попыток увеличивается, а после третьей неудачной попытки учетная запись переводится в состояние «Заблокирована». Каждый результат входа передается в процедуру регистрации события безопасности. Листинг размещен в приложении Б.

```
SET @Hash = CONVERT(VARCHAR(150), HASHBYTES('SHA2_256', @Parol + '|' + @Login), 2);
```

Процедура возвращает код, по которому клиентское приложение определяет результат проверки подлинности. Значение 0 соответствует успешному входу, значение 1 соответствует неверному паролю, значение 2 соответствует заблокированной учетной записи, значение 3 соответствует отсутствию учетной записи с указанным логином, значение 4 соответствует попытке входа вне разрешенного интервала времени. Через выходные параметры приложение получает роль пользователя и текст сообщения для вывода на форму.

Процедура usp_Poisk_Kandidata дополняет защиту персональных данных. Для роли «Специалист» реквизиты документов, страхового номера, телефона и адреса электронной почты выдаются маскированными, то есть пользователь видит только часть символов. Тем самым реализуется требование об ограничении доступа пользователя только к тем данным, которые необходимы для его работы.

Процедура usp_Kandidat_Udalit запрещает удаление персональных данных кандидата, если по нему уже заключен трудовой договор или созданы заявки. Такие записи имеют юридическое значение для кадрового учета, поэтому попытка удаления отклоняется, а сведения о ней сохраняются в журнале событий безопасности с результатом «Отказано».

Процедуры изменения данных проверяют корректность значений до начала изменения базы данных. Проверяются обязательность заполнения

полей, положительное значение номера, наличие символа «@» в адресе электронной почты, принадлежность подразделения и состояния заявки заданному перечню, существование связанных записей. При нарушении условия формируется сообщение при помощи инструкции THROW, при этом используются номера сообщений от 50001 и выше, которые не заняты системными сообщениями СУБД.

Изменение данных выполняется в транзакции с конструкцией TRY-CATCH. Если ошибка возникает в процессе записи, транзакция откатывается, а сведения об ошибке направляются в журнал событий безопасности. Такой порядок исключает частичное изменение связанных таблиц.

Разграничение доступа выполнено средствами СУБД. Созданы три учетные записи уровня сервера, три пользователя базы данных и три роли базы данных. Права назначены ролям, а пользователи включены в роли. Состав прав приведен в таблице 6. Скрипт разграничения доступа сохранен в файле PZ10_03_Roli_i_prava.sql.

Таблица 6 – Разграничение прав доступа ролей базы данных АСЗИ

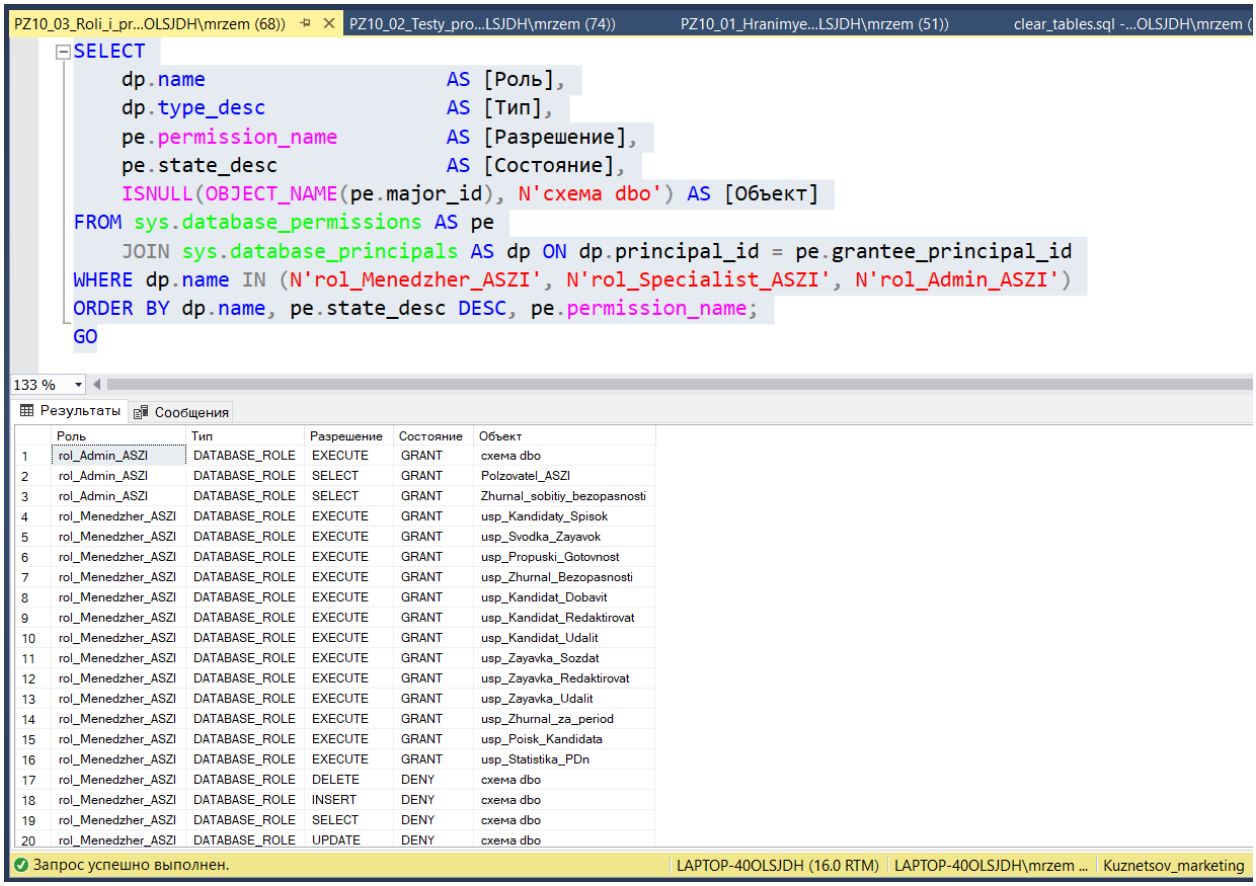
Роль базы данных	**Учетная запись**	**Разрешенные действия**	**Запрещенные действия**
rol_Menedzher_ASZI	log_Menedzher	Выполнение всех процедур работы с кандидатами, заявками, сводками и журналом событий	Прямой доступ к таблицам базы данных
rol_Specialist_ASZI	log_Specialist	Выполнение процедур просмотра списков, поиска с маскированием данных, создания и изменения заявок	Выполнение процедур удаления и работы с журналом событий, прямой доступ к таблицам
rol_Admin_ASZI	log_Admin_ASZI	Выполнение всех процедур базы данных, просмотр таблиц журнала событий и учетных записей	Изменение данных предметной области в обход процедур

Для роли «Специалист» изменение персональных данных кандидатов закрыто явными запретами на выполнение процедур usp_Kandidat_Dannye,

usp_Kandidat_Dobavit, usp_Kandidat_Redaktirovat и usp_Kandidat_Udalit. Явный запрет сохраняет силу даже в том случае, если в дальнейшем право на выполнение процедур будет выдано всей схеме базы данных.

Принцип минимально необходимых прав реализован запретом прямого доступа к таблицам для прикладных ролей инструкцией DENY SELECT, INSERT, UPDATE и DELETE на схему dbo. Это исключает обход проверок, реализованных в хранимых процедурах, при работе из клиентского приложения или из среды управления базой данных.

Контроль назначенных прав выполняется выборкой из системного представления sys.database_permissions. Результат проверки приведен на рисунке 4.



	Роль	Тип	Разрешение	Состояние	Объект
1	rol_Admin_ASZI	DATABASE_ROLE	EXECUTE	GRANT	схема dbo
2	rol_Admin_ASZI	DATABASE_ROLE	SELECT	GRANT	Polzovatel_ASZI
3	rol_Admin_ASZI	DATABASE_ROLE	SELECT	GRANT	Zhurnal_sobitiy_bezopasnosti
4	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Kandidaty_Spisok
5	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Svodka_Zayavok
6	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Propuski_Gotovnost
7	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Zhurnal_Bezopasnosti
8	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Kandidat_Dobavit
9	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Kandidat_Redaktirovat
10	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Kandidat_Udalit
11	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Zayavka_Sozdat
12	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Zayavka_Redaktirovat
13	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Zayavka_Udalit
14	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Zhurnal_za_period
15	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Poisk_Kandidata
16	rol_Menedzher_ASZI	DATABASE_ROLE	EXECUTE	GRANT	usp_Statistika_PDn
17	rol_Menedzher_ASZI	DATABASE_ROLE	DELETE	DENY	схема dbo
18	rol_Menedzher_ASZI	DATABASE_ROLE	INSERT	DENY	схема dbo
19	rol_Menedzher_ASZI	DATABASE_ROLE	SELECT	DENY	схема dbo
20	rol_Menedzher_ASZI	DATABASE_ROLE	UPDATE	DENY	схема dbo

Рисунок 4 – Назначенные права ролей базы данных АСЗИ

Проверка процедур выполнялась скриптом PZ10_02_Testy_procedur.sql. Скрипт последовательно вызывает процедуры, проверяет работу процедуры защиты информации и выводит состояние журнала событий безопасности.

Основные проверки и полученные результаты приведены в таблице 7.

Таблица 7 – Проверки работоспособности хранимых процедур АСЗИ

Проверяемое действие	Вызов	Ожидаемый результат
Вывод списка кандидатов	EXEC usp_Kandidaty_Spisok	Таблица из пяти строк с состоянием согласия, пропуска и договора
Сводка по заявкам	EXEC usp_Svodka_Zayavok	Три строки по подразделениям АХУ, бухгалтерия и ИТ
Поиск с полными данными	EXEC usp_Poisk_Kandidata @Fragment_FIO = N'Иванов', @Rol = N'Менеджер'	Персональные данные выданы полностью
Поиск с маскированием	EXEC usp_Poisk_Kandidata @Fragment_FIO = N'Иванов', @Rol = N'Специалист'	Реквизиты документов и контактов заменены символами маски
Добавление кандидата	EXEC usp_Kandidat_Dobavit с номером 106	Запись добавлена, событие зарегистрировано в журнале
Удаление кандидата с договором	EXEC usp_Kandidat_Udalit @Nomer_kandidata = 101	Операция отклонена, в журнале результат «Отказано»
Создание заявки с ресурсом	EXEC usp_Zayavka_Sozdat с параметрами @Kod_resursa и @Kolichestvo	Заявка и ее позиция записаны в одной транзакции, выходной параметр содержит присвоенный номер 407
Отклонение заявки с чужим договором	EXEC usp_Zayavka_Sozdat с номером договора другого кандидата	Операция отклонена сообщением 50012, транзакция отменена
Список кандидатов для выбора	EXEC usp_Kandidaty_Vybor	Таблица из пяти строк с номером и фамилией кандидата
Список ресурсов для выбора	EXEC usp_Resursy_Spisok	Таблица из пяти строк с кодом и наименованием ресурса
Список договоров кандидата	EXEC usp_Dogovory_Spisok @Nomer_kandidata = 101	Одна строка с номером договора, должностью и датой
Сведения о параметрах процедуры	EXEC usp_Procedura_Parametry @Imya_procedury = N'usp_Zayavka_Sozdat'	Десять строк с именами и типами параметров, выходной параметр помечен признаком Vyvod
Подготовка трудового договора	EXEC usp_Dogovor_Dobavit, номер подсказывает usp_Dogovor_Novyy_nomer	Договор создан, сведения записаны в журнал, кандидат получает должность для заявок
Список процедур по роли	EXEC usp_Dostupnye_Procedury @Rol = N'Специалист'	В списке нет процедур работы с персональными данными и журналом событий
Отказ в параметрах запрещённой процедуры	EXEC usp_Procedura_Parametry @Imya_procedury = N'usp_Kandidat_Dannye', @Rol = N'Специалист'	Операция отклонена сообщением 50021
Регистрация согласия и пропуска	EXEC usp_Soglasie_Sohranit, EXEC usp_Propusk_Sohranit	Согласие зарегистрировано, номер пропуска присвоен автоматически
Удаление договора с заявками	EXEC usp_Dogovor_Udalit	Операция отклонена сообщением 50025, событие записано в журнал
Перечень записей, связанных с кандидатом	EXEC usp_Kandidat_Svyazi @Nomer_kandidata = 101	Строки с трудовым договором, заявками, согласием и пропуском и признак Препятствует_удалению. Перечень показывает приложение перед удалением кандидата
Проверка свободного номера договора	EXEC usp_Dogovor_Novyy_nomer @Nomer_dogovora = @N OUTPUT	Выходной параметр содержит номер, следующий за наибольшим, например 206
Готовность пропусков по уровням	EXEC usp_Propuski_Gotovnost	Строки по уровням доступа 1 - 4 с количеством готовых, заказанных и новых пропусков
Удаление кандидата с договором	EXEC usp_Kandidat_Udalit @Nomer_kandidata = 101	Операция отклонена, в журнале результат «Отказано»

Продолжение таблицы 7

Проверяемое действие	Вызов	Ожидаемый результат
Успешный вход в систему	EXEC usp_Vhod_v_sistemu с логином petrova	Код возврата 0, роль «Менеджер», событие зарегистрировано
Три неудачные попытки входа	Троекратный вызов usp_Vhod_v_sistemu с неверным паролем	Коды возврата 1, 1, 2, учетная запись переведена в состояние «Заблокирована»

Результат выполнения процедуры usp_Vhod_v_sistemu приведен на рисунках 5 и 6. После трех неудачных попыток входа поле Kolich_neudach_vhodov принимает значение 3, поле Status принимает значение «Заблокирована», а поле Data_blokirovki заполняется текущей датой и временем.

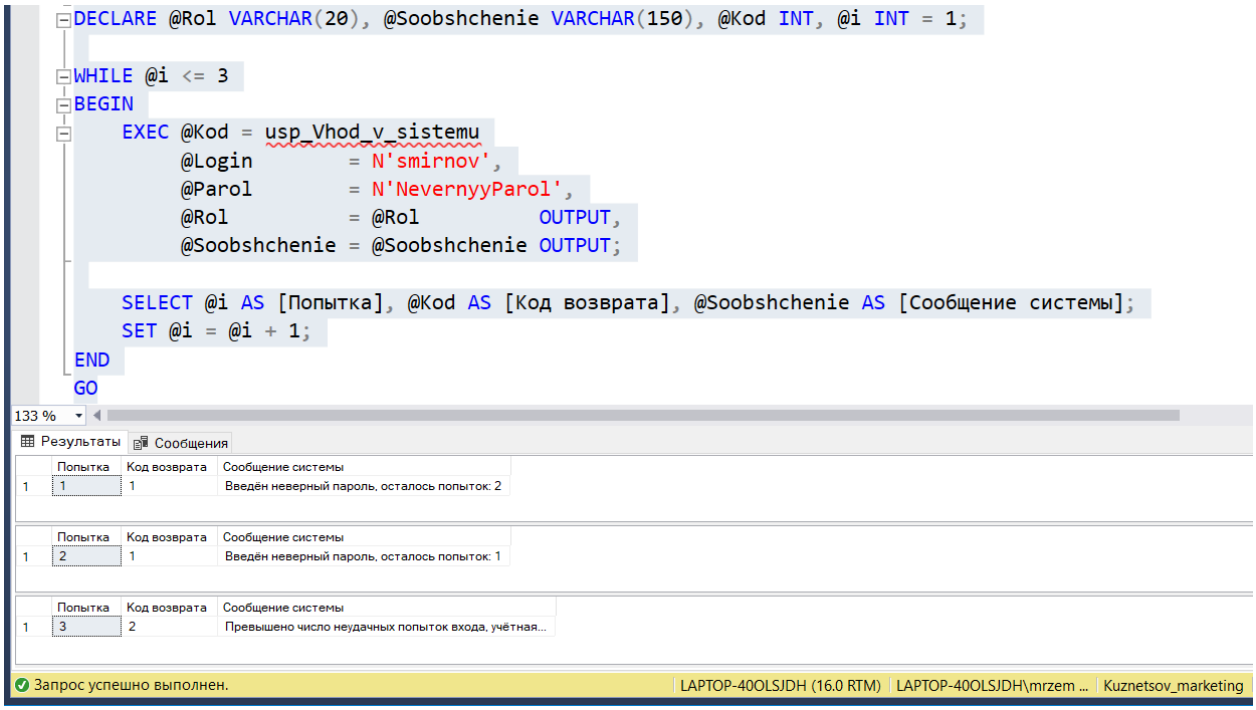


Рисунок 5 – Процесс выполнения процедуры usp_Vhod_v_sistemu

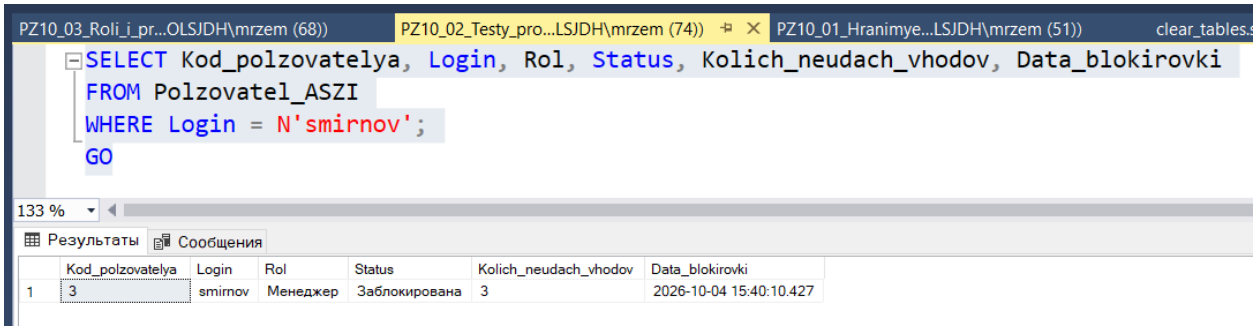


Рисунок 6 – УЗ заблокирована

Задание №3. Выполнить и описать процессы создания проекта MS VS и подключения (программно!!!) к ранее созданной базе данных АСЗИ своего варианта.

Ход выполнения задания

Разработка клиентского приложения велась в среде Microsoft Visual Studio. Работа выполнена в следующем порядке.

1. Запущена среда Microsoft Visual Studio и выбрана команда создания нового проекта.

2. В качестве шаблона выбран шаблон “Приложение Windows Forms (.NET Framework)”, который создает приложение с графическим интерфейсом на языке C#.

3. Задано имя проекта ASZI_OK, указан каталог размещения. Имя решения совпадает с именем проекта.

4. В свойствах проекта выбрана целевая платформа .NET Framework 4.7.2.

5. В проект добавлены файлы форм, модуль класса работы с базой данных RabotaSBazoy.cs, файл конфигурации App.config и папка Baza для размещения файла базы данных при развертывании приложения.

Процесс продемонстрирован на рисунке 7.

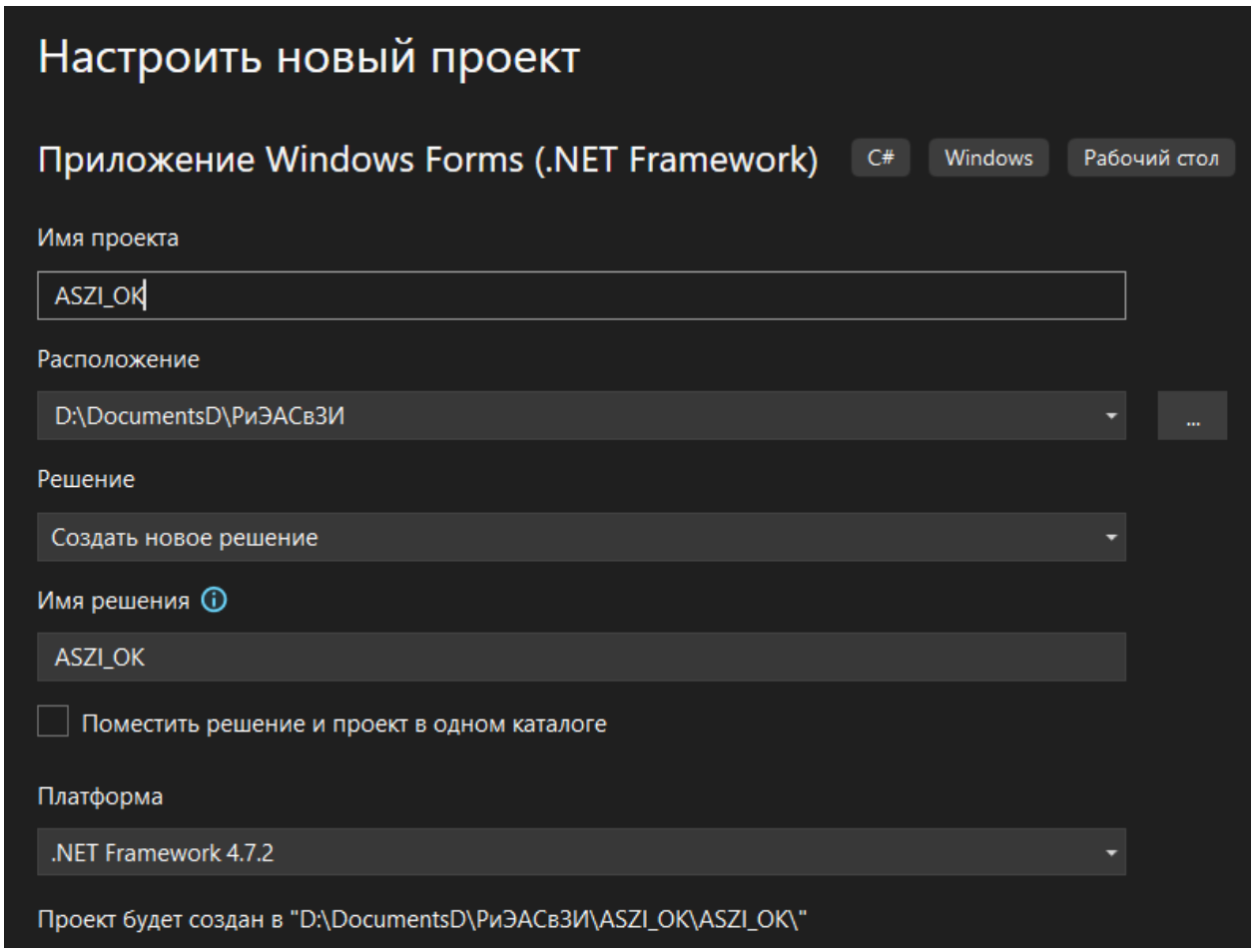


Рисунок 7 – Создание проекта по шаблону Windows Forms App (.NET Framework) в среде Microsoft Visual Studio

Состав файлов проекта приведен на рисунке 8 и в таблице 8.

Таблица 8 – Состав файлов проекта ASZI_OK

Файл проекта	**Назначение**
ASZI_OK.sln и ASZI_OK.csproj	Файл решения и файл проекта, содержащие сведения о составе проекта и параметрах сборки
Program.cs	Точка входа приложения. Перед запуском главной формы выполняется вход в систему
RabotaSBazoy.cs	Класс доступа к базе данных. Содержит строку соединения и методы выполнения команд и хранимых процедур
Form1.cs, Form1.Designer.cs	Модуль главной формы приложения и код, созданный конструктором форм
Form2.cs – Form16.cs	Пятнадцать диалоговых окон приложения
App.config	Файл конфигурации приложения
Properties/AssemblyInfo.cs	Сведения о сборке, включая наименование приложения и данные о разработчике
Baza/Прочитай_меня.txt	Пояснение о размещении файла базы данных при развертывании приложения
Ikony.cs	Создание и отрисовка иконок для интерфейса

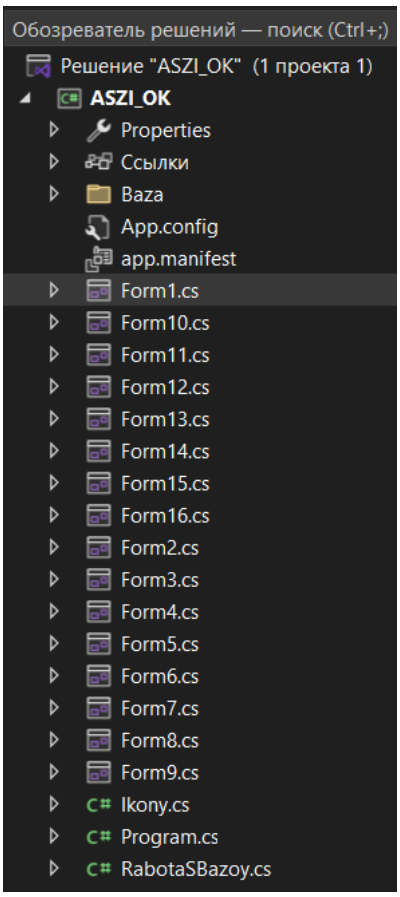


Рисунок 8 – Состав проекта ASZI_OK в обозревателе решений

Подключение к базе данных выполнено программно, без применения мастеров и источников данных, размещаемых в проекте. Для этого в модуль RabotaSBazoy.cs подключено пространство имен System.Data.SqlClient, в котором находятся классы SqlConnection, SqlCommand, SqlDataAdapter, SqlParameter и SqlDataReader.

Строка соединения задана статическим полем класса и предусматривает два варианта подключения. Первый вариант используется при работе с зарегистрированной БД на экземпляре сервера. Второй вариант применяется после установки приложения на рабочую станцию, когда база данных размещается рядом с исполняемым файлом и подключается по имени файла.

Параметр Data Source задает экземпляр СУБД, параметр Initial Catalog задает имя базы данных, параметр AttachDbFilename задает путь к файлу базы данных, параметр Integrated Security включает проверку подлинности средствами операционной системы, параметр User Instance разрешает работу с пользовательским экземпляром СУБД. Открытые логины и пароли в строке

соединения не указываются, что соответствует требованиям по защите информации.

Имя экземпляра СУБД в параметре Data Source зависит от способа установки SQL Server. Для экземпляра по умолчанию, который имеет имя MSSQLSERVER, в строке соединения указывается localhost, (local), точка либо имя компьютера в сети. Имя именованного экземпляра записывается после обратной косой черты, например, .\SQLEXPRESS. Запись имени несуществующего экземпляра приводит к ошибке «Сервер не найден или недоступен» с кодом 26, поскольку клиентская библиотека не может определить имя экземпляра.

Класс доступа к базе данных содержит четыре метода и один служебный метод. Метод MyExecuteNonQuery выполняет команды изменения данных, которые не возвращают набор строк. Метод MyExecuteQuery возвращает таблицу по тексту запроса, метод MyExecuteProcedure вызывает хранимую процедуру без параметров, метод MyExecuteProcedureWithReturn вызывает процедуру с параметрами и получает код возврата, метод ZapisVZhurnal регистрирует событие безопасности. Все методы используют конструкцию using, что гарантирует закрытие соединения даже при возникновении исключительной ситуации. Полный листинг класса приведен в приложении Г.

Проверка подключения выполняется при первом отображении главной формы в методе Form1_Shown. Метод вызывает заполнение таблиц и выводит сообщение о результате. Если соединение установить не удалось, выводится сообщение о неудачном подключении с текстом исключительной ситуации. Результат проверки приведен на рисунке 9.

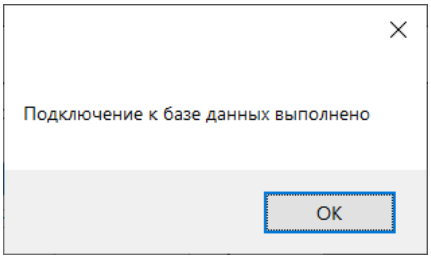


Рисунок 9 – Успешное подключение к БД

Для проверки настроек подключения подготовлен скрипт PZ11_01_Diagnostika_podklucheniya.sql. Скрипт выводит имя экземпляра СУБД, режим проверки подлинности, состав учётных записей сервера и их отображение в базу данных, проверяет права текущего пользователя и выполняет тестовый вызов процедур.

Отдельно следует учитывать, что прикладные учётные записи АСЗИ (petrova, sidorova, smirnov, ivanova) хранятся в таблице Polzovatel_ASZI и в списке входов сервера не отображаются. В списке входов сервера находятся только учётные записи log_Menedzher, log_Specialist и log_Admin_ASZI, а пользователи базы данных usr_Menedzher, usr_Specialist и usr_Admin_ASZI отображаются в разделе Databases, Kuznetsov_marketing, Security, Users. Такое разделение соответствует проектному решению: средствами СУБД проверяется доступ к данным, а проверка подлинности работников оператора выполняется процедурой usp_Vhod_v_sistemu по данным таблицы Polzovatel_ASZI.

Задание №4. Выполнить и описать процесс создания главной формы клиентского приложения АСЗИ своего варианта.

Ход выполнения задания

Главная форма приложения названа Form1 и является основным окном, через которое пользователь выполняет все операции с данными. Для формы настроены свойства, перечисленные в таблице 9.

Таблица 9 – Свойства главной формы приложения

Свойство	Значение	Назначение
Text	Защищенная автоматизированная информационная система отдела кадров	Заголовок окна приложения
StartPosition	CenterScreen	Окно открывается по центру экрана
MaximizeBox	True	Разрешает развертывание окна, поскольку таблицы данных имеют значительную ширину
FormBorderStyle	Sizable	Размер окна изменяется пользователем
MinimumSize	Размер 880 на 620 точек	Предел уменьшения окна, при котором элементы ещё не перекрываются
ClientSize	Размер 1090 на 660 точек	Размер рабочей области окна, достаточный для таблиц и панели команд
AutoScaleMode	Font	Размеры элементов пересчитываются при изменении системного шрифта
Name	Form1	Имя класса формы, используемое в программном коде

Внешний вид главной формы после проектирования приведен на рисунке 10.

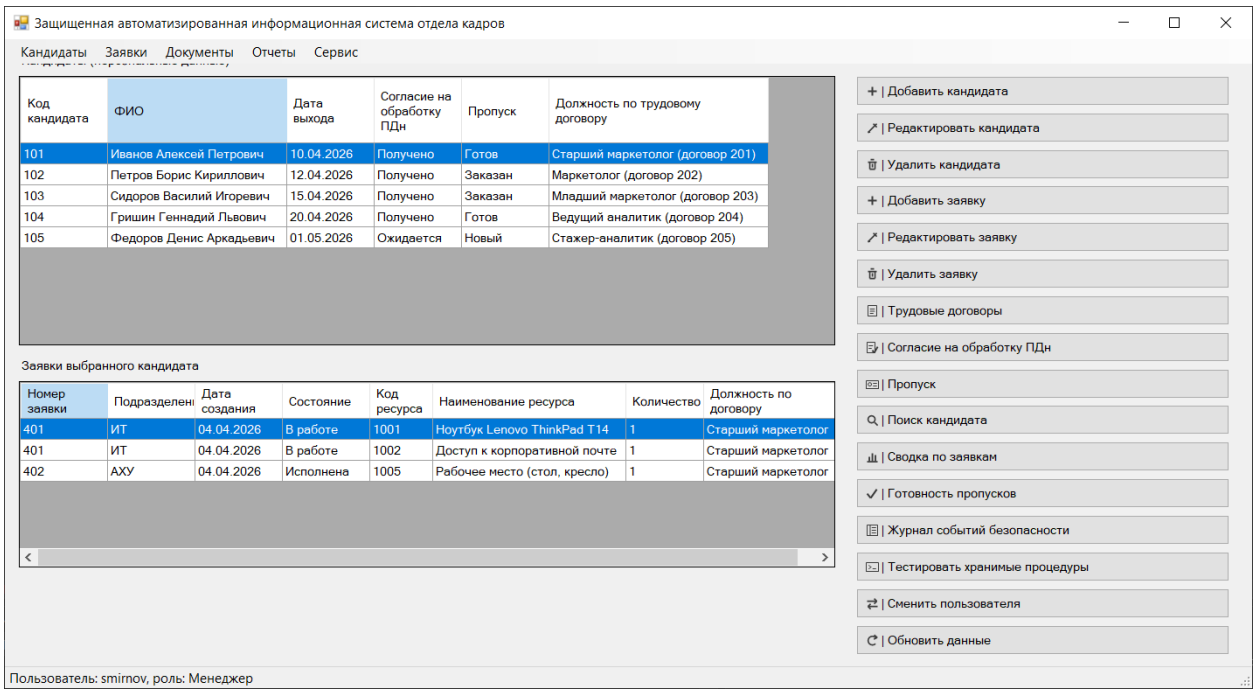


Рисунок 10 – Макет главной формы приложения АСЗИ отдела кадров

На главной форме размещены две таблицы данных, шестнадцать кнопок управления, строка меню и строка состояния. Состав элементов управления приведен в таблице 10.

Таблица 10 – Элементы управления главной формы приложения

Элемент управления	Тип	Назначение
label1	Label	Поясняющий текст «Кандидаты (персональные данные)»
label2	Label	Поясняющий текст «Заявки выбранного кандидата»
dataGridView1	DataGridView	Список кандидатов с состоянием согласия, пропуска и трудового договора
dataGridView2	DataGridView	Заявки и заказанные ресурсы выбранного кандидата
button1	Button	Команда «Добавить кандидата»
button2	Button	Команда «Редактировать кандидата»
button3	Button	Команда «Удалить кандидата»
button4	Button	Команда «Добавить заявку»
button5	Button	Команда «Редактировать заявку»
button6	Button	Команда «Удалить заявку»
button7	Button	Команда «Трудовые договоры», открывает окно Form13
button8	Button	Команда «Согласие на обработку ПДн», открывает окно Form15
button9	Button	Команда «Пропуск», открывает окно Form16

Продолжение таблицы 10

Элемент управления	Тип	Назначение
button10	Button	Команда «Поиск кандидата»
button11	Button	Команда «Сводка по заявкам»
button12	Button	Команда «Готовность пропусков»
button13	Button	Команда «Журнал событий безопасности»
button14	Button	Команда «Тестировать хранимые процедуры»
button15	Button	Команда «Сменить пользователя»
button16	Button	Команда «Обновить данные», повторяет чтение обеих таблиц
menuStrip1	MenuStrip	Строка меню с разделами «Кандидаты», «Заявки», «Документы», «Отчеты», «Сервис»
statusStrip1	StatusStrip	Строка состояния с логином и ролью текущего пользователя
statusLabel1	ToolStripStatusLabel	Вывод сведений о пользователе, выполнившем вход в систему

Рабочие станции имеют разное разрешение экрана, поэтому интерфейс формы настроен так, чтобы элементы подстраивались под размер окна. Настройка выполнена свойством Anchor, которое задает расстояния от элемента до краев формы, сохраняемые при изменении ее размера. Состав привязок приведен в таблице 11.

Таблица 11 – Привязка элементов управления главной формы к краям окна

Элемент управления	Привязка к краям	Поведение при растягивании окна
dataGridView1	Верх, лево, право	Увеличивается по ширине вместе с окном, высота сохраняется
dataGridView2	Верх, низ, лево, право	Увеличивается и по ширине, и по высоте, занимая свободное место
button1 ... button16	Верх, право	Панель команд сохраняет положение относительно правого края окна
label1, label2	Верх, лево	Надписи остаются на своих местах у левого края
statusStrip1	Закреплен у нижнего края	Автоматически растягивается по ширине окна

Аналогично настроены окна Form8 и Form9 (таблица сводки увеличивается по ширине, кнопка сохраняет положение у нижнего края), окно Form10 (таблица журнала растягивается по обеим сторонам), окно Form11 (строка параметров, таблица результата и поля сообщения системы), окно

Form12 (таблица результата поиска). Для окон Form8 ... Form12 задано свойство MinimumSize, которое не позволяет уменьшить окно до размера, при котором элементы перекрываются.

Диалоговые окна ввода данных Form2 ... Form7 имеют фиксированный размер, свойство FormBorderStyle которых установлено в значение FixedSingle. Такой режим исключает случайное изменение размеров и появление пустого места в окне ввода.

Дополнительно в проект добавлен файл app.manifest, в котором включена поддержка масштабирования интерфейса. Параметр dpiAware учитывает системное масштабирование экрана, значение PerMonitorV2 обеспечивает правильный размер элементов при переносе окна на монитор с другой плотностью точек. Без этого файла на экране с масштабом от 125 процентов элементы окна отображались бы размытыми или слишком мелкими.

Для таблиц dataGridView1 и dataGridView2 настроены свойства, обеспечивающие защиту от случайного изменения данных. Свойство EditMode установлено в значение EditProgrammatically, свойство ReadOnly установлено в значение True, свойство SelectionMode установлено в значение FullRowSelect, свойство RowHeadersVisible установлено в значение False. Редактирование данных выполняется только программно через хранимые процедуры, поэтому произвольное изменение ячеек клавиатурой невозможно.

Разделы меню дублируют команды кнопок и позволяют выполнять все операции с клавиатуры. Структура меню приведена в таблице 12.

Таблица 12 – Структура меню главной формы

Раздел меню	**Команды раздела**
Кандидаты	Добавить, Редактировать, Удалить, Поиск
Заявки	Добавить, Редактировать, Удалить
Документы	Трудовые договоры, Согласие на обработку ПДн, Пропуск
Отчеты	Сводка по заявкам, Готовность пропусков, Журнал событий безопасности
Сервис	Тестировать хранимые процедуры, Сменить пользователя, О программе

Строка состояния выводит логин и роль пользователя, который прошел вход в систему. Эти сведения используются также при разграничении доступных команд, что позволяет пользователю видеть текущий режим работы приложения. Главная форма, редактируемая в Microsoft Visual Studio, показана на рисунке 11.

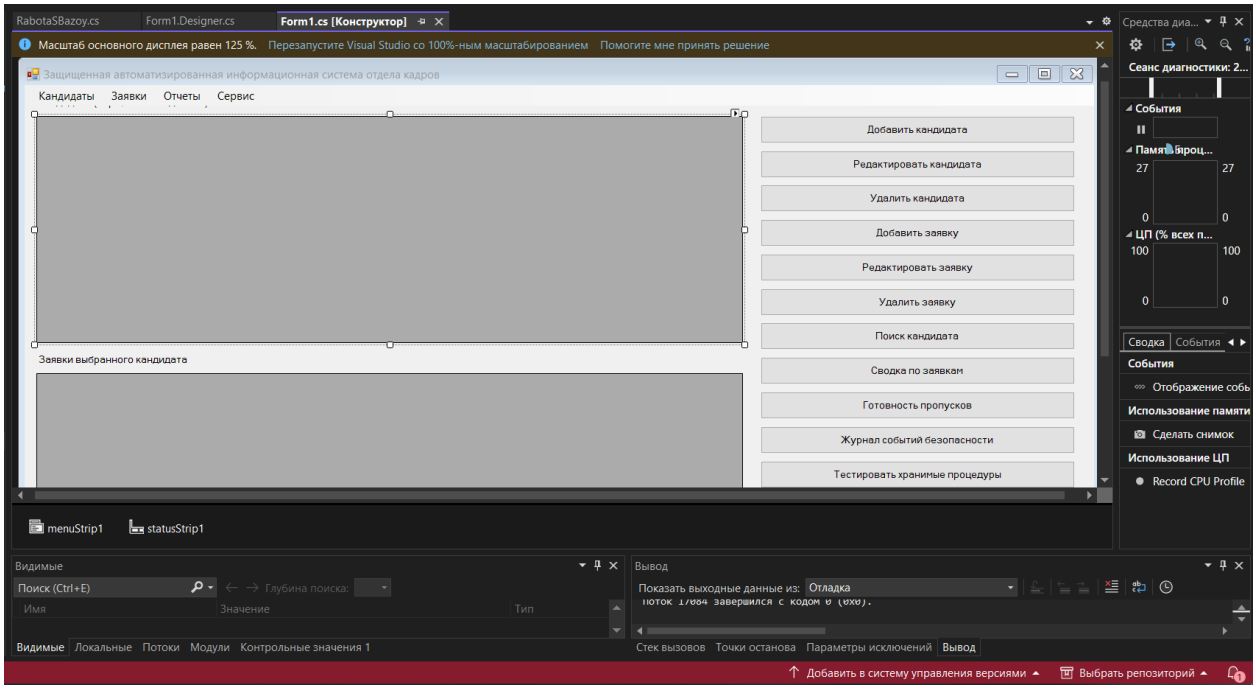


Рисунок 11 – Главная форма приложения в режиме конструктора

При первом отображении формы вызывается метод Form1_Shown, который заполняет таблицу кандидатов, заполняет таблицу заявок выбранного кандидата, выводит сведения о пользователе и применяет ограничения по роли. Ограничения по роли закрывают команды удаления записей и команды работы с журналом событий безопасности для пользователя с ролью «Специалист» на уровне интерфейса, дополнительно те же операции запрещены на уровне базы данных инструкцией DENY, о чем было сказано ранее.

Окно редактирования кандидата заполняется полными данными. Таблица кандидатов содержит только часть реквизитов, поэтому при открытии окна вызывается хранимая процедура usp_Kandidat_Dannye, которая возвращает паспортные данные, ИНН, СНИЛС, телефон, адрес электронной

почты и дату выхода. Номер кандидата в окне редактирования изменению не подлежит, поскольку является первичным ключом. Обращение к персональным данным регистрируется в журнале событий безопасности событием «Просмотр ПДн кандидата».

При переходе на другую строку таблицы кандидатов срабатывает обработчик события CellEnter. Обработчик определяет номер кандидата в первой колонке текущей строки и вызывает хранимую процедуру, которая возвращает заявки и ресурсы этого кандидата. За счет этого таблицы связаны и ведут себя как главная и подчиненная области данных.

Для просмотра актуальных сведений на главной форме предусмотрена команда «Обновить данные». Кнопка button16 и одноименный пункт меню «Сервис» повторно читают список кандидатов и заявки выбранного кандидата, при этом выбор строки сохраняется, поэтому таблица заявок остается по тому же кандидату. Таблицы обновляются и автоматически после закрытия окон работы с трудовыми договорами, согласиями и пропусками: подготовленный договор или оформленный пропуск сразу видны в списке кандидатов, и повторно входить в приложение не требуется.

Перед удалением кандидата приложение вызывает процедуру usp_Kandidat_Svyazi и показывает перечень связанных записей: трудовой договор, заявки, согласие на обработку персональных данных и пропуск. Проверки в процедуре usp_Kandidat_Udalit при этом остаются прежними, то есть правила удаления юридически значимых записей не изменились, а пользователь получает объяснение причины отказа и может удалить лишний документ.

При закрытии формы вызывается обработчик Form1_FormClosing, который регистрирует в журнале событий безопасности завершение сеанса работы пользователя. Такой прием обеспечивает полноту регистрируемых событий.

Задание №5. Выполнить и описать процесс создания диалоговых окон клиентского приложения АСЗИ своего варианта.

Ход выполнения задания

Для приложения разработано одиннадцать диалоговых окон. Каждое окно решает одну задачу и открывается из главной формы. Состав окон приведен в таблице 13.

Таблица 13 – Состав диалоговых окон клиентского приложения

Форма	Заголовок окна	Состав элементов управления	Назначение
Form2	Вход в систему	Три надписи, два поля ввода, две кнопки	Ввод логина и пароля, вывод сообщения о результате проверки подлинности
Form3	Добавить кандидата	Восемь надписей, восемь полей ввода, две кнопки	Ввод персональных данных нового кандидата
Form4	Редактировать кандидата	Восемь надписей, восемь полей ввода, две кнопки	Изменение данных выбранного кандидата
Form5	Добавить заявку	Семь надписей, семь полей ввода, две кнопки	Создание заявки в смежное подразделение
Form6	Редактировать заявку	Три надписи, три поля ввода, две кнопки	Изменение подразделения и состояния заявки
Form7	Подтверждение удаления	Две надписи, две кнопки	Подтверждение удаления записи о кандидате или заявки
Form8	Сводка по заявкам	Надпись, таблица данных, кнопка	Отображение сводки по подразделениям
Form9	Готовность пропусков	Надпись, таблица данных, кнопка	Отображение статистики готовности пропусков
Form10	Журнал событий безопасности	Четыре надписи, два поля ввода, таблица данных, две кнопки	Просмотр журнала событий за период
Form11	Тестирование хранимых процедур	Четыре надписи, три поля ввода, таблица данных, две кнопки	Выбор процедуры, ввод параметров и просмотр результата
Form12	Поиск кандидата	Две надписи, поле ввода, таблица данных, две кнопки	Поиск кандидатов по фрагменту фамилии
Form13	Трудовой договор	Семь надписей, шесть полей ввода, две кнопки	Просмотр трудовых договоров кандидата, подготовка, изменение и удаление договора
Form14	Согласие на обработку персональных данных	Пять надписей, три поля ввода, выпадающий список, две кнопки	Подготовка и изменение трудового договора, номер договора подсказывает приложение
Form15	Пропуск будущего сотрудника	Пять надписей, два поля ввода, два выпадающих списка, две кнопки	Регистрация согласия кандидата на обработку персональных данных
Form16	Трудовой договор	Семь надписей, шесть полей ввода, две кнопки	Оформление пропуска, уровень доступа и состояние готовности выбираются из списков

Для всех диалоговых окон установлены общие свойства. Свойство StartPosition имеет значение CenterScreen, поэтому окно открывается по

центру экрана. Свойство MaximizeBox имеет значение False, поскольку размер диалогового окна фиксирован. Свойство Text содержит наименование выполняемой операции.

Кнопкам назначены значения свойства DialogResult. Кнопка подтверждения операции имеет значение OK, кнопка отмены имеет значение No. Благодаря этому главная форма после закрытия окна анализирует результат вызова метода ShowDialog и выполняет изменение данных только при подтверждении операции пользователем. Такой прием исключает случайное выполнение операции добавления, изменения или удаления.

Полям ввода назначено значение свойства Modifiers, равное Public. Модификатор доступа public делает элементы управления видимыми из других модулей, поэтому главная форма считывает введенные значения после закрытия диалогового окна и заполняет поля окон данными выбранной строки таблицы. Для окон Form11 и Form12 дополнительно объявлены публичными таблицы данных и надписи, в которые выводится результат выполнения операции.

Окно Form7 используется для двух сценариев. При удалении кандидата в надпись выводится номер и фамилия кандидата, при удалении заявки заголовок окна изменяется и выводится номер заявки с наименованием подразделения. Такой подход уменьшает число однотипных форм в проекте.

Окно Form2 открывается перед появлением главной формы, поэтому проверка подлинности выполняется до предоставления доступа к данным. Поле пароля имеет настроенное свойство PasswordChar, поэтому вводимые символы отображаются в виде звездочек. Надпись label3 предназначена для вывода сообщения системы. Внешний вид окна приведен на рисунке 12.

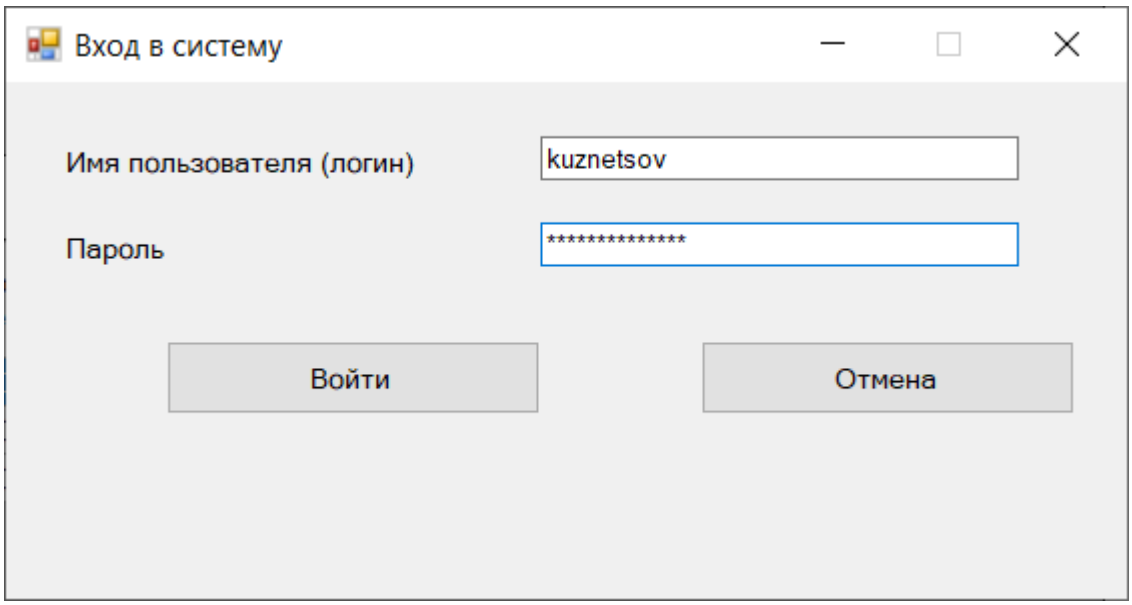


Рисунок 12 – Диалоговое окно входа в систему

Окна Form3 и Form4 содержат одинаковый состав полей, которые соответствуют атрибутам таблицы Kandidat. Внешний вид окон приведен на рисунках 13 и 14.

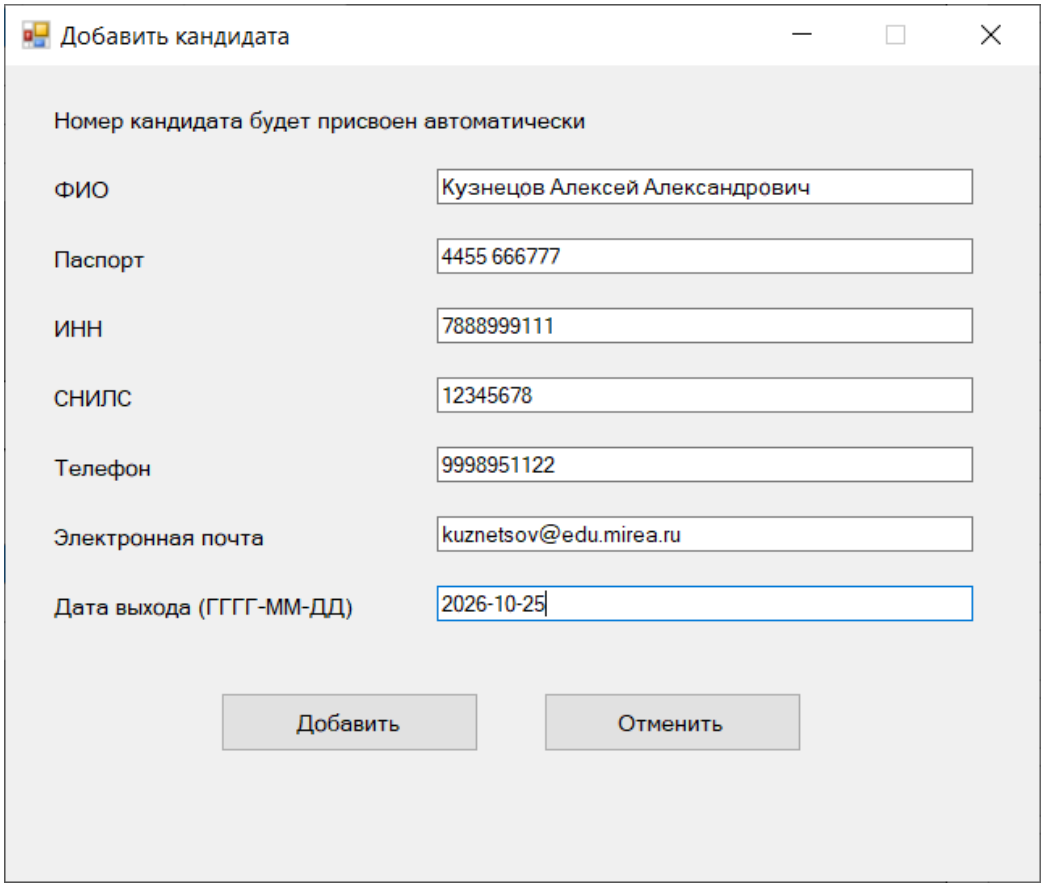


Рисунок 13 – Диалоговое окно добавления кандидата

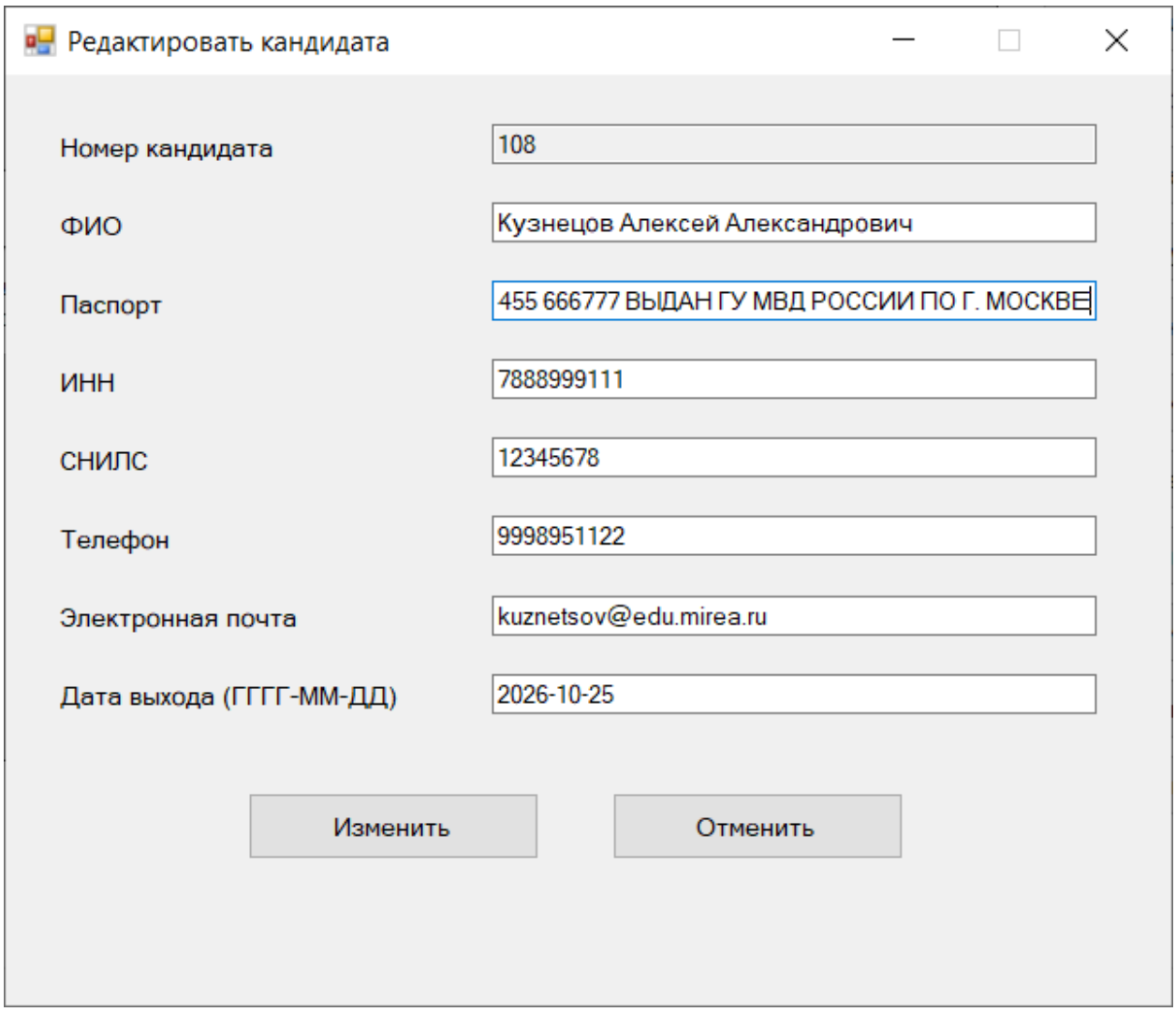


Рисунок 14 – Диалоговое окно редактирования данных кандидата

Поля окна Form4 заполняются главной формой полностью по данным хранимой процедуры usp_Kandidat_Dannye, поле номера кандидата объявлено доступным только для чтения. В окне Form6 доступным только для чтения объявлено поле номера заявки.

Окно Form13 предназначено для работы с трудовыми договорами выбранного кандидата. Договор готовится в приложении, а окно подсказывает свободный номер договора вызовом процедуры usp_Dogovor_Novyy_nomer: этот номер присваивается договору при подписании у генерального директора. Подготовку и изменение договоров выполняет специалист по делопроизводству, менеджеру по подбору доступен просмотр. Внешний вид окна договоров приведен на рисунке 15, окно подготовки договора приведено на рисунке 16.

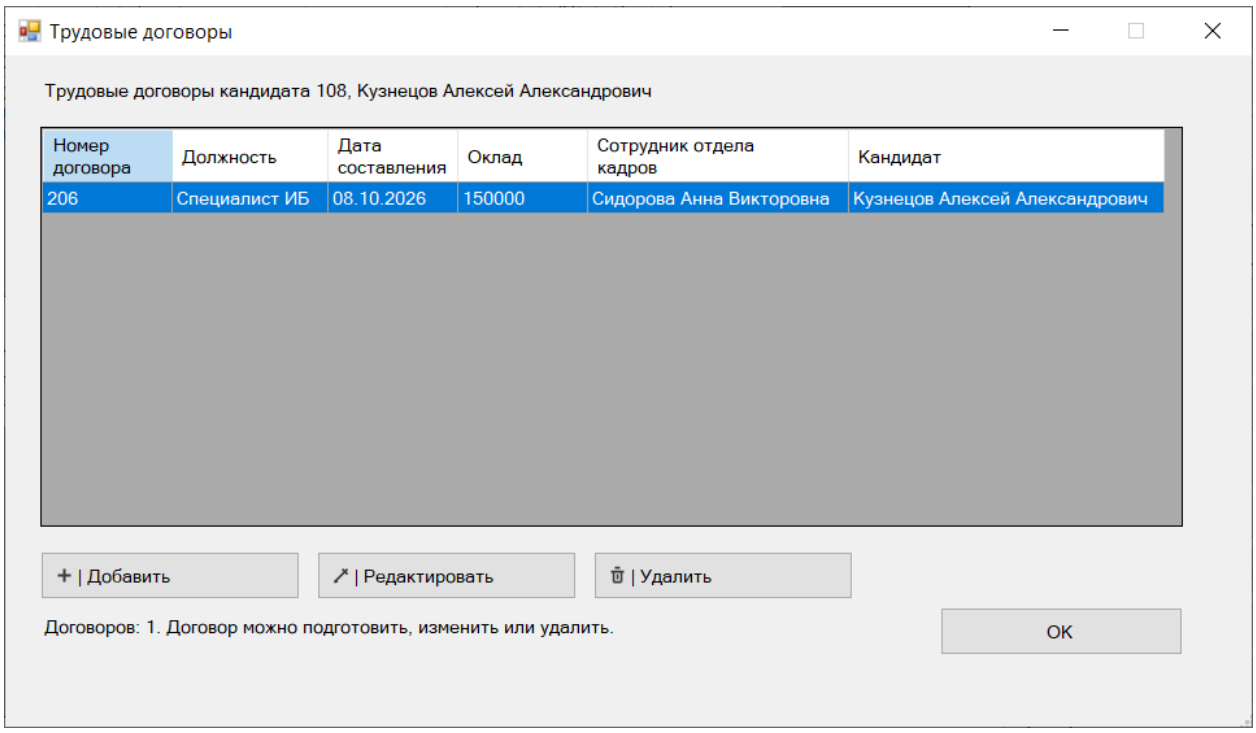


Рисунок 15 – Диалоговое окно трудовых договоров кандидата

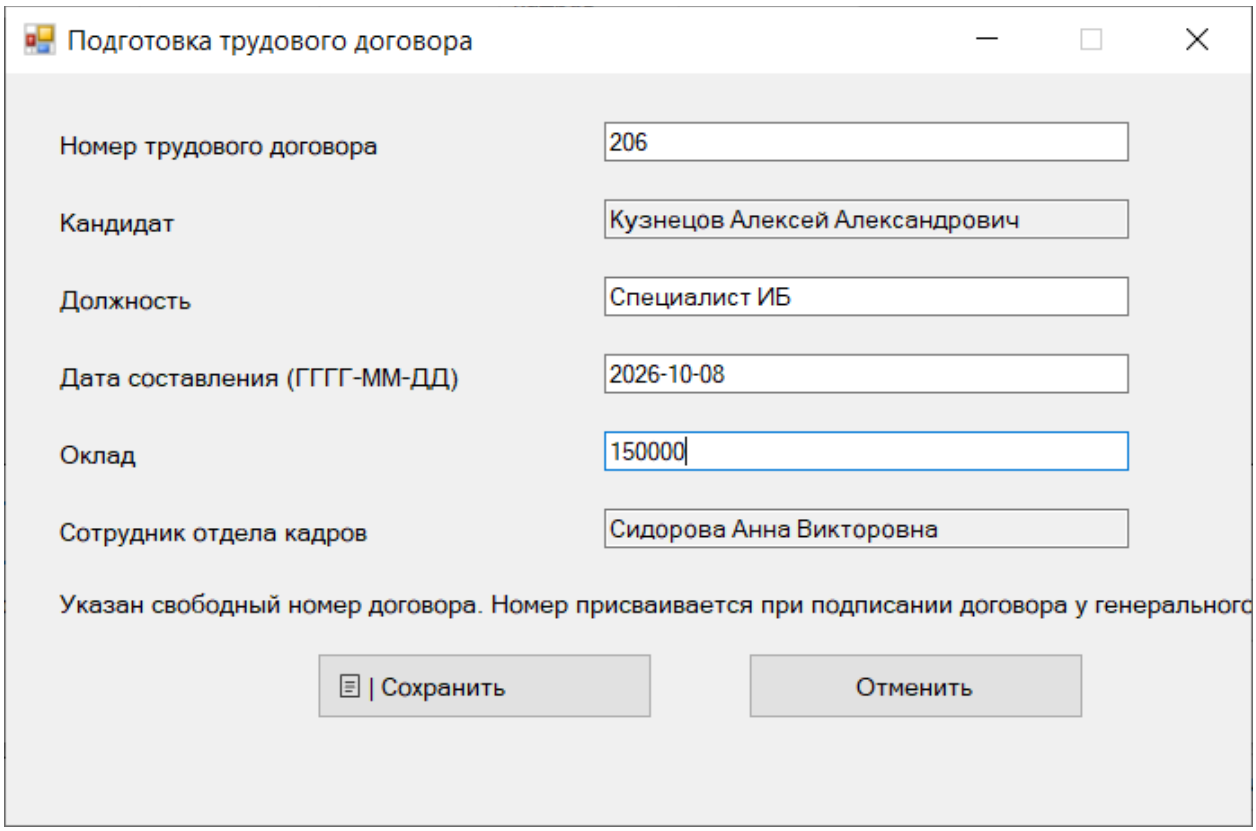


Рисунок 16 – Диалоговое окно подготовки трудового договора

Окно согласия на обработку персональных данных показывает текущее состояние согласия, а если согласие еще не оформлено, подставляет состояние «Ожидается» и текущую дату. Окно пропуска позволяет оформить пропуск и

указать уровень доступа; номер пропуска присваивается автоматически, если в поле указан ноль. Внешний вид окон приведен на рисунках 17 и 18.

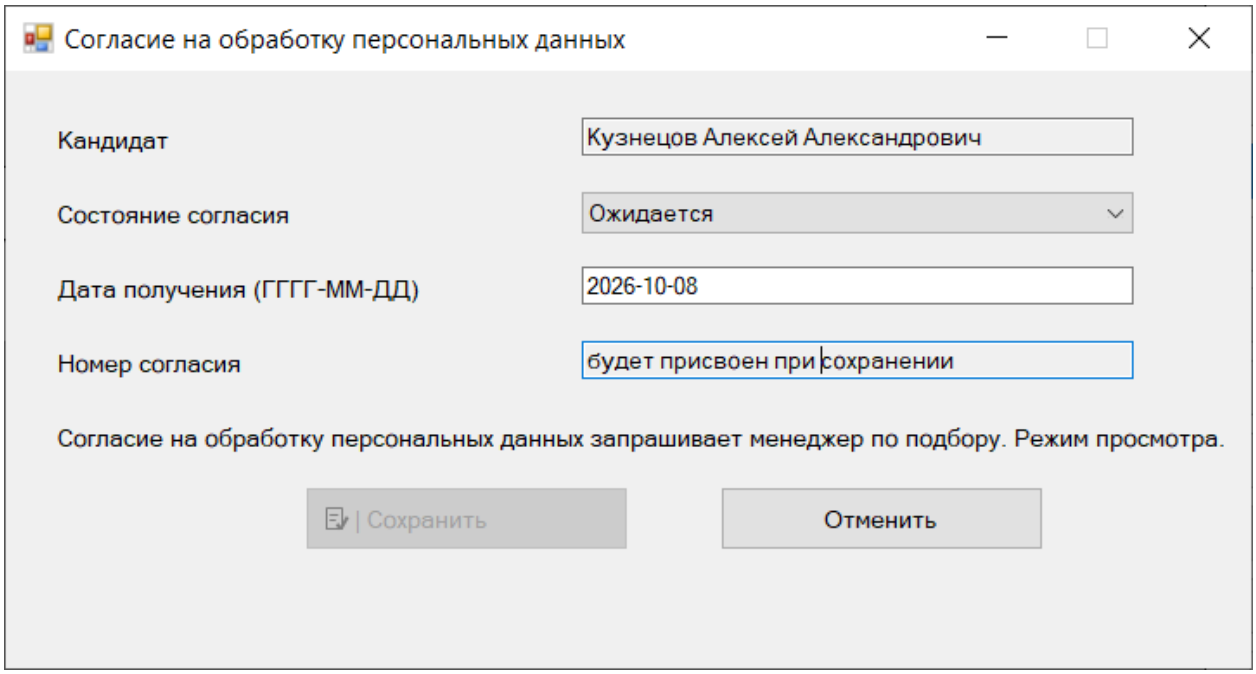


Рисунок 17 – Диалоговое окно согласия на обработку персональных данных

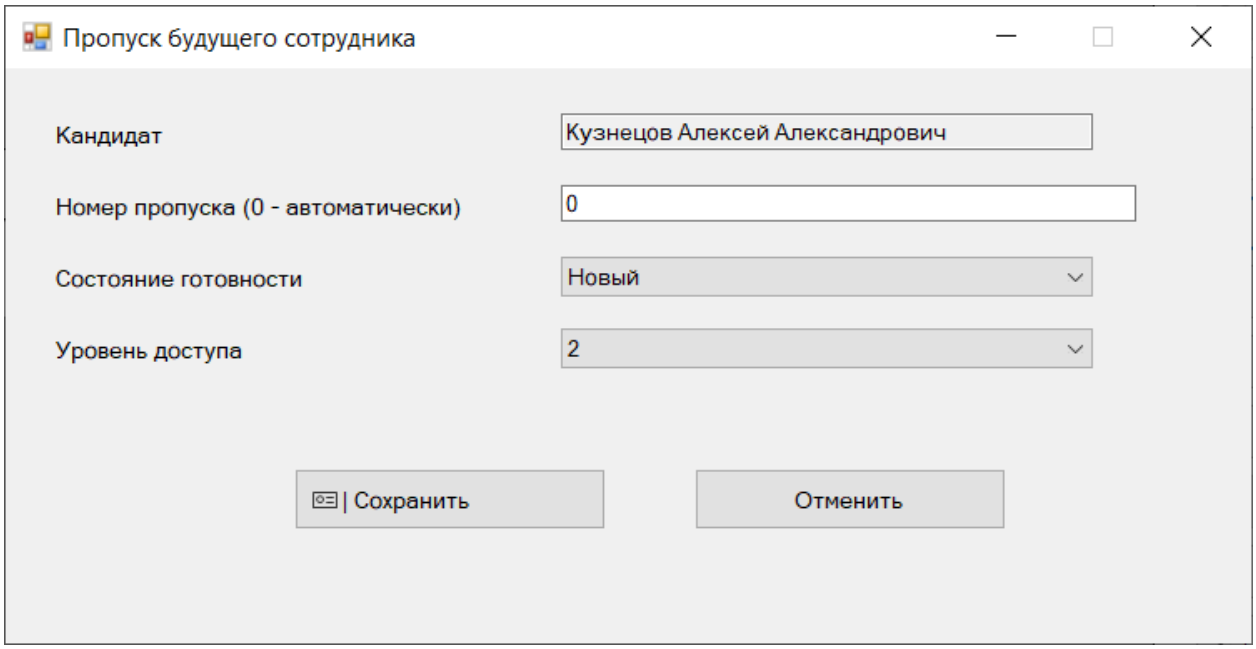


Рисунок 18 – Диалоговое окно пропуска будущего сотрудника

Окно Form5 предназначено для создания заявки, окно Form6 предназначено для изменения подразделения и состояния существующей заявки. Внешний вид окна добавления заявки приведен на рисунке 19.

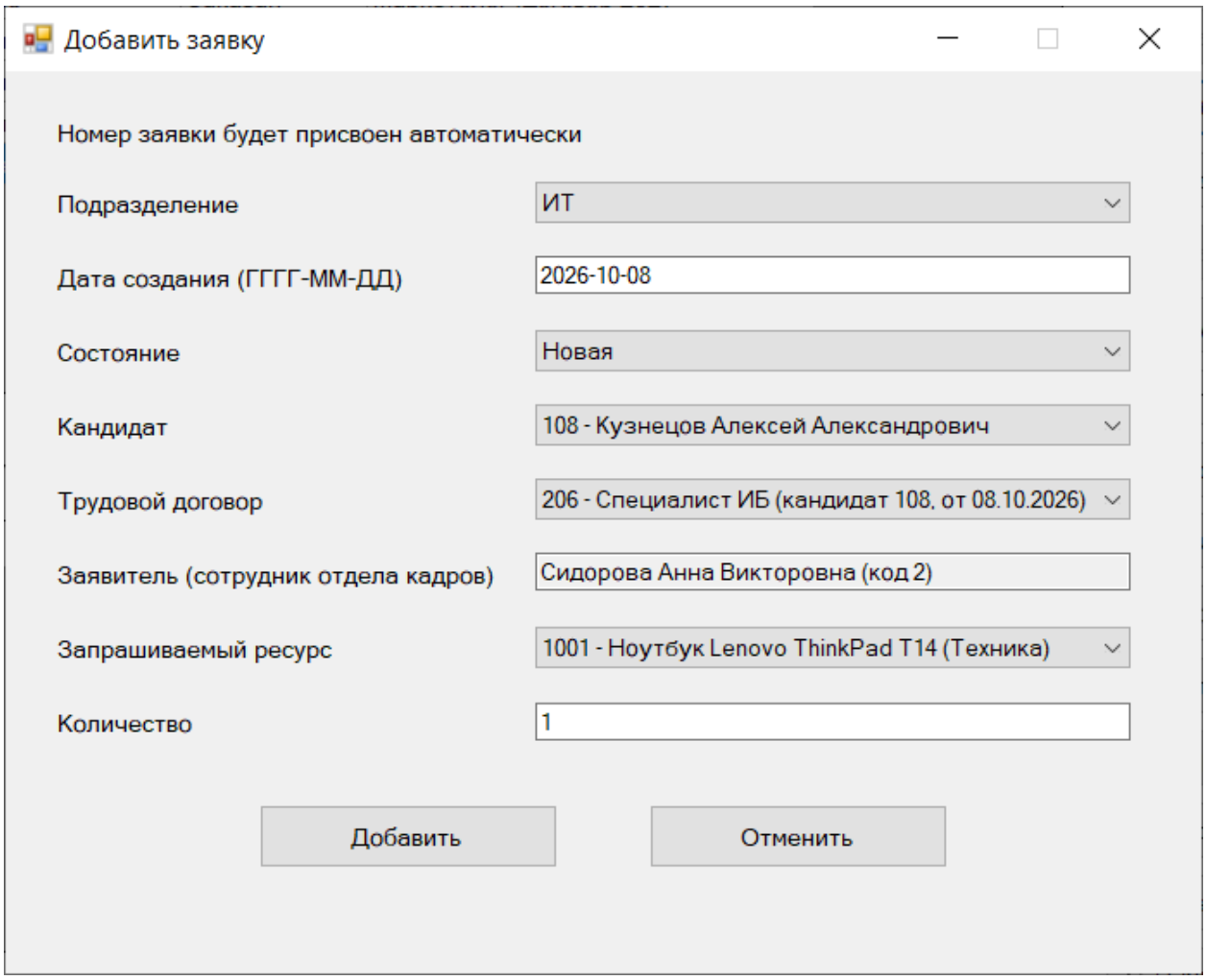


Рисунок 19 – Диалоговое окно добавления заявки

Окно подтверждения удаления Form7 выводит сведения об удаляемой записи и требует подтверждения операции. Внешний вид окна приведен на рисунке 20.

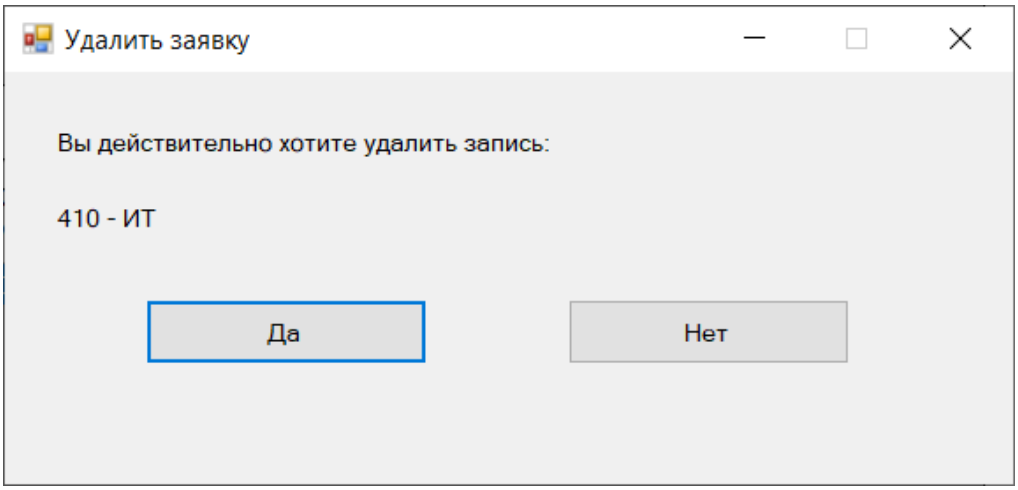


Рисунок 20 – Диалоговое окно подтверждения удаления

Окна Form8 и Form9 предназначены для отображения сводных данных. Источником данных служат хранимые процедуры без параметров, вызываемые главной формой при открытии окна. Внешний вид окна сводки по заявкам приведен на рисунке 21.

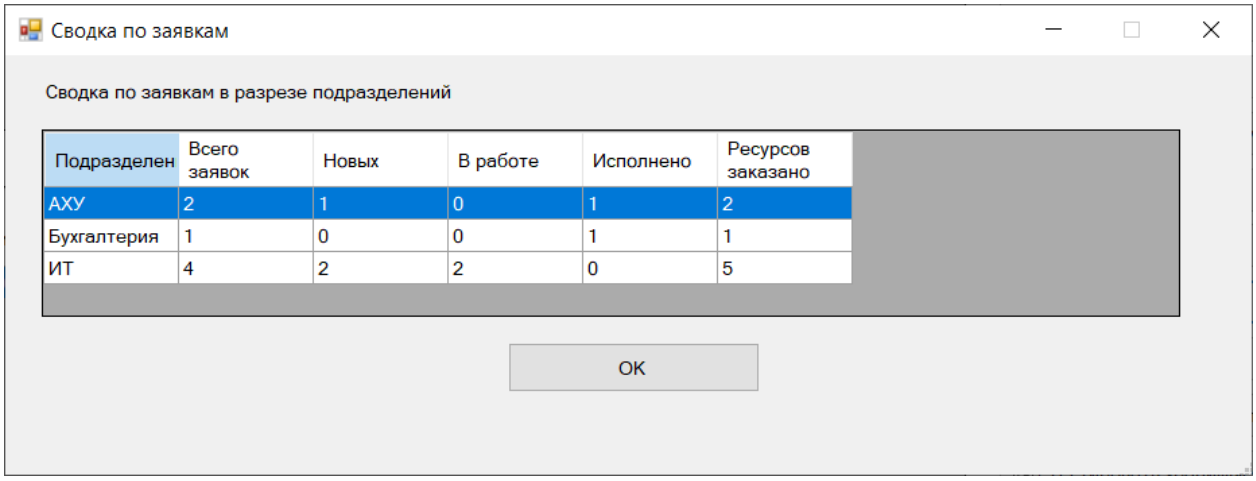


Рисунок 21 – Диалоговое окно сводки по заявкам

Окно Form10 отображает журнал событий безопасности. Если поля периода не заполнены, вызывается процедура без параметров, которая выводит последние сто событий. Если период задан, вызывается процедура с параметрами. Внешний вид окна приведен на рисунке 22.

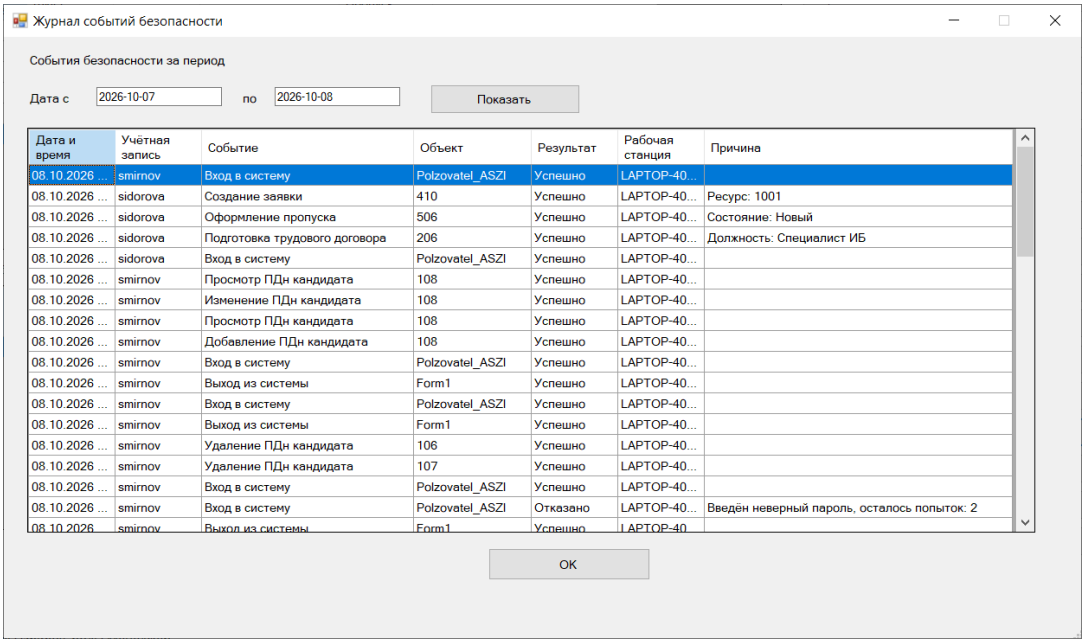


Рисунок 22 – Диалоговое окно журнала событий безопасности

Окно Form11 предназначено для проверки хранимых процедур. Имя процедуры выбирается из выпадающего списка, который заполняется по данным процедуры usp_Dostupnye_Procedury, поэтому в списке отображаются только процедуры, разрешенные роли текущего пользователя, а новые процедуры появляются автоматически. После выбора процедуры приложение запрашивает сведения о ее параметрах и создает для каждого параметра отдельное поле ввода. Внешний вид окна приведен на рисунке 23.

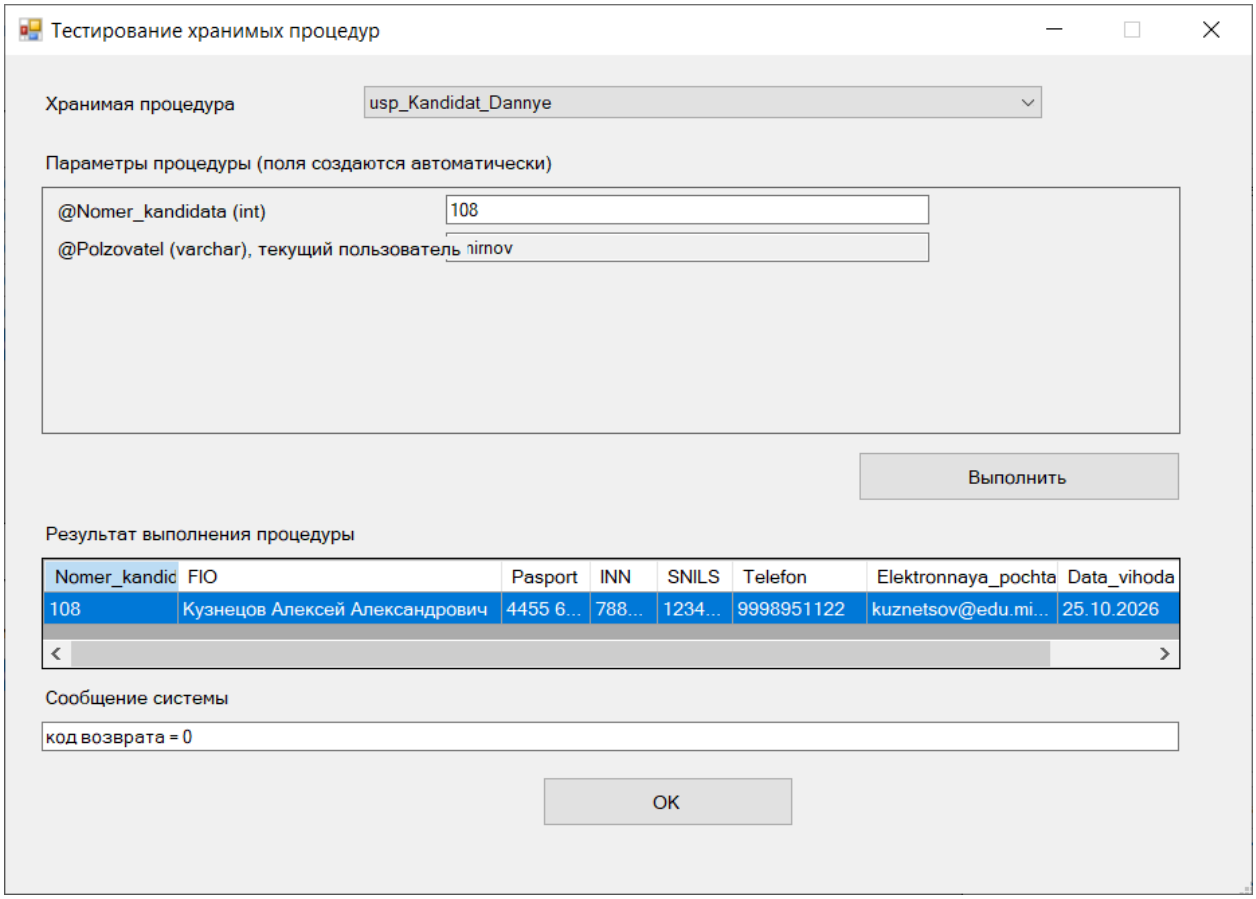


Рисунок 23 – Диалоговое окно тестирования хранимых процедур

Задание №6. Выполнить и описать процесс написания программного кода клиентского приложения АСЗИ своего варианта.

Ход выполнения задания

Программный код приложения разделен на три части. Модуль Program.cs содержит точку входа, проверку подлинности пользователя и запуск главной формы. Модуль RabotaSBazoy.cs содержит методы доступа к базе данных. Модули форм содержат обработчики событий и вспомогательные методы, относящиеся к интерфейсу.

В модулях форм подключено пространство имен System.Data.SqlClient, обеспечивающее работу с объектами доступа к данным, и пространство имен System.Data, в котором находятся классы DataTable, DataSet, DataView и перечисление CommandType. Порядок директив using и объявление класса формы приведены ниже.

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace ASZI_OK
{
   public partial class Form1 : Form
   {
      // Учётная запись пользователя, прошедшего вход в систему
      public static string Tekushchiy_polzovatel = "";

      // Роль пользователя, полученная при входе в систему
      public static string Tekushchaya_rol = "";
```

Состав обработчиков и методов главной формы приведен в таблице 14.

Таблица 14 – Обработчики событий и методы главной формы

Обработчик или метод	Событие либо назначение	Выполняемые действия
Form1_Shown	Первое отображение формы	Заполнение таблиц, вывод сведений о пользователе, применение ограничений по роли, вывод сообщения о подключении
Form1_FormClosing	Закрытие формы	Регистрация события безопасности о завершении сеанса работы
FillKandidaty	Вспомогательный метод	Вызов процедуры usp_Kandidaty_Spisok и привязка результата к таблице dataGridView1
FillZayavki	Вспомогательный метод	Вызов процедуры usp_Zayavki_Kandidata с номером выбранного кандидата и привязка результата к таблице dataGridView2
OgranicheniyaPoRoli	Вспомогательный метод	Закрытие команд изменения и удаления данных кандидатов и работы с журналом событий для роли «Специалист»
NomerVybrannogoKandidata	Вспомогательный метод	Определение номера кандидата, выбранного в таблице dataGridView1
ZapolnitFormuRedaktirovaniya	Вспомогательный метод	Заполнение всех полей окна редактирования по данным процедуры usp_Kandidat_Dannye
SformirovatParametryKandidata	Вспомогательный метод	Проверка значений полей окна кандидата и подготовка параметров вызова процедуры
SformirovatParametryZayavki	Вспомогательный метод	Проверка значений полей окна заявки и подготовка параметров вызова процедуры
ZapolnitFormuZayavki	Вспомогательный метод	Заполнение справочников окна заявки: дата, сведения о сотруднике, списки кандидатов и ресурсов
VypolnitProceduru	Вспомогательный метод	Вызов процедуры изменения данных с обработкой исключительной ситуации SqlException
button1_Click	Кнопка «Добавить кандидата»	Открытие окна Form3, считывание введенных значений, вызов процедуры usp_Kandidat_Dobavit
button2_Click	Кнопка «Редактировать кандидата»	Заполнение всех полей окна Form4 по данным процедуры usp_Kandidat_Dannye, вызов процедуры usp_Kandidat_Redaktirovat
button3_Click	Кнопка «Удалить кандидата»	Проверка связанных записей процедурой usp_Kandidat_Svyazi с выводом перечня, подтверждение операции в окне Form7, вызов процедуры usp_Kandidat_Udalit
button4_Click	Кнопка «Добавить заявку»	Открытие окна Form5, вызов процедуры usp_Zayavka_Sozdat
button5_Click	Кнопка «Редактировать заявку»	Заполнение окна Form6, вызов процедуры usp_Zayavka_Redaktirovat
button6_Click	Кнопка «Удалить заявку»	Подтверждение операции, вызов процедуры usp_Zayavka_Udalit
NastroitIkonki	Вспомогательный метод	Назначение кнопкам и командам меню стилизованных иконок, которые рисует класс Ikony
OformitKnopku	Вспомогательный метод	Формирование подписи кнопки по образцу «иконка | название команды»
button7_Click	Кнопка «Поиск кандидата»	Открытие окна Form13 для кандидата, выбранного в таблице, обновление таблиц после закрытия окна
button8_Click	Кнопка «Сводка по заявкам»	Открытие окна Form15 с текущими сведениями о согласии кандидата, обновление таблиц после закрытия окна
button9_Click	Кнопка «Готовность пропусков»	Открытие окна Form16 с текущими сведениями о пропуске кандидата, обновление таблиц после закрытия окна
button10_Click	Кнопка «Журнал событий безопасности»	Открытие окна Form10

Продолжение таблицы 14

Обработчик или метод	Событие либо назначение	Выполняемые действия
button11_Click	Кнопка «Тестировать хранимые процедуры»	Вызов процедуры usp_Svodka_Zayavok, открытие окна Form8
button12_Click	Кнопка «Сменить пользователя»	Вызов процедуры usp_Propuski_Gotovnost, открытие окна Form9
button13_Click	Кнопка «Журнал событий безопасности»	Открытие окна Form10
button14_Click	Кнопка «Тестировать хранимые процедуры»	Открытие окна Form11
button15_Click	Кнопка «Сменить пользователя»	Повторный вход в систему без перезапуска приложения
button16_Click	Кнопка «Обновить данные»	Повторное чтение списка кандидатов и заявок выбранного кандидата
ObnovitDannye	Вспомогательный метод	Обновление обеих таблиц главной формы с восстановлением выбранной строки. Вызывается командой «Обновить данные», пунктом меню и автоматически после закрытия окон работы с договорами, согласиями и пропусками
dataGridView1_CellEnter	Переход на другую строку таблицы кандидатов	Обновление таблицы заявок выбранного кандидата
MenuOProgramme_Click	Команда меню «О программе»	Вывод сведений о приложении
comboBox1_SelectedIndexChanged	Выбор процедуры в окне Form11	Создание полей ввода для параметров выбранной процедуры по данным usp_Procedura_Parametry
UstanovitParametry	Вспомогательный метод окна Form11	Создание надписей и полей ввода для параметров процедуры с пометкой выходных и необязательных параметров

Все обращения к базе данных выполняются вызовом хранимых процедур с параметрами. Значения параметров передаются объектами SqlParameter, поэтому текст команды формируется на стороне сервера и не зависит от содержимого полей формы. Такой порядок исключает возможность изменения логики запроса данными, введенными пользователем, то есть защищает приложение от внедрения SQL-инструкций. Пример вызова процедуры добавления кандидата приведен ниже.

```
// Кнопка «Добавить кандидата»
private void button1_Click(object sender, EventArgs e)
{
    Form3 f = new Form3();

    if (f.ShowDialog() == DialogResult.OK)
    {
        SqlParameter[] p =
        {
            new SqlParameter("@Nomer_kandidata", SqlDbType.Int) { Value =
```

```
Convert.ToInt32(f.textBox1.Text) },
                    new SqlParameter("@FIO", SqlDbType.VarChar, 40) { Value = f.textBox2.Text },
                    new SqlParameter("@Pasport", SqlDbType.VarChar, 150) { Value = f.textBox3.Text },
                    new SqlParameter("@INN", SqlDbType.BigInt) { Value = Convert.ToInt64(f.textBox4.Text) },
                    new SqlParameter("@SNILS", SqlDbType.BigInt) { Value = Convert.ToInt64(f.textBox5.Text)
},
                    new SqlParameter("@Telefon", SqlDbType.BigInt) { Value =
Convert.ToInt64(f.textBox6.Text) },
                    new SqlParameter("@Elektronnaya_pochta", SqlDbType.VarChar, 40) { Value =
f.textBox7.Text },
                    new SqlParameter("@Data_vihoda", SqlDbType.DateTime) { Value =
Convert.ToDateTime(f.textBox8.Text) },
                    new SqlParameter("@Polzovatel", SqlDbType.VarChar, 40) { Value = Tekushchiy_polzovatel
}
                };

                if (VypolnitProceduru("usp_Kandidat_Dobavit", p))
                  FillKandidaty();
              }
            }
```

Параметр @Polzovatel передается во все процедуры изменения данных. Значение этого параметра попадает в журнал событий безопасности, поэтому по каждой записи журнала можно установить, какая учетная запись выполнила операцию.

Изменение данных выполняется методом VypolnitProceduru, который перехватывает исключительную ситуацию класса SqlException. Сообщения, сформированные процедурами инструкцией THROW, выводятся пользователю в окне сообщения. Операция при этом не выполняется, а сведения об ошибке остаются в журнале событий безопасности. Такой порядок обеспечивает понятность работы приложения для пользователя и сохранение сведений о событиях для администратора АСЗИ.

```
// Вызов хранимой процедуры изменения данных с обработкой ошибок
private bool VypolnitProceduru(string imya_procedury, SqlParameter[] p)
{
  try
```

```
        {
            RabotaSBazoy.MyExecuteProcedureWithReturn(imya_procedury, p);
            MessageBox.Show("Операция выполнена успешно.");
            return true;
        }
        catch (SqlException exep)
        {
            MessageBox.Show("Операция не выполнена. " + exep.Message);
            return false;
        }
    }
```

Работа с трудовыми договорами, согласиями и пропусками выполнена теми же приемами: параметризованные вызовы хранимых процедур, обработка исключительных ситуаций классом SqlException и повторное открытие окна ввода при ошибке проверки значений.

Проверка вводимых данных выполняется на двух уровнях. На уровне интерфейса проверяются заполнение обязательных полей и возможность преобразования введенного текста к требуемому типу данных. На уровне базы данных проверяются обязательность полей, допустимые значения, форматы и ссылочная целостность. Двойная проверка позволяет исключить как случайные ошибки пользователя, так и попытки обращения к базе данных в обход приложения.

Преобразование значений выполняется методами TryParse, которые не создают исключительную ситуацию при неверном значении, а возвращают признак успешного преобразования. Если поле заполнено неверно, выводится сообщение с указанием поля и требуемого формата, а окно ввода открывается повторно с сохранением введенных значений. Такой порядок исключает появление необработанного исключения FormatException, которое возникает при попытке преобразования пустой строки методами Convert.

```
    private SqlParameter[] SformirovatParametryKandidata(string nomer, string fio, string pasport,
        string inn, string snils, string telefon, string pochta, string data, string polzovatel)
    {
        int nomer_kandidata;
```

```
            long inn_chislo = 0, snils_chislo = 0, telefon_chislo = 0;
            DateTime data_vihoda;

            if (!int.TryParse(nomer.Trim(), out nomer_kandidata) || nomer_kandidata <= 0)
            {
                MessageBox.Show("Поле «Номер кандидата» заполнено неверно. " +
                                "Укажите целое положительное число.");
                return null;
            }

            if (fio.Trim() == "")
            {
                MessageBox.Show("Не заполнено поле «ФИО».");
                return null;
            }

            if (!DateTime.TryParse(data.Trim(), out data_vihoda))
            {
                MessageBox.Show("Поле «Дата выхода» заполнено неверно. Укажите дату, например 2026-
05-12.");
                return null;
            }

            SqlParameter[] p =
            {
                new SqlParameter("@Nomer_kandidata", SqlDbType.Int) { Value = nomer_kandidata },
                new SqlParameter("@FIO", SqlDbType.VarChar, 40) { Value = fio.Trim() },
                new SqlParameter("@INN", SqlDbType.BigInt)
                {
                    Value = (inn.Trim() == "") ? (object)DBNull.Value : inn_chislo
                },
                new SqlParameter("@Data_vihoda", SqlDbType.DateTime) { Value = data_vihoda },
                new SqlParameter("@Polzovatel", SqlDbType.VarChar, 40) { Value = polzovatel }
            };

            return p;
        }
```

Необязательные поля, оставленные пустыми, передаются в базу данных как NULL. Значения, введенные неверно, не доходят до сервера, поэтому пользователь получает понятное сообщение, а приложение продолжает работу без прерывания.

Задание №7. Выполнить и описать процесс вызова хранимых процедур из клиентского приложения АСЗИ своего варианта.

Ход выполнения задания

Вызов хранимой процедуры из приложения выполняется объектом SqlCommand, для которого задано имя процедуры и свойство CommandType, равное значению StoredProcedure. Значения передаются объектами SqlParameter, а результат выполнения получается тремя способами, перечисленными в таблице 15.

Таблица 15 – Способы получения результатов вызова хранимых процедур

Способ получения результата	Объекты библиотеки	Применение в приложении
Набор данных	SqlDataAdapter и DataTable	Заполнение таблиц dataGridView1 и dataGridView2, вывод сводок и журнала событий
Выходные параметры	SqlParameter с направлением ParameterDirection.Output	Получение роли пользователя и текста сообщения от процедуры usp_Vhod_v_sistemu
Код возврата	SqlParameter с направлением ParameterDirection.ReturnValue	Определение результата проверки подлинности и результата изменения данных

Соответствие команд интерфейса и вызываемых процедур приведено в таблице 16.

Таблица 16 – Соответствие команд интерфейса и вызываемых хранимых процедур

Команда интерфейса	Вызываемая процедура	Способ вызова
Запуск приложения	usp_Vhod_v_sistemu	Выходные параметры и код возврата
Загрузка главной формы	usp_Kandidaty_Spisok	Набор данных, процедура без параметров
Открыть окно добавления заявки	usp_Kandidaty_Vybor, usp_Resursy_Spisok	Набор данных, процедуры без параметров, заполнение выпадающих списков
Выбор кандидата в окне заявки	usp_Dogovory_Spisok	Набор данных, входной параметр, обновление списка договоров

Продолжение таблицы 16

Команда интерфейса	Вызываемая процедура	Способ вызова
Выбор строки таблицы кандидатов	usp_Zayavki_Kandidata	Набор данных, входной параметр
Открыть окно редактирования кандидата	usp_Kandidat_Dannye	Набор данных, входные параметры
Добавить или изменить кандидата	usp_Kandidat_Dobavit, usp_Kandidat_Redaktirovat	Код возврата, входные параметры
Удалить кандидата	ВЫДЕЛЕНО. usp_Kandidat_Svyazi, usp_Kandidat_Udalit	Набор данных с перечнем связанных записей и код возврата
Добавить или изменить заявку	usp_Zayavka_Sozdat, usp_Zayavka_Redaktirovat	Код возврата, входные параметры, выходной параметр с присвоенным номером
Удалить заявку	usp_Zayavka_Udalit	Код возврата, входные параметры
Поиск кандидата	usp_Poisk_Kandidata	Набор данных, входные параметры
Сводка по заявкам	usp_Svodka_Zayavok	Набор данных, процедура без параметров
Готовность пропусков	usp_Propuski_Gotovnost	Набор данных, процедура без параметров
Журнал событий безопасности	usp_Zhurnal_Bezopasnosti, usp_Zhurnal_za_period	Набор данных, процедура без параметров и процедура с параметрами
Все изменения данных	usp_Zapis_v_zhurnal	Регистрация события, вызывается процедурами и приложением
Окно тестирования процедур	usp_Procedura_Parametry	Набор данных, входные параметры, сведения о параметрах выбранной процедуры
Открыть окно трудовых договоров	usp_Dogovory_Kandidata, usp_Dogovor_Novyy_nomer	Набор данных, процедура без параметров
Подготовить или изменить договор	usp_Dogovor_Dobavit, usp_Dogovor_Redaktirovat, usp_Dogovor_Dannye	Код возврата, входные и выходные параметры
Удалить договор	usp_Dogovor_Udalit	Код возврата, входные параметры
Оформить согласие на обработку ПДн	usp_Soglasie_Kandidata, usp_Soglasie_Sohranit	Набор данных и код возврата
Оформить пропуск	usp_Propusk_Kandidata, usp_Propusk_Sohranit	Набор данных и код возврата
Заполнить список процедур по роли	usp_Dostupnye_Procedury	Набор данных, входной параметр
Обновить данные на главной форме	usp_Kandidaty_Spisok, usp_Zayavki_Kandidata	Наборы данных, повторное чтение таблиц по команде пользователя и после закрытия окон

Процедура usp_Vhod_v_sistemu вызывается из окна входа в систему. Для получения роли пользователя и текста сообщения объявлены два выходных параметра, для получения результата проверки объявлен параметр кода возврата. Фрагмент кода приведен ниже.

```
SqlCommand cmd = new SqlCommand("usp_Vhod_v_sistemu", cn);
cmd.CommandType = CommandType.StoredProcedure;

cmd.Parameters.Add("@Login", SqlDbType.VarChar, 40).Value = textBox1.Text.Trim();
cmd.Parameters.Add("@Parol", SqlDbType.VarChar, 40).Value = textBox2.Text;
```

```
SqlParameter pRol = cmd.Parameters.Add("@Rol", SqlDbType.VarChar, 20);
pRol.Direction = ParameterDirection.Output;

SqlParameter pSoobshchenie = cmd.Parameters.Add("@Soobshchenie", SqlDbType.VarChar,
150);
pSoobshchenie.Direction = ParameterDirection.Output;

SqlParameter pKod = cmd.Parameters.Add("@Kod", SqlDbType.Int);
pKod.Direction = ParameterDirection.ReturnValue;

cmd.ExecuteNonQuery();

if (Convert.ToInt32(pKod.Value) == 0)
{
   Form1.Tekushchiy_polzovatel = textBox1.Text.Trim();
   Form1.Tekushchaya_rol = Convert.ToString(pRol.Value);
   this.DialogResult = DialogResult.OK;
}
```

Окно входа закрывается кодом OK только в том случае, когда процедура вернула нулевое значение. В остальных случаях выводится сообщение системы, поле пароля очищается, а запуск приложения не выполняется. Проверка подлинности выполняется до открытия главной формы, что соответствует требованию о недопустимости доступа к защищаемой информации без прохождения идентификации и аутентификации.

Окно Form11 предназначено для проверки процедур без изменения программного кода. Пользователь указывает имя процедуры и строку параметров, а приложение формирует команду, выполняет ее и выводит результат. Параметры процедуры usp_Vhod_v_sistemu распознаются как выходные, поскольку их значения формируются внутри процедуры. После выполнения в поле сообщения системы выводятся значения выходных параметров и код возврата. Результат вызова процедуры приведен на рисунке 24.

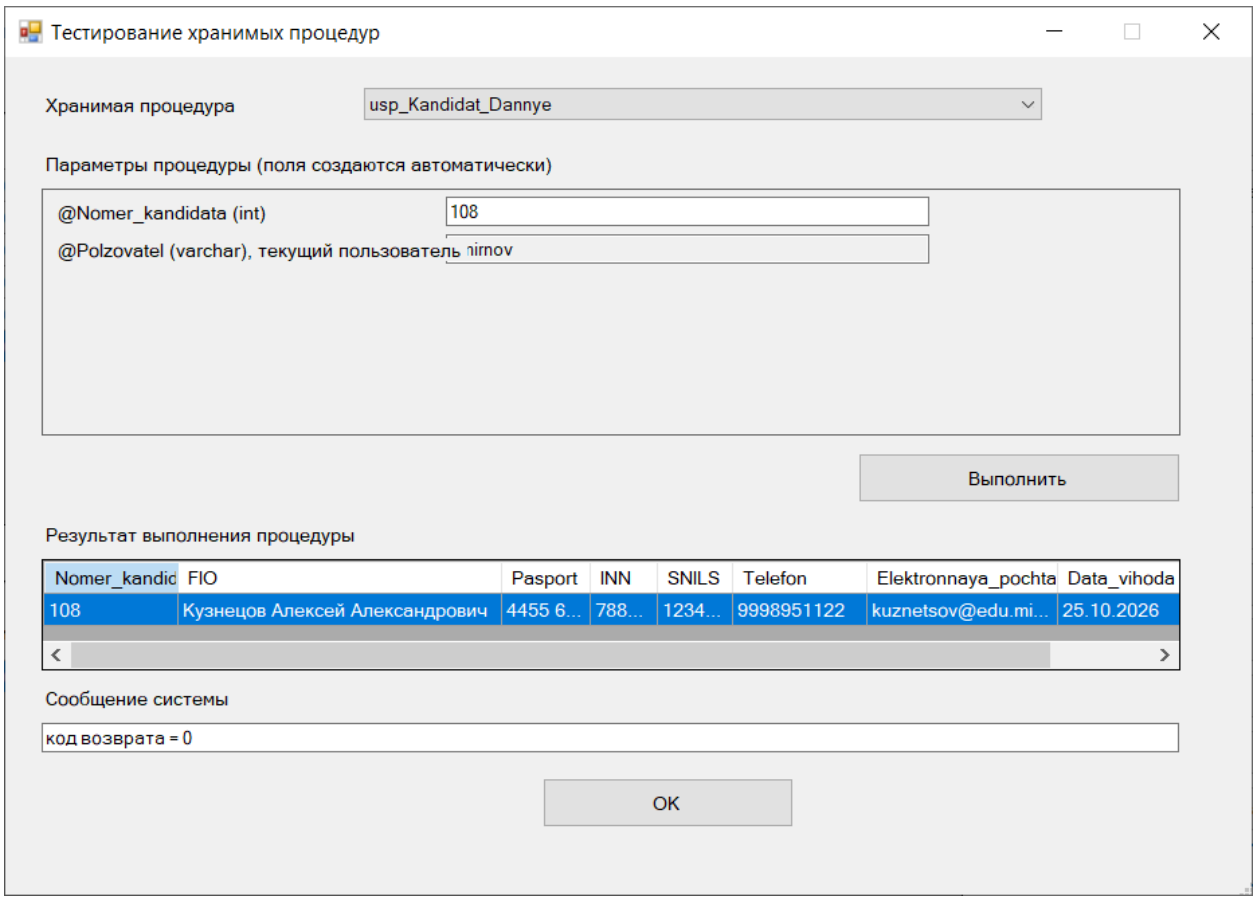


Рисунок 24 – Результат вызова хранимой процедуры из приложения

Каждый вызов процедуры из окна тестирования регистрируется в журнале событий безопасности с указанием имени процедуры и результата выполнения. Благодаря этому использование окна тестирования не создает возможности незаметного обращения к данным.

Задание №8. Выполнить и описать процесс создания инсталлятора клиентского приложения АСЗИ своего варианта.

Ход выполнения задания

Перед созданием установочного пакета решение собрано в конфигурации Release командой «Сборка», «Собрать решение». В папке “bin\Release” размещены исполняемый файл приложения, файл конфигурации и вспомогательные файлы библиотеки. Файл базы данных Kuznetsov_marketing.mdf подготовлен к включению в пакет, для этого перед копированием база данных отключена от экземпляра СУБД командой Detach.

В качестве средства сборки использовано расширение Microsoft Visual Studio Installer Projects, которое устанавливается как дополнение к среде Microsoft Visual Studio. Состав установочного пакета включает:

Исполняемые файлы ASZI_OK.exe, ASZI_OK.exe.config, библиотеки .NET Framework 4.7.2 или выше; файл БД потребуется в случае локальной работы без сервера.

На рисунке 25 продемонстрировано официальное расширение от Microsoft для создания инсталляторов.

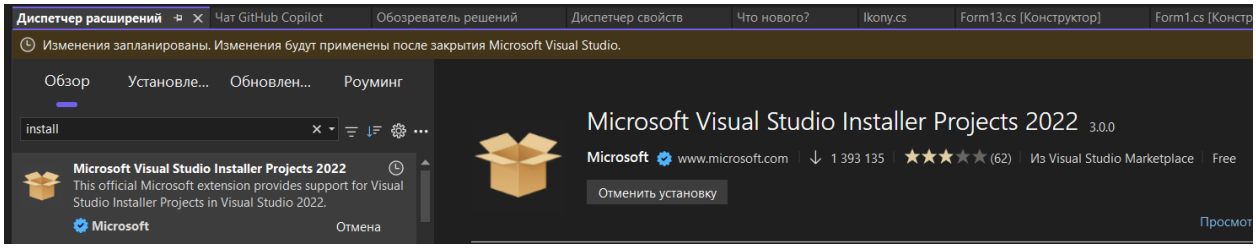


Рисунок 25 – Microsoft Visual Studio Installer Projects

Далее закрываем Visual Studio и устанавливаем расширение – рисунки 26-27.

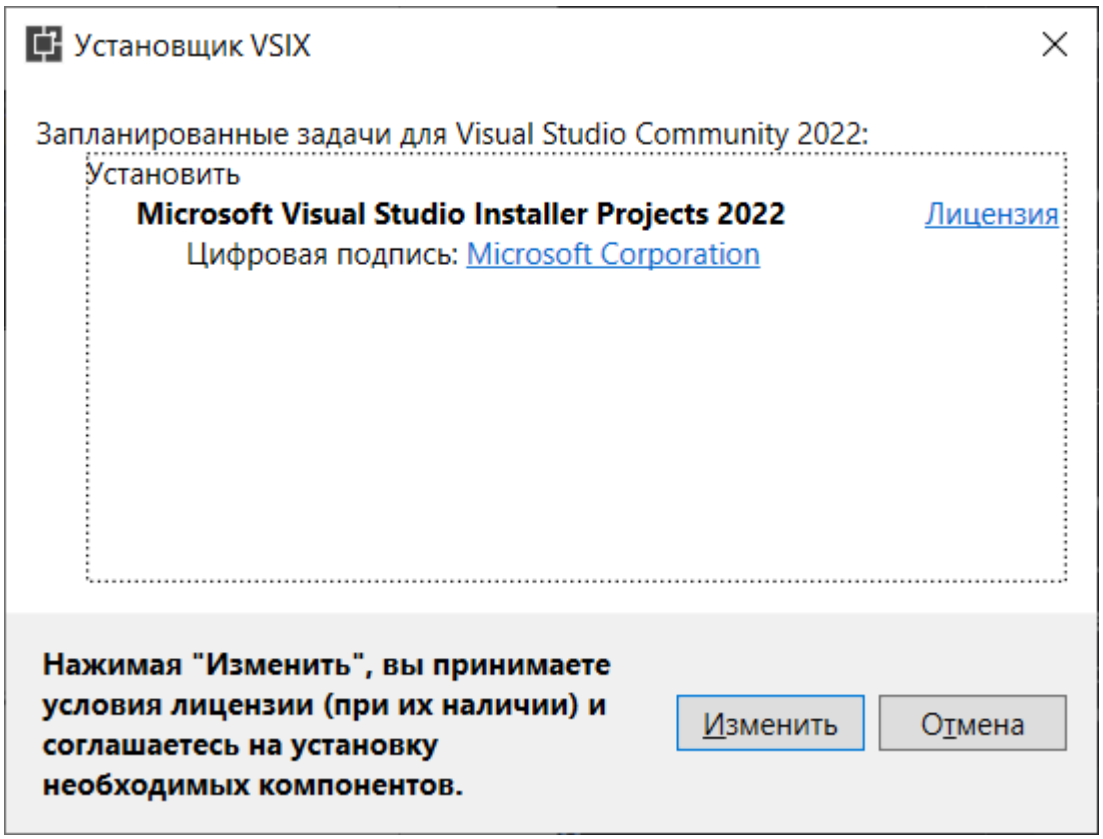


Рисунок 26 – Список устанавливаемых компонентов

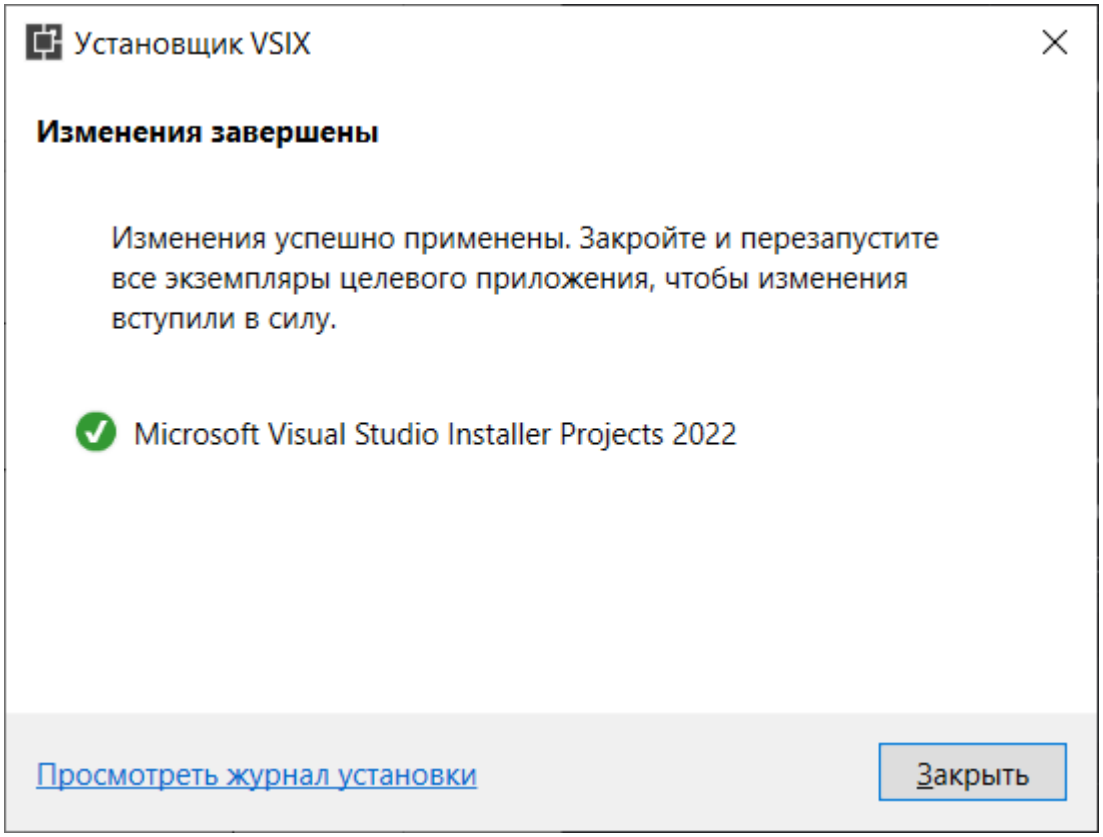


Рисунок 27 – Расширение успешно установлено

Далее в обозревателе решений выбираем проект ASZI_OK, нажимаем

правой кнопкой мыши – «Добавить» – «Создать проект…» – рисунок 28.

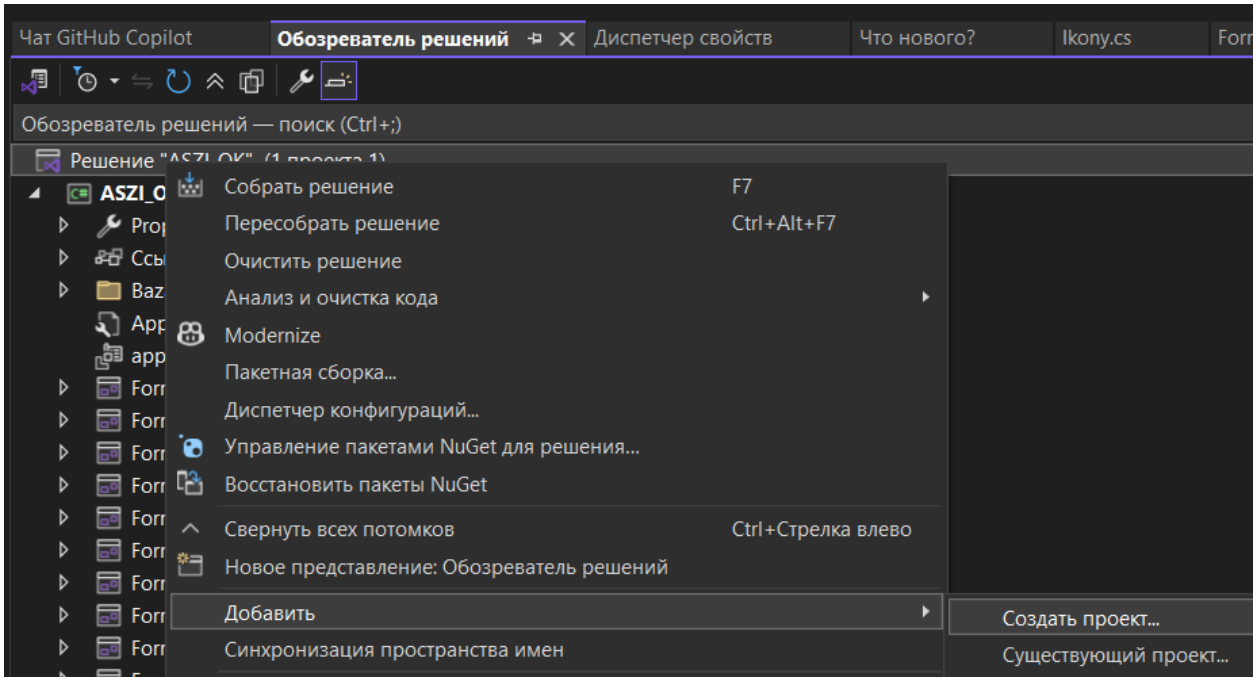


Рисунок 28 – Добавляем проект к текущему

В открывшемся окне выбираем «Setup Project», к нашему решению добавляется новый проект, в котором мы настраиваем файлы, которые будут включены в установщик. «Основные выходные файлы» отвечают за сам исполняемый файл расширения .exe, для добавления БД создадим новую папку в «Application Folder» и включим нужные файлы. Процесс продемонстрирован на рисунках 29-32.

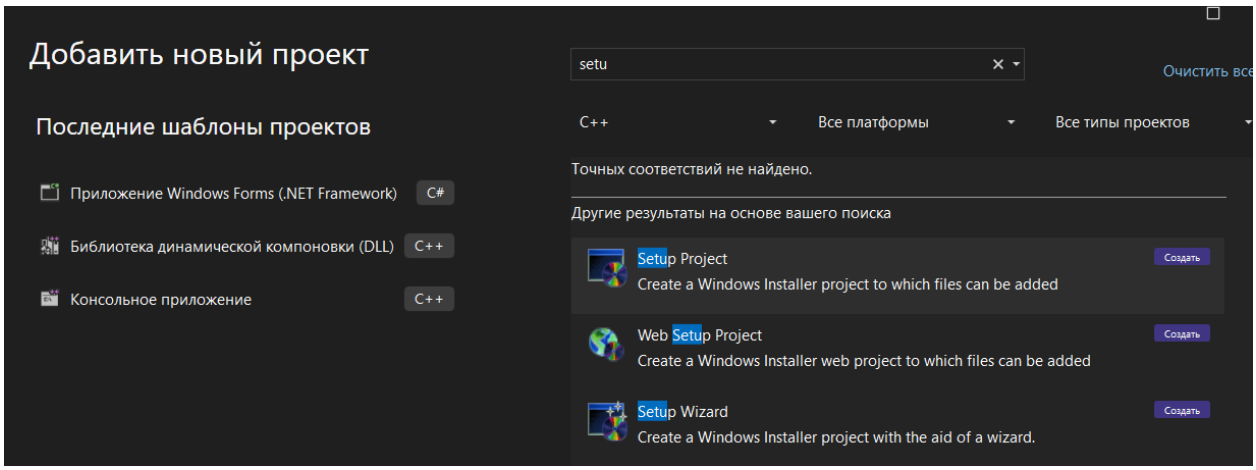


Рисунок 29 – Создаем проект типа «Setup Project»

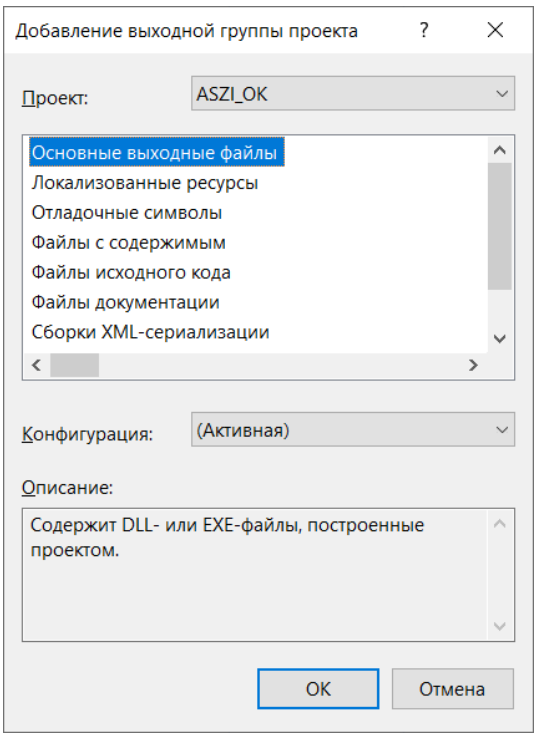


Рисунок 30 – Добавляем основные выходные файлы

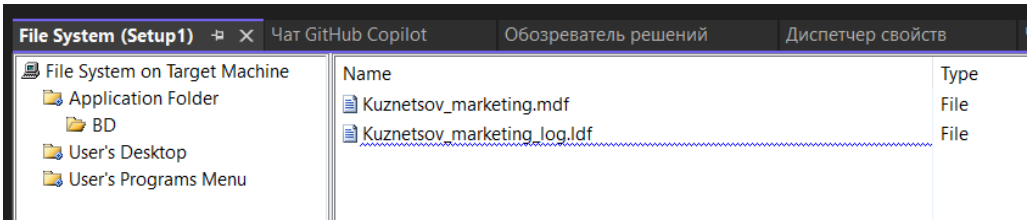


Рисунок 31 – Создаем папку и добавляем в нее файлы БД

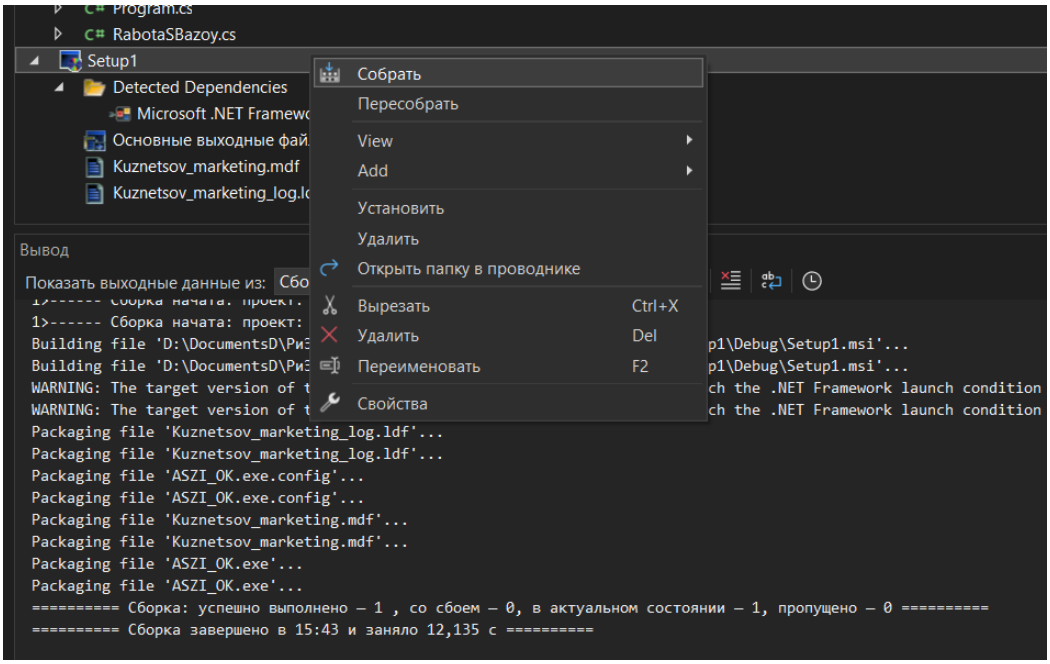


Рисунок 32 – Осуществляем сборку проекта инсталлятора

После сборки получаем на выходе приложения и пакет установщика. После запуска и выбора пути установки получаем готовую рабочую область с программой и БД. Процесс продемонстрирован рисунками 33-35.

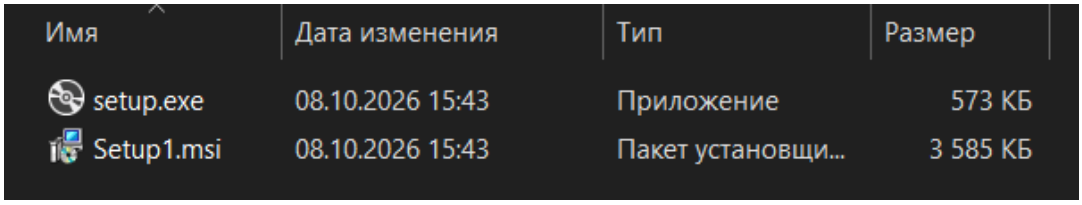


Рисунок 33 – Результат сборки

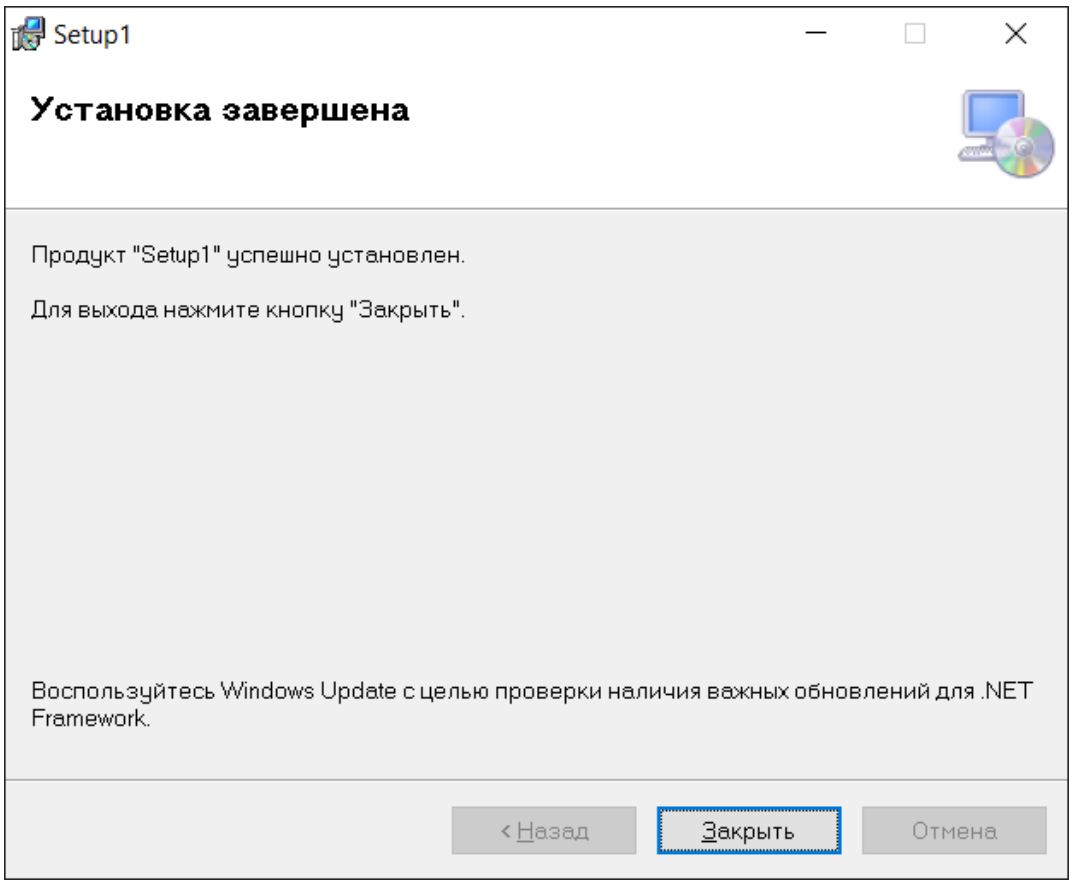


Рисунок 34 – Финальное диалоговое окно установщика

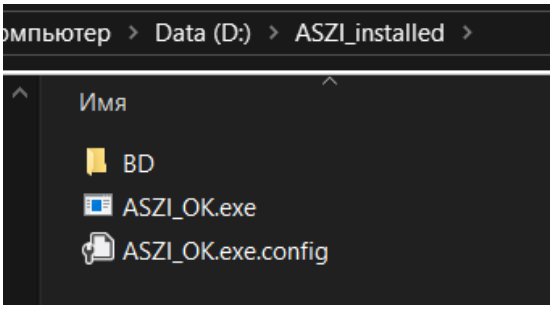


Рисунок 35 – Установленные файлы проекта

Выводы по проделанной работе

В результате выполнения практической работы получены следующие результаты.

Создана база данных АСЗИ отдела кадров маркетинговой компании ООО «Маркетинг с умом». Структура базы данных соответствует модели данных, построенной на предыдущих занятиях, и дополнена служебными таблицами учета пользователей и регистрации событий безопасности.

Разработаны тридцать две хранимые процедуры, из которых семь процедур не имеют параметров и двадцать пять принимают параметры. Процедуры решают задачи выборки, изменения и защиты данных. Проверка подлинности пользователя, автоматическая блокировка учетной записи после трех неудачных попыток входа, регистрация событий безопасности, запрет удаления юридически значимых записей и маскирование персональных данных по роли пользователя относятся к задачам защиты информации.

Выполнено разграничение доступа средствами СУБД. Созданы три роли базы данных с разным объемом прав, прямой доступ к таблицам для прикладных ролей закрыт, поэтому обход проверок, реализованных в хранимых процедурах, невозможен.

Создан проект Microsoft Visual Studio по шаблону приложения Windows Forms. Подключение к базе данных выполнено программно по строке соединения, предусмотрены два варианта подключения, используемые при работе в компьютерном классе и после установки приложения.

Разработана главная форма приложения и одиннадцать диалоговых окон. Команды управления записями доступны как с кнопок, так и из меню, все опасные операции выполняются после подтверждения пользователем.

Написан программный код приложения. Обращения к базе данных выполняются параметризованными вызовами хранимых процедур, что исключает внедрение SQL-инструкций через поля формы, а исключительные ситуации обрабатываются с выводом понятных пользователю сообщений.

Выполнен вызов хранимых процедур из приложения с получением

результатов наборами данных, выходными параметрами и кодом возврата. Создано окно тестирования процедур, применение которого также фиксируется в журнале событий безопасности.

Создан установочный пакет приложения, работоспособность которого проверена на чистой виртуальной машине.

Требования по защите информации, сформулированные в начале работы, реализованы на нескольких уровнях. На уровне базы данных применены хеширование паролей, ограничения целостности, разграничение доступа и учет неудачных попыток входа. На уровне объектов базы данных применены проверка подлинности, регистрация событий безопасности, запрет удаления юридически значимых записей и маскирование персональных данных. На уровне клиентского приложения применены проверка вводимых данных, подтверждение опасных операций, ограничение команд по роли пользователя и обработка исключительных ситуаций.

Список использованных источников

1. Федин Ф.О. Методическая разработка. Часть 12. Разработка серверной и клиентской части АСЗИ. Москва, 2026. 46 с.

2. Федин Ф.О. Методическая разработка. Часть 5. Проектирование подмоделей физического уровня представления данных АСЗИ. Москва, 2025.

3. Microsoft. Документация по SQL Server. Инструкция CREATE PROCEDURE (Transact-SQL). Режим доступа: https://learn.microsoft.com/sql/t-sql/statements/create-procedure-transact-sql.

4. Microsoft. Документация по ADO.NET. Класс SqlCommand и работа с хранимыми процедурами. Режим доступа: https://learn.microsoft.com/dotnet/framework/data/adonet.

5. Microsoft. Документация по приложениям Windows Forms. Элементы управления DataGridView, MenuStrip, StatusStrip. Режим доступа: https://learn.microsoft.com/dotnet/desktop/winforms.

6. Приказ ФСТЭК России от 18 февраля 2013 г. № 21 «Об утверждении Состава и содержания организационных и технических мер по обеспечению безопасности персональных данных при их обработке в информационных системах персональных данных».

7. Постановление Правительства Российской Федерации от 1 ноября 2012 г. № 1119 «Об утверждении требований к защите персональных данных при их обработке в информационных системах персональных данных».

8. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. Москва, Стандартинформ, 2010.

9. Михайлов А.В. Разработка приложений на языке C# в среде Microsoft Visual Studio. Учебное пособие. Москва, 2022. 320 с.

10. ГОСТ Р 56939-2024. Защита информации. Разработка безопасного программного обеспечения. Общие требования. Москва, Стандартинформ, 2024.

Приложение А

Листинг скрипта создания базы данных АСЗИ

Файл PZ9_01_BD_marketing_create.sql

```
USE master;
GO

IF DB_ID(N'Kuznetsov_marketing') IS NOT NULL
BEGIN
   ALTER DATABASE Kuznetsov_marketing SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
   DROP DATABASE Kuznetsov_marketing;
END
GO

CREATE DATABASE Kuznetsov_marketing
   ON PRIMARY
   (
      NAME     = N'Kuznetsov_marketing',
      FILENAME = N'D:\ASZI_OK\BD\Kuznetsov_marketing.mdf',
      SIZE     = 8MB,
      MAXSIZE  = 200MB,
      FILEGROWTH = 8MB
   )
   LOG ON
   (
      NAME     = N'Kuznetsov_marketing_log',
      FILENAME = N'D:\ASZI_OK\BD\Kuznetsov_marketing_log.ldf',
      SIZE     = 8MB,
      MAXSIZE  = 100MB,
      FILEGROWTH = 8MB
   );
GO

ALTER DATABASE Kuznetsov_marketing SET RECOVERY FULL;
GO

USE Kuznetsov_marketing;
GO

/* ------------------------------------------------------------------
   D1 - порядковый номер, D2 - дата, D3 - строка до 40 символов,
   D4 - строка до 150 символов, D5 - строка до 20 символов,
```

```
        D6 - вещественное число, D7 - длинное целое
        ------------------------------------------------------------------- */
        IF EXISTS (SELECT 1 FROM sys.types WHERE name = N'D1' AND is_user_defined = 1) DROP TYPE
D1;
        IF EXISTS (SELECT 1 FROM sys.types WHERE name = N'D2' AND is_user_defined = 1) DROP TYPE
D2;
        IF EXISTS (SELECT 1 FROM sys.types WHERE name = N'D3' AND is_user_defined = 1) DROP TYPE
D3;
        IF EXISTS (SELECT 1 FROM sys.types WHERE name = N'D4' AND is_user_defined = 1) DROP TYPE
D4;
        IF EXISTS (SELECT 1 FROM sys.types WHERE name = N'D5' AND is_user_defined = 1) DROP TYPE
D5;
        IF EXISTS (SELECT 1 FROM sys.types WHERE name = N'D6' AND is_user_defined = 1) DROP TYPE
D6;
        IF EXISTS (SELECT 1 FROM sys.types WHERE name = N'D7' AND is_user_defined = 1) DROP TYPE
D7;
        GO

        CREATE TYPE D1 FROM INT         NOT NULL;
        GO
        CREATE TYPE D2 FROM DATETIME    NULL;
        GO
        CREATE TYPE D3 FROM VARCHAR(40)  NULL;
        GO
        CREATE TYPE D4 FROM VARCHAR(150) NULL;
        GO
        CREATE TYPE D5 FROM VARCHAR(20)  NULL;
        GO
        CREATE TYPE D6 FROM FLOAT       NULL;
        GO
        CREATE TYPE D7 FROM BIGINT      NULL;
        GO

        CREATE TABLE Kandidat
        (
          Nomer_kandidata     D1 IDENTITY(101,1),
          Pasport             D4,
          INN                 D7,
          SNILS               D7,
          Telefon             D7,
          Elektronnaya_pochta  D3,
          Data_vihoda         D2,
```

```
    FIO                D3
)
GO
ALTER TABLE Kandidat
    ADD CONSTRAINT XPKKandidat PRIMARY KEY CLUSTERED (Nomer_kandidata ASC)
GO

CREATE TABLE Sotrudnik
(
    Kod_sotrudnika  D1,
    FIO             D3,
    Dolzhnost       D5
)
GO
ALTER TABLE Sotrudnik
    ADD CONSTRAINT XPKSotrudnik PRIMARY KEY CLUSTERED (Kod_sotrudnika ASC)
GO

CREATE TABLE Soglasie_na_obrabotku_PDn
(
    Nomer_soglasiya   D1,
    Kod_sotrudnika    D1 NULL,
    Nomer_kandidata   D1 NULL,
    Status            D5,
    Data_polucheniya  D2
)
GO
ALTER TABLE Soglasie_na_obrabotku_PDn
    ADD CONSTRAINT XPKSoglasie_na_obrabotku_PDn PRIMARY KEY CLUSTERED
(Nomer_soglasiya ASC)
GO

CREATE TABLE Trudovoy_dogovor
(
    Nomer_dogovora       D1,
    Nomer_kandidata      D1 NULL,
    Kod_sotrudnika       D1 NULL,
    Data_sostavleniya    D2,
    Oklad                D6,
    Dolzhnost_kandidata  D5
)
GO
```

```
ALTER TABLE Trudovoy_dogovor
   ADD CONSTRAINT XPKTrudovoy_dogovor PRIMARY KEY CLUSTERED (Nomer_dogovora ASC)
GO

CREATE TABLE Zayavka
(
   Nomer_zayavki     D1 IDENTITY(401,1),
   Nomer_kandidata   D1 NULL,
   Kod_sotrudnika    D1 NULL,
   Nomer_dogovora    D1 NULL,
   Otdel             D5,
   Data_sozdaniya    D2,
   Status            D5
)
GO
ALTER TABLE Zayavka
   ADD CONSTRAINT XPKZayavka PRIMARY KEY CLUSTERED (Nomer_zayavki ASC)
GO

CREATE TABLE Resurs
(
   Kod_resursa          D1,
   Naimenovanie_resursa D4,
   Tip_resursa          D3
)
GO
ALTER TABLE Resurs
   ADD CONSTRAINT XPKResurs PRIMARY KEY CLUSTERED (Kod_resursa ASC)
GO

CREATE TABLE Pozitsii_zayavki
(
   Nomer_zayavki  D1,
   Kod_resursa    D1,
   Kolichestvo    D7
)
GO
ALTER TABLE Pozitsii_zayavki
   ADD CONSTRAINT XPKPozitsii_zayavki PRIMARY KEY CLUSTERED (Nomer_zayavki ASC,
Kod_resursa ASC)
GO
```

```
CREATE TABLE Propusk
(
   Nomer_propuska     D1,
   Nomer_kandidata    D1 NULL,
   Kod_sotrudnika     D1 NULL,
   Status_gotovnosti  D5,
   Uroven_dostupa     D7
)
GO
ALTER TABLE Propusk
   ADD CONSTRAINT XPKPropusk PRIMARY KEY CLUSTERED (Nomer_propuska ASC)
GO

CREATE TABLE Otchet
(
   Nomer_otcheta   D1,
   Kod_sotrudnika  D1 NULL,
   Mesyats         D7,
   God             D7
)
GO
ALTER TABLE Otchet
   ADD CONSTRAINT XPKOtchet PRIMARY KEY CLUSTERED (Nomer_otcheta ASC)
GO

ALTER TABLE Soglasie_na_obrabotku_PDn
   ADD CONSTRAINT R_10 FOREIGN KEY (Nomer_kandidata) REFERENCES
Kandidat(Nomer_kandidata)
   ON DELETE NO ACTION ON UPDATE NO ACTION
GO
ALTER TABLE Soglasie_na_obrabotku_PDn
   ADD CONSTRAINT R_15 FOREIGN KEY (Kod_sotrudnika) REFERENCES
Sotrudnik(Kod_sotrudnika)
   ON DELETE NO ACTION ON UPDATE NO ACTION
GO
ALTER TABLE Trudovoy_dogovor
   ADD CONSTRAINT R_16 FOREIGN KEY (Nomer_kandidata) REFERENCES
Kandidat(Nomer_kandidata)
   ON DELETE NO ACTION ON UPDATE NO ACTION
GO
ALTER TABLE Trudovoy_dogovor
   ADD CONSTRAINT R_13 FOREIGN KEY (Kod_sotrudnika) REFERENCES
```

```
Sotrudnik(Kod_sotrudnika)
        ON DELETE NO ACTION ON UPDATE NO ACTION
    GO
    ALTER TABLE Zayavka
        ADD CONSTRAINT R_8 FOREIGN KEY (Nomer_kandidata) REFERENCES
Kandidat(Nomer_kandidata)
        ON DELETE NO ACTION ON UPDATE NO ACTION
    GO
    ALTER TABLE Zayavka
        ADD CONSTRAINT R_18 FOREIGN KEY (Nomer_dogovora) REFERENCES
Trudovoy_dogovor(Nomer_dogovora)
        ON DELETE NO ACTION ON UPDATE NO ACTION
    GO
    ALTER TABLE Zayavka
        ADD CONSTRAINT R_19 FOREIGN KEY (Kod_sotrudnika) REFERENCES
Sotrudnik(Kod_sotrudnika)
        ON DELETE NO ACTION ON UPDATE NO ACTION
    GO
    ALTER TABLE Pozitsii_zayavki
        ADD CONSTRAINT R_5 FOREIGN KEY (Nomer_zayavki) REFERENCES Zayavka(Nomer_zayavki)
        ON DELETE NO ACTION ON UPDATE NO ACTION
    GO
    ALTER TABLE Pozitsii_zayavki
        ADD CONSTRAINT R_4 FOREIGN KEY (Kod_resursa) REFERENCES Resurs(Kod_resursa)
        ON DELETE NO ACTION ON UPDATE NO ACTION
    GO
    ALTER TABLE Propusk
        ADD CONSTRAINT R_11 FOREIGN KEY (Nomer_kandidata) REFERENCES
Kandidat(Nomer_kandidata)
        ON DELETE NO ACTION ON UPDATE NO ACTION
    GO
    ALTER TABLE Propusk
        ADD CONSTRAINT R_14 FOREIGN KEY (Kod_sotrudnika) REFERENCES
Sotrudnik(Kod_sotrudnika)
        ON DELETE NO ACTION ON UPDATE NO ACTION
    GO
    ALTER TABLE Otchet
        ADD CONSTRAINT R_17 FOREIGN KEY (Kod_sotrudnika) REFERENCES
Sotrudnik(Kod_sotrudnika)
        ON DELETE NO ACTION ON UPDATE NO ACTION
    GO
```

```
CREATE TABLE Polzovatel_ASZI
(
   Kod_polzovatelya       D1,
   Login                  D3,
   Parol_hash             D4,
   Rol                    D5,
   Kod_sotrudnika         D1 NULL,
   Status                 D5,
   Data_sozdaniya         D2,
   Kolich_neudach_vhodov  D1,
   Data_blokirovki        D2
)
GO
ALTER TABLE Polzovatel_ASZI
   ADD CONSTRAINT XPKPolzovatel_ASZI PRIMARY KEY CLUSTERED (Kod_polzovatelya ASC)
GO
ALTER TABLE Polzovatel_ASZI
   ADD CONSTRAINT UQ_Polzovatel_Login UNIQUE (Login)
GO
ALTER TABLE Polzovatel_ASZI
   ADD CONSTRAINT R_20 FOREIGN KEY (Kod_sotrudnika) REFERENCES
Sotrudnik(Kod_sotrudnika)
   ON DELETE NO ACTION ON UPDATE NO ACTION
GO

CREATE TABLE Zhurnal_sobitiy_bezopasnosti
(
   Nomer_zapisi    BIGINT IDENTITY(1,1) NOT NULL,
   Data_vremya     DATETIME     NOT NULL,   -- дата и время события
   Polzovatel      VARCHAR(40)  NULL,       -- учётная запись (login) инициатора
   Sobytie         VARCHAR(40)  NULL,       -- Вход в систему, Изменение ПДн, Удаление записи и др.
   Obekt           VARCHAR(40)  NULL,       -- объект воздействия
   Rezultat        VARCHAR(20)  NULL,       -- Успешно, Отказано, Ошибка
   Rab_stanciya    VARCHAR(150) NULL,       -- имя ЭВМ (HOST_NAME())
   Prichina        VARCHAR(150) NULL,       -- причина отказа (при наличии)
   CONSTRAINT XPKZhurnal PRIMARY KEY CLUSTERED (Nomer_zapisi ASC)
)
GO
CREATE INDEX IX_Zhurnal_Data ON Zhurnal_sobitiy_bezopasnosti (Data_vremya DESC)
GO
```

Приложение Б

Листинг хранимых процедур АСЗИ

Файл PZ10_01_Hranimye_procedury.sql

```
USE Kuznetsov_marketing;
GO

IF OBJECT_ID(N'usp_Kandidaty_Spisok', N'P') IS NOT NULL DROP PROCEDURE usp_Kandidaty_Spisok;
GO
CREATE PROCEDURE usp_Kandidaty_Spisok
AS
BEGIN
  SET NOCOUNT ON;

  SELECT
    k.Nomer_kandidata                    AS [Код кандидата],
    k.FIO                                AS [ФИО],
    CONVERT(VARCHAR(10), k.Data_vihoda, 104)   AS [Дата выхода],
    ISNULL(s.Status, N'Согласие не получено')  AS [Согласие на обработку ПДн],
    ISNULL(p.Status_gotovnosti, N'Не заказан') AS [Пропуск],
    ISNULL(d.Dolzhnost_kandidata + N' (договор ' +
        CONVERT(VARCHAR(10), d.Nomer_dogovora) + N')',
        N'Договор не подготовлен')       AS [Должность по трудовому договору]

  FROM Kandidat AS k
    LEFT JOIN Soglasie_na_obrabotku_PDn AS s ON s.Nomer_kandidata = k.Nomer_kandidata
    LEFT JOIN Propusk                   AS p ON p.Nomer_kandidata = k.Nomer_kandidata
    LEFT JOIN Trudovoy_dogovor          AS d ON d.Nomer_kandidata = k.Nomer_kandidata
  ORDER BY k.Nomer_kandidata ASC;
END
GO

IF OBJECT_ID(N'usp_Svodka_Zayavok', N'P') IS NOT NULL DROP PROCEDURE usp_Svodka_Zayavok;
GO
CREATE PROCEDURE usp_Svodka_Zayavok
AS
BEGIN
  SET NOCOUNT ON;

  SELECT
```

```
    z.Otdel                                AS [Подразделение],
    COUNT(*)                               AS [Всего заявок],
    SUM(CASE WHEN z.Status = N'Новая'      THEN 1 ELSE 0 END) AS [Новых],
    SUM(CASE WHEN z.Status = N'В работе'   THEN 1 ELSE 0 END) AS [В работе],
    SUM(CASE WHEN z.Status = N'Исполнена'  THEN 1 ELSE 0 END) AS [Исполнено],
    ISNULL(SUM(pos.Kolvo_resursov), 0)     AS [Ресурсов заказано]
  FROM Zayavka AS z
    OUTER APPLY
    (
      SELECT SUM(pz.Kolichestvo) AS Kolvo_resursov
      FROM Pozitsii_zayavki AS pz
      WHERE pz.Nomer_zayavki = z.Nomer_zayavki
    ) AS pos
  GROUP BY z.Otdel
  ORDER BY z.Otdel ASC;
END
GO

IF OBJECT_ID(N'usp_Propuski_Gotovnost', N'P') IS NOT NULL DROP PROCEDURE
usp_Propuski_Gotovnost;
GO
CREATE PROCEDURE usp_Propuski_Gotovnost
AS
BEGIN
  SET NOCOUNT ON;

  SELECT
    p.Uroven_dostupa                   AS [Уровень доступа],
    COUNT(*)                           AS [Всего пропусков],
    SUM(CASE WHEN p.Status_gotovnosti = N'Готов'
          THEN 1 ELSE 0 END)           AS [Готов],
    SUM(CASE WHEN p.Status_gotovnosti = N'Заказан'
          THEN 1 ELSE 0 END)           AS [Заказан],
    SUM(CASE WHEN p.Status_gotovnosti = N'Новый'
          THEN 1 ELSE 0 END)           AS [Новый]
  FROM Propusk AS p
  GROUP BY p.Uroven_dostupa
  ORDER BY p.Uroven_dostupa ASC;
END
GO

IF OBJECT_ID(N'usp_Zhurnal_Bezopasnosti', N'P') IS NOT NULL DROP PROCEDURE
```

```
usp_Zhurnal_Bezopasnosti;
    GO
    CREATE PROCEDURE usp_Zhurnal_Bezopasnosti
    AS
    BEGIN
      SET NOCOUNT ON;

      SELECT TOP (100)
        j.Data_vremya                AS [Дата и время],
        j.Polzovatel                 AS [Учётная запись],
        j.Sobytie                    AS [Событие],
        j.Obekt                      AS [Объект],
        j.Rezultat                   AS [Результат],
        j.Rab_stanciya               AS [Рабочая станция],
        ISNULL(j.Prichina, N'')      AS [Причина]
      FROM Zhurnal_sobitiy_bezopasnosti AS j
      ORDER BY j.Data_vremya DESC, j.Nomer_zapisi DESC;
    END
    GO

    IF OBJECT_ID(N'usp_Kandidaty_Vybor', N'P') IS NOT NULL DROP PROCEDURE
usp_Kandidaty_Vybor;
    GO
    CREATE PROCEDURE usp_Kandidaty_Vybor
    AS
    BEGIN
      SET NOCOUNT ON;

      SELECT
        Nomer_kandidata AS [Kod],
        CONVERT(VARCHAR(10), Nomer_kandidata) + N' - ' + FIO AS [Naimenovanie]
      FROM Kandidat
      ORDER BY Nomer_kandidata ASC;
    END
    GO

    IF OBJECT_ID(N'usp_Resursy_Spisok', N'P') IS NOT NULL DROP PROCEDURE usp_Resursy_Spisok;
    GO
    CREATE PROCEDURE usp_Resursy_Spisok
    AS
    BEGIN
      SET NOCOUNT ON;
```

```
    SELECT
      Kod_resursa AS [Kod],
      CONVERT(VARCHAR(10), Kod_resursa) + N' - ' + Naimenovanie_resursa +
        N' (' + Tip_resursa + N')' AS [Naimenovanie]
    FROM Resurs
    ORDER BY Kod_resursa ASC;
  END
  GO

  IF OBJECT_ID(N'usp_Dogovor_Novyy_nomer', N'P') IS NOT NULL DROP PROCEDURE usp_Dogovor_Novyy_nomer;
  GO
  CREATE PROCEDURE usp_Dogovor_Novyy_nomer
      @Nomer_dogovora INT OUTPUT
  AS
  BEGIN
    SET NOCOUNT ON;

    SELECT @Nomer_dogovora = ISNULL(MAX(Nomer_dogovora), 200) + 1 -- AS [Nomer_dogovora]
    FROM Trudovoy_dogovor;
  END
  GO

  IF OBJECT_ID(N'usp_Zapis_v_zhurnal', N'P') IS NOT NULL DROP PROCEDURE usp_Zapis_v_zhurnal;
  GO
  CREATE PROCEDURE usp_Zapis_v_zhurnal
    @Polzovatel  VARCHAR(40),
    @Sobytie     VARCHAR(40),
    @Obekt       VARCHAR(40),
    @Rezultat    VARCHAR(20),
    @Prichina    VARCHAR(150) = NULL
  AS
  BEGIN
    SET NOCOUNT ON;

    INSERT INTO Zhurnal_sobitiy_bezopasnosti
      (Data_vremya, Polzovatel, Sobytie, Obekt, Rezultat, Rab_stanciya, Prichina)
    VALUES
      (GETDATE(), ISNULL(@Polzovatel, SYSTEM_USER), @Sobytie, @Obekt,
       ISNULL(@Rezultat, N'Успешно'), CONVERT(VARCHAR(150), HOST_NAME()), @Prichina);
  END
```

```
GO

IF OBJECT_ID(N'usp_Vhod_v_sistemu', N'P') IS NOT NULL DROP PROCEDURE usp_Vhod_v_sistemu;
GO
CREATE PROCEDURE usp_Vhod_v_sistemu
   @Login         VARCHAR(40),
   @Parol         VARCHAR(40),
   @Rol           VARCHAR(20)  OUTPUT,
   @Soobshchenie   VARCHAR(150) OUTPUT,
   @Kod_sotrudnika INT         OUTPUT,
   @FIO_sotrudnika VARCHAR(40)  OUTPUT
AS
BEGIN
   SET NOCOUNT ON;

   DECLARE @Kod_polzovatelya INT,
         @Hash            VARCHAR(150),
         @Etalon_hash     VARCHAR(150),
         @Status          VARCHAR(20),
         @Popytki         INT,
         @Chas_nachala    INT = 6,
         @Chas_okonchaniya INT = 23;

   SET @Rol = NULL;
   SET @Soobshchenie = N'';
   SET @Kod_sotrudnika = NULL;
   SET @FIO_sotrudnika = NULL;
   SET @Hash = CONVERT(VARCHAR(150), HASHBYTES('SHA2_256', @Parol + '|' + @Login), 2);

   SELECT @Kod_polzovatelya = Kod_polzovatelya,
        @Etalon_hash     = Parol_hash,
        @Status          = Status,
        @Popytki         = Kolich_neudach_vhodov
   FROM Polzovatel_ASZI
   WHERE Login = @Login;

   -- 1. Проверка существования учётной записи
   IF @Kod_polzovatelya IS NULL
   BEGIN
      SET @Soobshchenie = N'Учётная запись не найдена';
      EXEC usp_Zapis_v_zhurnal @Login, N'Вход в систему', N'Polzovatel_ASZI', N'Отказано',
@Soobshchenie;
```

```
        RETURN 3;
    END

    -- 2. Проверка состояния учётной записи
    IF @Status = N'Заблокирована'
    BEGIN
        SET @Soobshchenie = N'Учётная запись заблокирована, обратитесь к администратору АСЗИ';
        EXEC usp_Zapis_v_zhurnal @Login, N'Вход в систему', N'Polzovatel_ASZI', N'Отказано', @Soobshchenie;
        RETURN 2;
    END

    -- 3. Проверка допустимого интервала времени доступа к АСЗИ
    IF DATEPART(HOUR, GETDATE()) < @Chas_nachala OR DATEPART(HOUR, GETDATE()) >= @Chas_okonchaniya
    BEGIN
        SET @Soobshchenie = N'Вход в систему вне разрешённого интервала времени';
        EXEC usp_Zapis_v_zhurnal @Login, N'Вход в систему', N'Polzovatel_ASZI', N'Отказано', @Soobshchenie;
        RETURN 4;
    END

    -- 4. Сравнение хеша введённого пароля с эталонным значением
    IF @Hash <> @Etalon_hash
    BEGIN
        SET @Popytki = @Popytki + 1;

        IF @Popytki >= 3
        BEGIN
            UPDATE Polzovatel_ASZI
            SET Kolich_neudach_vhodov = @Popytki,
                Status              = N'Заблокирована',
                Data_blokirovki     = GETDATE()
            WHERE Kod_polzovatelya = @Kod_polzovatelya;

            SET @Soobshchenie = N'Превышено число неудачных попыток входа, учётная запись заблокирована';
            EXEC usp_Zapis_v_zhurnal @Login, N'Блокировка учётной записи', N'Polzovatel_ASZI', N'Успешно', @Soobshchenie;
            RETURN 2;
        END
```

```
            UPDATE Polzovatel_ASZI
            SET Kolich_neudach_vhodov = @Popytki
            WHERE Kod_polzovatelya = @Kod_polzovatelya;

            SET @Soobshchenie = N'Введён неверный пароль, осталось попыток: ' +
CONVERT(VARCHAR(5), 3 - @Popytki);
            EXEC usp_Zapis_v_zhurnal @Login, N'Вход в систему', N'Polzovatel_ASZI', N'Отказано',
@Soobshchenie;
            RETURN 1;
        END

        -- 5. Успешная проверка подлинности
        UPDATE Polzovatel_ASZI
        SET Kolich_neudach_vhodov = 0
        WHERE Kod_polzovatelya = @Kod_polzovatelya;

        SELECT @Rol           = p.Rol,
            @Kod_sotrudnika = p.Kod_sotrudnika,
            @FIO_sotrudnika = s.FIO
        FROM Polzovatel_ASZI AS p
            LEFT JOIN Sotrudnik AS s ON s.Kod_sotrudnika = p.Kod_sotrudnika
        WHERE p.Kod_polzovatelya = @Kod_polzovatelya;

        SET @Soobshchenie = N'Вход в систему выполнен, роль пользователя: ' + @Rol;
        EXEC usp_Zapis_v_zhurnal @Login, N'Вход в систему', N'Polzovatel_ASZI', N'Успешно', NULL;

        RETURN 0;
    END
    GO

    IF OBJECT_ID(N'usp_Kandidat_Dobavit', N'P') IS NOT NULL DROP PROCEDURE
usp_Kandidat_Dobavit;
    GO
    CREATE PROCEDURE usp_Kandidat_Dobavit
        @FIO              VARCHAR(40),
        @Pasport          VARCHAR(150),
        @INN              BIGINT,
        @SNILS            BIGINT,
        @Telefon          BIGINT,
        @Elektronnaya_pochta VARCHAR(40),
        @Data_vihoda      DATETIME,
            @Nomer_kandidata    INT OUTPUT,
```

```
    @Polzovatel         VARCHAR(40) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Контроль обязательных полей и форматов
    IF @FIO IS NULL OR LTRIM(RTRIM(@FIO)) = N''
        THROW 50002, N'Не заполнено поле ФИО кандидата.', 1;

    IF @Elektronnaya_pochta IS NULL OR CHARINDEX(N'@', @Elektronnaya_pochta) = 0
        THROW 50003, N'Адрес электронной почты заполнен неверно.', 1;

    IF EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
        THROW 50004, N'Кандидат с указанным номером уже зарегистрирован в системе.', 1;

            -- Объявляем переменную для конвертированного номера кандидата
    DECLARE @Nomer_kandidata_Str VARCHAR(40);

            BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO Kandidat
            (Pasport, INN, SNILS, Telefon, Elektronnaya_pochta, Data_vihoda, FIO)
        VALUES
            (@Pasport, @INN, @SNILS, @Telefon, @Elektronnaya_pochta, @Data_vihoda, @FIO);

                        SET @Nomer_kandidata = CONVERT(INT, SCOPE_IDENTITY());
                        SET @Nomer_kandidata_Str = CONVERT(VARCHAR(40), @Nomer_kandidata)
        EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Добавление ПДн кандидата',
            @Nomer_kandidata_Str, N'Успешно', NULL;

        COMMIT TRANSACTION;
        RETURN 0;
    END TRY
    BEGIN CATCH
                        DECLARE @ErrMsg NVARCHAR(4000);
        SET @ErrMsg = ERROR_MESSAGE();

        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

        EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Добавление ПДн кандидата',
            @Nomer_kandidata_Str, N'Ошибка', @ErrMsg;
```

```
        THROW;
    END CATCH
END
GO

IF OBJECT_ID(N'usp_Kandidat_Redaktirovat', N'P') IS NOT NULL DROP PROCEDURE usp_Kandidat_Redaktirovat;
GO
CREATE PROCEDURE usp_Kandidat_Redaktirovat
    @Nomer_kandidata     INT,
    @FIO                 VARCHAR(40),
    @Pasport             VARCHAR(150),
    @INN                 BIGINT,
    @SNILS               BIGINT,
    @Telefon             BIGINT,
    @Elektronnaya_pochta VARCHAR(40),
    @Data_vihoda         DATETIME,
    @Polzovatel          VARCHAR(40) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
        THROW 50005, N'Кандидат с указанным номером не найден.', 1;

    IF @FIO IS NULL OR LTRIM(RTRIM(@FIO)) = N''
        THROW 50002, N'Не заполнено поле ФИО кандидата.', 1;

    DECLARE @Nomer_kandidata_Str VARCHAR(40);
    SET @Nomer_kandidata_Str = CONVERT(VARCHAR(40), @Nomer_kandidata);

    BEGIN TRY
        BEGIN TRANSACTION;

        UPDATE Kandidat
        SET FIO              = @FIO,
            Pasport          = @Pasport,
            INN              = @INN,
            SNILS            = @SNILS,
            Telefon          = @Telefon,
            Elektronnaya_pochta = @Elektronnaya_pochta,
            Data_vihoda      = @Data_vihoda
```

```
        WHERE Nomer_kandidata = @Nomer_kandidata;

        EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Изменение ПДн кандидата',
            @Nomer_kandidata_Str, N'Успешно', NULL;

        COMMIT TRANSACTION;
        RETURN 0;
    END TRY
    BEGIN CATCH
                DECLARE @ErrMsg NVARCHAR(4000);
        SET @ErrMsg = ERROR_MESSAGE();

        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

        EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Изменение ПДн кандидата',
            @Nomer_kandidata_Str, N'Ошибка', @ErrMsg;
        THROW;
    END CATCH
END
GO

IF OBJECT_ID(N'usp_Kandidat_Udalit', N'P') IS NOT NULL DROP PROCEDURE usp_Kandidat_Udalit;
GO
CREATE PROCEDURE usp_Kandidat_Udalit
    @Nomer_kandidata INT,
    @Polzovatel      VARCHAR(40) = NULL
AS
BEGIN
    SET NOCOUNT ON;

        DECLARE @Nomer_kandidata_Str VARCHAR(40);
    SET @Nomer_kandidata_Str = CONVERT(VARCHAR(40), @Nomer_kandidata);

        IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
        THROW 50005, N'Кандидат с указанным номером не найден.', 1;

    IF EXISTS (SELECT 1 FROM Trudovoy_dogovor WHERE Nomer_kandidata = @Nomer_kandidata)

    BEGIN
                DECLARE @Nomera_dogovorov VARCHAR(150);
                SELECT @Nomera_dogovorov = STUFF((SELECT N', ' + CONVERT(VARCHAR(20),
d.Nomer_dogovora)
```

```
                                FROM Trudovoy_dogovor AS d
                                WHERE d.Nomer_kandidata = @Nomer_kandidata
                                ORDER BY d.Nomer_dogovora
                                FOR XML PATH('')), 1, 2, N'');
                        --SET @Nomera_dogovorov = N'По кандидату заключён трудовой договор ' +
@Nomera_dogovorov
            EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Удаление ПДн кандидата',
                @Nomer_kandidata_Str, N'Отказано',
                @Nomera_dogovorov;
                        DECLARE @Soobshchenie_dogovor NVARCHAR(2048) =
                N'Удаление запрещено, по кандидату подготовлен трудовой договор № ' +
                CONVERT(NVARCHAR(100), @Nomera_dogovorov) +
                N'. Удалите договор и повторите удаление кандидата.';

            THROW 50006, @Soobshchenie_dogovor, 1;

        END

        IF EXISTS (SELECT 1 FROM Zayavka WHERE Nomer_kandidata = @Nomer_kandidata)
        BEGIN
                        DECLARE @Nomera_zayavok VARCHAR(200);
                        DECLARE @Nomera_zayavok_Str VARCHAR(200);

            SELECT @Nomera_zayavok = STUFF((SELECT N', ' + CONVERT(VARCHAR(20),
z.Nomer_zayavki)
                                FROM Zayavka AS z
                                WHERE z.Nomer_kandidata = @Nomer_kandidata
                                ORDER BY z.Nomer_zayavki
                                FOR XML PATH('')), 1, 2, N'');
                        SET @Nomera_zayavok_Str = N'Заявки кандидата: ' + @Nomera_zayavok;
            EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Удаление ПДн кандидата',
                @Nomer_kandidata_Str, N'Отказано',
                @Nomera_zayavok_Str;
                        DECLARE @Soobshchenie_zayavka NVARCHAR(2048) =
                N'Удаление запрещено, по кандидату созданы заявки № ' +
                CONVERT(NVARCHAR(200), @Nomera_zayavok) +
                N'. Удалите заявки и повторите удаление кандидата.';

            THROW 50007, @Soobshchenie_zayavka, 1;

        END
```

```
    BEGIN TRY
      BEGIN TRANSACTION;

      DELETE FROM Soglasie_na_obrabotku_PDn WHERE Nomer_kandidata = @Nomer_kandidata;
      DELETE FROM Propusk                  WHERE Nomer_kandidata = @Nomer_kandidata;
      DELETE FROM Kandidat                 WHERE Nomer_kandidata = @Nomer_kandidata;

      EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Удаление ПДн кандидата',
        @Nomer_kandidata_Str, N'Успешно', NULL;

      COMMIT TRANSACTION;
      RETURN 0;
    END TRY
    BEGIN CATCH
                DECLARE @Kod_oshibki    INT          = ERROR_NUMBER();
      DECLARE @Stroka_oshibki INT          = ERROR_LINE();
      DECLARE @Mesto_oshibki  NVARCHAR(128) = ISNULL(ERROR_PROCEDURE(), N'процедура не определена');
      DECLARE @Tekst_oshibki  NVARCHAR(2048) =
        ISNULL(ERROR_MESSAGE(), N'сообщение не передано сервером');

      IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

      BEGIN TRY
        EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Удаление ПДн кандидата',
          @Nomer_kandidata_Str, N'Ошибка', @Tekst_oshibki;
      END TRY
                BEGIN CATCH
                END CATCH
                DECLARE @Soobshchenie_oshibka NVARCHAR(2048) =
        N'Операция не выполнена. Код ' + CONVERT(NVARCHAR(10), @Kod_oshibki) +
        N', ' + @Mesto_oshibki + N', строка ' +
        CONVERT(NVARCHAR(10), @Stroka_oshibki) + N': ' + @Tekst_oshibki;

      DECLARE @Kod_dlya_vybrosa INT =
        CASE WHEN @Kod_oshibki >= 50000 THEN @Kod_oshibki ELSE 50040 END;

      THROW @Kod_dlya_vybrosa, @Soobshchenie_oshibka, 1;
        END CATCH
  END
  GO
```

```
IF OBJECT_ID(N'usp_Zayavka_Sozdat', N'P') IS NOT NULL DROP PROCEDURE usp_Zayavka_Sozdat;
GO
CREATE PROCEDURE usp_Zayavka_Sozdat
   @Otdel          VARCHAR(20),
   @Data_sozdaniya  DATETIME,
   @Status         VARCHAR(20),
   @Nomer_kandidata INT,
   @Kod_sotrudnika  INT,
   @Nomer_dogovora  INT,
         @Kod_resursa     INT,
         @Kolichestvo     INT,
         @Nomer_zayavki   INT OUTPUT,
   @Polzovatel      VARCHAR(40) = NULL
AS
BEGIN
   SET NOCOUNT ON;

   IF @Otdel NOT IN (N'ИТ', N'АХУ', N'Бухгалтерия')
      THROW 50008, N'Указано недопустимое подразделение заявки.', 1;

   IF @Status NOT IN (N'Новая', N'В работе', N'Исполнена')
      THROW 50009, N'Указано недопустимое состояние заявки.', 1;

   IF @Data_sozdaniya IS NULL
      THROW 50016, N'Не заполнена дата создания заявки.', 1;

   IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
      THROW 50011, N'Указанный кандидат не найден в базе данных.', 1;

   IF NOT EXISTS (SELECT 1 FROM Trudovoy_dogovor
               WHERE Nomer_dogovora = @Nomer_dogovora
                 AND Nomer_kandidata = @Nomer_kandidata)
      THROW 50012, N'Указанный трудовой договор не относится к выбранному кандидату.', 1;

         IF NOT EXISTS (SELECT 1 FROM Sotrudnik WHERE Kod_sotrudnika = @Kod_sotrudnika)
                  THROW 50017, N'Указанный сотрудник отдела кадров не найден в базе данных.', 1;

         IF NOT EXISTS (SELECT 1 FROM Resurs WHERE Kod_resursa = @Kod_resursa)
      THROW 50018, N'Указанный ресурс не найден в базе данных.', 1;

   IF @Kolichestvo IS NULL OR @Kolichestvo <= 0
      THROW 50019, N'Количество ресурса должно быть положительным числом.', 1;
```

```
    DECLARE @Nomer_zayavki_Str VARCHAR(40);
    DECLARE @Kod_resursa_Str VARCHAR(40);

    BEGIN TRY
      BEGIN TRANSACTION;

      INSERT INTO Zayavka
        (Nomer_kandidata, Kod_sotrudnika, Nomer_dogovora, Otdel, Data_sozdaniya, Status)
      VALUES
        (@Nomer_kandidata, @Kod_sotrudnika, @Nomer_dogovora, @Otdel, @Data_sozdaniya,
@Status);

                SET @Nomer_zayavki = CONVERT(INT, SCOPE_IDENTITY());
                SET @Nomer_zayavki_Str = CONVERT(VARCHAR(40), @Nomer_zayavki);
                SET @Kod_resursa_Str = N'Ресурс: ' + CONVERT(VARCHAR(40), @Kod_resursa)
                INSERT INTO Pozitsii_zayavki (Nomer_zayavki, Kod_resursa, Kolichestvo)
      VALUES (@Nomer_zayavki, @Kod_resursa, @Kolichestvo);

      EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Создание заявки',
        @Nomer_zayavki_Str, N'Успешно',
        @Kod_resursa_Str;

      COMMIT TRANSACTION;
      RETURN 0;
    END TRY
    BEGIN CATCH
          DECLARE @ErrMsg NVARCHAR(4000);
      SET @ErrMsg = ERROR_MESSAGE();
      IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
      EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Создание заявки',
        @Nomer_zayavki_Str, N'Ошибка', @ErrMsg;
      THROW;
    END CATCH
  END
  GO

  IF OBJECT_ID(N'usp_Zayavka_Redaktirovat', N'P') IS NOT NULL DROP PROCEDURE
usp_Zayavka_Redaktirovat;
  GO
  CREATE PROCEDURE usp_Zayavka_Redaktirovat
    @Nomer_zayavki INT,
```

```
    @Otdel        VARCHAR(20),
    @Status       VARCHAR(20),
    @Polzovatel   VARCHAR(40) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Zayavka WHERE Nomer_zayavki = @Nomer_zayavki)
        THROW 50013, N'Заявка с указанным номером не найдена.', 1;

    IF @Otdel NOT IN (N'ИТ', N'АХУ', N'Бухгалтерия')
        THROW 50008, N'Указано недопустимое подразделение заявки.', 1;

    IF @Status NOT IN (N'Новая', N'В работе', N'Исполнена')
        THROW 50009, N'Указано недопустимое состояние заявки.', 1;

    UPDATE Zayavka
    SET Otdel  = @Otdel,
        Status = @Status
    WHERE Nomer_zayavki = @Nomer_zayavki;

    DECLARE @Nomer_zayavki_Str VARCHAR(40);
    SET @Nomer_zayavki_Str = CONVERT(VARCHAR(40), @Nomer_zayavki);
    EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Изменение заявки',
         @Nomer_zayavki_Str, N'Успешно', NULL;

    RETURN 0;
END
GO

IF OBJECT_ID(N'usp_Zayavka_Udalit', N'P') IS NOT NULL DROP PROCEDURE usp_Zayavka_Udalit;
GO
CREATE PROCEDURE usp_Zayavka_Udalit
    @Nomer_zayavki INT,
    @Polzovatel    VARCHAR(40) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Zayavka WHERE Nomer_zayavki = @Nomer_zayavki)
        THROW 50013, N'Заявка с указанным номером не найдена.', 1;
```

```
    DECLARE @Nomer_zayavki_Str VARCHAR(40);
    SET @Nomer_zayavki_Str = CONVERT(VARCHAR(40), @Nomer_zayavki);
    BEGIN TRY
      BEGIN TRANSACTION;

      DELETE FROM Pozitsii_zayavki WHERE Nomer_zayavki = @Nomer_zayavki;
      DELETE FROM Zayavka          WHERE Nomer_zayavki = @Nomer_zayavki;

      EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Удаление заявки',
        @Nomer_zayavki_Str, N'Успешно', NULL;

      COMMIT TRANSACTION;
      RETURN 0;
    END TRY
    BEGIN CATCH
          DECLARE @ErrMsg NVARCHAR(4000);
      SET @ErrMsg = ERROR_MESSAGE();
      IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
      EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Удаление заявки',
        @Nomer_zayavki_Str, N'Ошибка', @ErrMsg;
      THROW;
    END CATCH
  END
  GO

  IF OBJECT_ID(N'usp_Dogovory_Spisok', N'P') IS NOT NULL DROP PROCEDURE
usp_Dogovory_Spisok;
  GO
  CREATE PROCEDURE usp_Dogovory_Spisok
    @Nomer_kandidata INT = NULL
  AS
  BEGIN
    SET NOCOUNT ON;

    SELECT
      d.Nomer_dogovora AS [Kod],
      CONVERT(VARCHAR(10), d.Nomer_dogovora) + N' - ' + ISNULL(d.Dolzhnost_kandidata, N'') +
        N' (кандидат ' + CONVERT(VARCHAR(10), d.Nomer_kandidata) + N', от ' +
        CONVERT(VARCHAR(10), d.Data_sostavleniya, 104) + N')' AS [Naimenovanie]
    FROM Trudovoy_dogovor AS d
    WHERE @Nomer_kandidata IS NULL
      OR d.Nomer_kandidata = @Nomer_kandidata
```

```
    ORDER BY d.Nomer_dogovora ASC;
END
GO

IF OBJECT_ID(N'usp_Zhurnal_za_period', N'P') IS NOT NULL DROP PROCEDURE usp_Zhurnal_za_period;
GO
CREATE PROCEDURE usp_Zhurnal_za_period
    @Data_ot DATETIME,
    @Data_do DATETIME
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        j.Data_vremya     AS [Дата и время],
        j.Polzovatel      AS [Учётная запись],
        j.Sobytie         AS [Событие],
        j.Obekt           AS [Объект],
        j.Rezultat        AS [Результат],
        j.Rab_stanciya    AS [Рабочая станция],
        ISNULL(j.Prichina, N'') AS [Причина]
    FROM Zhurnal_sobitiy_bezopasnosti AS j
    WHERE j.Data_vremya >= @Data_ot
      AND j.Data_vremya <  DATEADD(DAY, 1, @Data_do)
    ORDER BY j.Data_vremya DESC;
END
GO

IF OBJECT_ID(N'usp_Poisk_Kandidata', N'P') IS NOT NULL DROP PROCEDURE usp_Poisk_Kandidata;
GO
CREATE PROCEDURE usp_Poisk_Kandidata
    @Fragment_FIO VARCHAR(40),
    @Rol          VARCHAR(20),
    @Polzovatel   VARCHAR(40) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    IF @Rol NOT IN (N'Менеджер', N'Специалист', N'Администратор')
        THROW 50014, N'Неизвестная роль пользователя.', 1;
```

```
        DECLARE @LogDetails NVARCHAR(100);
    SET @LogDetails = N'Роль: ' + CAST(@Rol AS NVARCHAR(20));

    EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Поиск ПДн кандидата', N'Kandidat', N'Успешно',
        @LogDetails;

    IF @Rol IN (N'Менеджер', N'Администратор')
    BEGIN
      SELECT
        Nomer_kandidata     AS [Код кандидата],
        FIO                 AS [ФИО],
        Pasport             AS [Паспорт],
        CONVERT(VARCHAR(20), INN)   AS [ИНН],
        CONVERT(VARCHAR(20), SNILS) AS [СНИЛС],
        CONVERT(VARCHAR(20), Telefon) AS [Телефон],
        Elektronnaya_pochta AS [Электронная почта]
      FROM Kandidat
      WHERE FIO LIKE N'%' + @Fragment_FIO + N'%'
      ORDER BY FIO ASC;
    END
    ELSE
    BEGIN
      SELECT
        Nomer_kandidata     AS [Код кандидата],
        FIO                 AS [ФИО],
        N'***' AS [Паспорт],
        N'***' + RIGHT(CONVERT(VARCHAR(20), INN), 4)   AS [ИНН],
        N'***' + RIGHT(CONVERT(VARCHAR(20), SNILS), 4) AS [СНИЛС],
        N'***' + RIGHT(CONVERT(VARCHAR(20), Telefon), 4) AS [Телефон],
        LEFT(Elektronnaya_pochta, 1) + N'***' AS [Электронная почта]
      FROM Kandidat
      WHERE FIO LIKE N'%' + @Fragment_FIO + N'%'
      ORDER BY FIO ASC;
    END
  END
  GO

  IF OBJECT_ID(N'usp_Zayavki_Kandidata', N'P') IS NOT NULL DROP PROCEDURE
usp_Zayavki_Kandidata;
  GO
  CREATE PROCEDURE usp_Zayavki_Kandidata
    @Nomer_kandidata INT
```

```
AS
BEGIN
  SET NOCOUNT ON;

  SELECT
    z.Nomer_zayavki                AS [Номер заявки],
    z.Otdel                        AS [Подразделение],
    CONVERT(VARCHAR(10), z.Data_sozdaniya, 104) AS [Дата создания],
    z.Status                       AS [Состояние],
    pz.Kod_resursa                 AS [Код ресурса],
    ISNULL(r.Naimenovanie_resursa, N'')    AS [Наименование ресурса],
    ISNULL(CONVERT(VARCHAR(20), pz.Kolichestvo), N'') AS [Количество],
                ISNULL(d.Dolzhnost_kandidata, N'')     AS [Должность по договору]
      FROM Zayavka AS z
    LEFT JOIN Pozitsii_zayavki AS pz ON pz.Nomer_zayavki = z.Nomer_zayavki
    LEFT JOIN Resurs           AS r  ON r.Kod_resursa    = pz.Kod_resursa
    LEFT JOIN Trudovoy_dogovor AS d  ON d.Nomer_dogovora = z.Nomer_dogovora
  WHERE z.Nomer_kandidata = @Nomer_kandidata
  ORDER BY z.Nomer_zayavki ASC, pz.Kod_resursa ASC;
END
GO

IF OBJECT_ID(N'usp_Kandidat_Dannye', N'P') IS NOT NULL DROP PROCEDURE
usp_Kandidat_Dannye;
GO
CREATE PROCEDURE usp_Kandidat_Dannye
  @Nomer_kandidata INT,
  @Polzovatel      VARCHAR(40) = NULL
AS
BEGIN
  SET NOCOUNT ON;

  IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
    THROW 50005, N'Кандидат с указанным номером не найден.', 1;

      DECLARE @Nomer_kandidata_Str VARCHAR(40);
  SET @Nomer_kandidata_Str = CONVERT(VARCHAR(40), @Nomer_kandidata);

  EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Просмотр ПДн кандидата',
    @Nomer_kandidata_Str, N'Успешно', NULL;

  SELECT
```

```
        Nomer_kandidata,
        FIO,
        Pasport,
        INN,
        SNILS,
        Telefon,
        Elektronnaya_pochta,
        Data_vihoda
    FROM Kandidat
    WHERE Nomer_kandidata = @Nomer_kandidata;
END
GO

IF OBJECT_ID(N'usp_Statistika_PDn', N'P') IS NOT NULL DROP PROCEDURE usp_Statistika_PDn;
GO
CREATE PROCEDURE usp_Statistika_PDn
    @God INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        @God                                AS [Год],
        COUNT(DISTINCT k.Nomer_kandidata)   AS [Кандидатов, чел.],
        COUNT(DISTINCT d.Nomer_dogovora)    AS [Договоров, шт.],
        COUNT(DISTINCT p.Nomer_propuska)    AS [Пропусков, шт.],
        SUM(CASE WHEN s.Status = N'Получено' THEN 1 ELSE 0 END) AS [Согласий получено],
        SUM(CASE WHEN s.Status <> N'Получено' OR s.Status IS NULL THEN 1 ELSE 0 END) AS
[Согласий не получено]
    FROM Kandidat AS k
        LEFT JOIN Soglasie_na_obrabotku_PDn AS s ON s.Nomer_kandidata = k.Nomer_kandidata
        LEFT JOIN Trudovoy_dogovor          AS d ON d.Nomer_kandidata = k.Nomer_kandidata
        LEFT JOIN Propusk                   AS p ON p.Nomer_kandidata = k.Nomer_kandidata
    WHERE YEAR(k.Data_vihoda) = @God;
END
GO

IF OBJECT_ID(N'usp_Procedura_Parametry', N'P') IS NOT NULL DROP PROCEDURE
usp_Procedura_Parametry;
GO
CREATE PROCEDURE usp_Procedura_Parametry
    @Imya_procedury VARCHAR(40),
```

```
        @Rol VARCHAR(20) = NULL
AS
BEGIN
  SET NOCOUNT ON;

  IF OBJECT_ID(@Imya_procedury, N'P') IS NULL
    THROW 50020, N'Хранимая процедура с указанным именем не найдена в базе данных.', 1;

  IF @Rol IS NOT NULL
  BEGIN
    CREATE TABLE #Dostupnye (Procedura VARCHAR(40));

    INSERT INTO #Dostupnye (Procedura)
    EXEC usp_Dostupnye_Procedury @Rol;

    IF NOT EXISTS (SELECT 1 FROM #Dostupnye WHERE Procedura = @Imya_procedury)
      THROW 50021, N'Процедура недоступна для роли текущего пользователя.', 1;
  END

  SELECT
    p.name            AS [Parametr],
    t.name            AS [Tip],
    p.max_length      AS [Razmer],
    p.is_output       AS [Vyvod],        -- [ИЗМЕНЕНО 07.10.2026]
    p.has_default_value AS [Neobyazatelnyy]
  FROM sys.parameters AS p
    JOIN sys.types AS t ON t.user_type_id = p.user_type_id
  WHERE p.object_id = OBJECT_ID(@Imya_procedury)
    AND p.parameter_id > 0
  ORDER BY p.parameter_id ASC;
END
GO

IF OBJECT_ID(N'usp_Dogovory_Kandidata', N'P') IS NOT NULL DROP PROCEDURE
usp_Dogovory_Kandidata;
GO
CREATE PROCEDURE usp_Dogovory_Kandidata
  @Nomer_kandidata INT
AS
BEGIN
  SET NOCOUNT ON;
```

```
    IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
        THROW 50011, N'Указанный кандидат не найден в базе данных.', 1;

    SELECT
        d.Nomer_dogovora                        AS [Номер договора],
        ISNULL(d.Dolzhnost_kandidata, N'')      AS [Должность],
        CONVERT(VARCHAR(10), d.Data_sostavleniya, 104)  AS [Дата составления],
        d.Oklad                                 AS [Оклад],
        ISNULL(s.FIO, N'')                      AS [Сотрудник отдела кадров],
        k.FIO                                   AS [Кандидат]
    FROM Trudovoy_dogovor AS d
        JOIN Kandidat   AS k ON k.Nomer_kandidata = d.Nomer_kandidata
        LEFT JOIN Sotrudnik AS s ON s.Kod_sotrudnika = d.Kod_sotrudnika
    WHERE d.Nomer_kandidata = @Nomer_kandidata
    ORDER BY d.Nomer_dogovora ASC;
END
GO

IF OBJECT_ID(N'usp_Dogovor_Dannye', N'P') IS NOT NULL DROP PROCEDURE usp_Dogovor_Dannye;
GO
CREATE PROCEDURE usp_Dogovor_Dannye
    @Nomer_dogovora INT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Trudovoy_dogovor WHERE Nomer_dogovora = @Nomer_dogovora)
        THROW 50030, N'Трудовой договор с указанным номером не найден в базе данных.', 1;

    -- Фамилия сотрудника выводится для заполнения окна договора,
    -- чтобы вместо кода сотрудника в окне была понятная подпись
    SELECT d.Nomer_dogovora, d.Nomer_kandidata, d.Kod_sotrudnika,
           d.Data_sostavleniya, d.Oklad, d.Dolzhnost_kandidata,
           ISNULL(s.FIO, N'') AS FIO_sotrudnika
    FROM Trudovoy_dogovor AS d
        LEFT JOIN Sotrudnik AS s ON s.Kod_sotrudnika = d.Kod_sotrudnika
    WHERE d.Nomer_dogovora = @Nomer_dogovora;
END
GO
```

```
IF OBJECT_ID(N'usp_Dogovor_Dobavit', N'P') IS NOT NULL DROP PROCEDURE
usp_Dogovor_Dobavit;
GO
CREATE PROCEDURE usp_Dogovor_Dobavit
  @Nomer_dogovora      INT,
  @Nomer_kandidata     INT,
  @Kod_sotrudnika      INT,
  @Data_sostavleniya   DATETIME,
  @Oklad               FLOAT,
  @Dolzhnost_kandidata VARCHAR(20),
  @Nomer_novogo        INT OUTPUT,         -- созданный договор, для заявок
  @Polzovatel          VARCHAR(40) = NULL
AS
BEGIN
  SET NOCOUNT ON;

  IF @Nomer_dogovora IS NULL OR @Nomer_dogovora <= 0
    THROW 50022, N'Номер трудового договора указан неверно.', 1;

  IF EXISTS (SELECT 1 FROM Trudovoy_dogovor WHERE Nomer_dogovora = @Nomer_dogovora)
    THROW 50022, N'Трудовой договор с таким номером уже существует, укажите другой номер.',
1;

  IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
    THROW 50011, N'Указанный кандидат не найден в базе данных.', 1;

  IF NOT EXISTS (SELECT 1 FROM Sotrudnik WHERE Kod_sotrudnika = @Kod_sotrudnika)
    THROW 50017, N'Указанный сотрудник отдела кадров не найден в базе данных.', 1;

  IF @Data_sostavleniya IS NULL
    THROW 50016, N'Не заполнена дата составления трудового договора.', 1;

  IF @Oklad IS NULL OR @Oklad <= 0
    THROW 50023, N'Оклад по трудовому договору должен быть положительным числом.', 1;

  IF @Dolzhnost_kandidata IS NULL OR LTRIM(RTRIM(@Dolzhnost_kandidata)) = N''
    THROW 50024, N'Не заполнена должность кандидата по трудовому договору.', 1;

  BEGIN TRY
    BEGIN TRANSACTION;

    INSERT INTO Trudovoy_dogovor
```

```
        (Nomer_dogovora, Nomer_kandidata, Kod_sotrudnika,
         Data_sostavleniya, Oklad, Dolzhnost_kandidata)
      VALUES
        (@Nomer_dogovora, @Nomer_kandidata, @Kod_sotrudnika,
         @Data_sostavleniya, @Oklad, @Dolzhnost_kandidata);

      SET @Nomer_novogo = @Nomer_dogovora;
                DECLARE @Nomer_novogo_Str VARCHAR(40);
                SET @Nomer_novogo_Str = CONVERT(VARCHAR(40), @Nomer_dogovora);

                DECLARE @Dolzhnost_kandidata_Str VARCHAR(40);
                SET @Dolzhnost_kandidata_Str = N'Должность: ' + @Dolzhnost_kandidata;



                EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Подготовка трудового договора',
        @Nomer_novogo_Str, N'Успешно',
        @Dolzhnost_kandidata_Str;

      COMMIT TRANSACTION;
      RETURN 0;
    END TRY
    BEGIN CATCH
      IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
                DECLARE @ErrMsg VARCHAR(200);
                SET @ErrMsg = ERROR_MESSAGE();
      EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Подготовка трудового договора',
        N'не создан', N'Ошибка', @ErrMsg;
      THROW;
    END CATCH
  END
  GO

  IF OBJECT_ID(N'usp_Dogovor_Redaktirovat', N'P') IS NOT NULL DROP PROCEDURE
usp_Dogovor_Redaktirovat;
  GO
  CREATE PROCEDURE usp_Dogovor_Redaktirovat
    @Nomer_dogovora      INT,
    @Kod_sotrudnika      INT,
    @Data_sostavleniya   DATETIME,
    @Oklad               FLOAT,
    @Dolzhnost_kandidata VARCHAR(20),
    @Polzovatel          VARCHAR(40) = NULL
```

```
AS
BEGIN
  SET NOCOUNT ON;

  IF NOT EXISTS (SELECT 1 FROM Trudovoy_dogovor WHERE Nomer_dogovora =
@Nomer_dogovora)
    THROW 50030, N'Трудовой договор с указанным номером не найден в базе данных.', 1;

  IF NOT EXISTS (SELECT 1 FROM Sotrudnik WHERE Kod_sotrudnika = @Kod_sotrudnika)
    THROW 50017, N'Указанный сотрудник отдела кадров не найден в базе данных.', 1;

  IF @Data_sostavleniya IS NULL
    THROW 50016, N'Не заполнена дата составления трудового договора.', 1;

  IF @Oklad IS NULL OR @Oklad <= 0
    THROW 50023, N'Оклад по трудовому договору должен быть положительным числом.', 1;

  IF @Dolzhnost_kandidata IS NULL OR LTRIM(RTRIM(@Dolzhnost_kandidata)) = N''
    THROW 50024, N'Не заполнена должность кандидата по трудовому договору.', 1;

  BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Trudovoy_dogovor
    SET Kod_sotrudnika      = @Kod_sotrudnika,
      Data_sostavleniya   = @Data_sostavleniya,
      Oklad               = @Oklad,
      Dolzhnost_kandidata = @Dolzhnost_kandidata
    WHERE Nomer_dogovora = @Nomer_dogovora;

          DECLARE @Nomer_dogovora_Str VARCHAR(40);
          SET @Nomer_dogovora_Str = CONVERT(VARCHAR(40), @Nomer_dogovora);

          DECLARE @Dolzhnost_kandidata_Str VARCHAR(100);
          SET @Dolzhnost_kandidata_Str = N'Должность: ' + @Dolzhnost_kandidata;

    EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Изменение трудового договора',
      @Nomer_dogovora_Str, N'Успешно',
      @Dolzhnost_kandidata_Str;

    COMMIT TRANSACTION;
    RETURN 0;
```

```
    END TRY
    BEGIN CATCH
      IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

                    SET @Nomer_dogovora_Str = CONVERT(VARCHAR(40), @Nomer_dogovora);
                    DECLARE @ErrMsg VARCHAR(200);
                    SET @ErrMsg = ERROR_MESSAGE();

      EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Изменение трудового договора',
        @Nomer_dogovora_Str, N'Ошибка', @ErrMsg;
      THROW;
    END CATCH
  END
  GO

  IF OBJECT_ID(N'usp_Dogovor_Udalit', N'P') IS NOT NULL DROP PROCEDURE usp_Dogovor_Udalit;
  GO
  CREATE PROCEDURE usp_Dogovor_Udalit
    @Nomer_dogovora INT,
    @Polzovatel     VARCHAR(40) = NULL
  AS
  BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Trudovoy_dogovor WHERE Nomer_dogovora =
@Nomer_dogovora)
      THROW 50030, N'Трудовой договор с указанным номером не найден в базе данных.', 1;

    IF EXISTS (SELECT 1 FROM Zayavka WHERE Nomer_dogovora = @Nomer_dogovora)
    BEGIN
                    DECLARE @Nomera_zayavok VARCHAR(200);
                    DECLARE @Nomera_zayavok_Str VARCHAR(200);
      SELECT @Nomera_zayavok = STUFF((SELECT N', ' + CONVERT(VARCHAR(20),
z.Nomer_zayavki)
                        FROM Zayavka AS z
                        WHERE z.Nomer_dogovora = @Nomer_dogovora
                        ORDER BY z.Nomer_zayavki
                        FOR XML PATH('')), 1, 2, N'');
                    SET @Nomera_zayavok_Str = N'Заявки по договору: ' + @Nomera_zayavok;
                    DECLARE @Nomer_dogovora_Str VARCHAR(40);
                    SET @Nomer_dogovora_Str = CONVERT(VARCHAR(40), @Nomer_dogovora);
                    DECLARE @ErrMsg VARCHAR(200);
```

```
                SET @ErrMsg = ERROR_MESSAGE();

            EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Удаление трудового договора',
                @Nomer_dogovora_Str, N'Отказано',
                @Nomera_zayavok_Str;
            DECLARE @Soobshchenie_otkaz NVARCHAR(2048) =
              N'Удаление договора запрещено, договор удерживают заявки № ' +
              CONVERT(NVARCHAR(200), @Nomera_zayavok) +
              N'. Удалите заявки в главном окне и повторите операцию.';

            THROW 50025, @Soobshchenie_otkaz, 1;
          END

          BEGIN TRY
            BEGIN TRANSACTION;

            DELETE FROM Trudovoy_dogovor WHERE Nomer_dogovora = @Nomer_dogovora;

                SET @Nomer_dogovora_Str = CONVERT(VARCHAR(40), @Nomer_dogovora);

            EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Удаление трудового договора',
                @Nomer_dogovora_Str, N'Успешно', NULL;

            COMMIT TRANSACTION;
            RETURN 0;
          END TRY
          BEGIN CATCH
            IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
            THROW;
          END CATCH
        END
        GO

        IF OBJECT_ID(N'usp_Soglasie_Kandidata', N'P') IS NOT NULL DROP PROCEDURE
usp_Soglasie_Kandidata;
        GO
        CREATE PROCEDURE usp_Soglasie_Kandidata
          @Nomer_kandidata INT
        AS
        BEGIN
          SET NOCOUNT ON;
```

```
    IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
      THROW 50011, N'Указанный кандидат не найден в базе данных.', 1;

    SELECT
      k.FIO                                    AS [Кандидат],
      ISNULL(s.Status, N'Не оформлено')        AS [Состояние согласия],
      ISNULL(CONVERT(VARCHAR(10), s.Data_polucheniya, 104), N'') AS [Дата получения],
      ISNULL(s.Nomer_soglasiya, 0)             AS [Номер согласия],
      ISNULL(so.FIO, N'')                      AS [Сотрудник отдела кадров]
    FROM Kandidat AS k
      OUTER APPLY (SELECT TOP (1) *
                   FROM Soglasie_na_obrabotku_PDn AS z
                   WHERE z.Nomer_kandidata = k.Nomer_kandidata
                   ORDER BY z.Data_polucheniya DESC, z.Nomer_soglasiya DESC) AS s
      LEFT JOIN Sotrudnik AS so ON so.Kod_sotrudnika = s.Kod_sotrudnika
    WHERE k.Nomer_kandidata = @Nomer_kandidata;
  END
  GO

  IF OBJECT_ID(N'usp_Soglasie_Sohranit', N'P') IS NOT NULL DROP PROCEDURE usp_Soglasie_Sohranit;
  GO
  CREATE PROCEDURE usp_Soglasie_Sohranit
    @Nomer_kandidata  INT,
    @Status           VARCHAR(20),
    @Data_polucheniya DATETIME,
    @Kod_sotrudnika   INT = NULL,
    @Polzovatel       VARCHAR(40) = NULL
  AS
  BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
      THROW 50011, N'Указанный кандидат не найден в базе данных.', 1;

    IF @Status NOT IN (N'Получено', N'Ожидается', N'Отказ')
      THROW 50026, N'Указано недопустимое состояние согласия на обработку персональных данных.', 1;

    IF @Data_polucheniya IS NULL
      THROW 50016, N'Не заполнена дата получения согласия.', 1;
```

```
DECLARE @Nomer_soglasiya INT;

SELECT TOP (1) @Nomer_soglasiya = Nomer_soglasiya
FROM Soglasie_na_obrabotku_PDn
WHERE Nomer_kandidata = @Nomer_kandidata
ORDER BY Data_polucheniya DESC, Nomer_soglasiya DESC;

BEGIN TRY
  BEGIN TRANSACTION;

  IF @Nomer_soglasiya IS NULL
  BEGIN
    SELECT @Nomer_soglasiya = ISNULL(MAX(Nomer_soglasiya), 300) + 1
    FROM Soglasie_na_obrabotku_PDn;

    INSERT INTO Soglasie_na_obrabotku_PDn
      (Nomer_soglasiya, Kod_sotrudnika, Nomer_kandidata, Status, Data_polucheniya)
    VALUES
      (@Nomer_soglasiya, @Kod_sotrudnika, @Nomer_kandidata, @Status, @Data_polucheniya);

                    DECLARE @Nomer_kandidata_Str VARCHAR(40);
                    SET @Nomer_kandidata_Str = CONVERT(VARCHAR(40),
@Nomer_kandidata);
                    DECLARE @Status_Str VARCHAR(40);
                    SET @Status_Str = N'Создано, состояние: ' + @Status;

    EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Оформление согласия на обработку ПДн',
      @Nomer_kandidata_Str, N'Успешно',
      @Status_Str;
  END
  ELSE
  BEGIN
    UPDATE Soglasie_na_obrabotku_PDn
    SET Status = @Status,
      Data_polucheniya = @Data_polucheniya,
      Kod_sotrudnika = ISNULL(@Kod_sotrudnika, Kod_sotrudnika)
    WHERE Nomer_soglasiya = @Nomer_soglasiya;

                    SET @Nomer_kandidata_Str = CONVERT(VARCHAR(40),
@Nomer_kandidata);
                    SET @Status_Str = N'Состояние: ' + @Status;
```

```
            EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Изменение согласия на обработку ПДн',
                @Nomer_kandidata_Str, N'Успешно',
                @Status_Str;
        END

        COMMIT TRANSACTION;
        RETURN 0;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
                SET @Nomer_kandidata_Str = CONVERT(VARCHAR(40), @Nomer_kandidata);
                DECLARE @Err_Msg VARCHAR(200);
                SET @Err_Msg = ERROR_MESSAGE();

        EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Оформление согласия на обработку ПДн',
            @Nomer_kandidata_Str, N'Ошибка', @Err_Msg;
        THROW;
    END CATCH
END
GO

IF OBJECT_ID(N'usp_Propusk_Kandidata', N'P') IS NOT NULL DROP PROCEDURE
usp_Propusk_Kandidata;
GO
CREATE PROCEDURE usp_Propusk_Kandidata
    @Nomer_kandidata INT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
        THROW 50011, N'Указанный кандидат не найден в базе данных.', 1;

    SELECT
        k.FIO                               AS [Кандидат],
        ISNULL(p.Nomer_propuska, 0)         AS [Номер пропуска],
        ISNULL(p.Status_gotovnosti, N'Не оформлен')  AS [Состояние пропуска],
        ISNULL(p.Uroven_dostupa, 0)         AS [Уровень доступа],
        ISNULL(so.FIO, N'')                 AS [Сотрудник отдела кадров]
    FROM Kandidat AS k
        OUTER APPLY (SELECT TOP (1) *
                FROM Propusk AS z
```

```
                WHERE z.Nomer_kandidata = k.Nomer_kandidata
                ORDER BY z.Nomer_propuska DESC) AS p
        LEFT JOIN Sotrudnik AS so ON so.Kod_sotrudnika = p.Kod_sotrudnika
      WHERE k.Nomer_kandidata = @Nomer_kandidata;
    END
    GO

    IF OBJECT_ID(N'usp_Propusk_Sohranit', N'P') IS NOT NULL DROP PROCEDURE usp_Propusk_Sohranit;
    GO
    CREATE PROCEDURE usp_Propusk_Sohranit
      @Nomer_kandidata   INT,
      @Nomer_propuska    INT,
      @Status_gotovnosti VARCHAR(20),
      @Uroven_dostupa    INT,
      @Kod_sotrudnika    INT = NULL,
      @Polzovatel        VARCHAR(40) = NULL
    AS
    BEGIN
      SET NOCOUNT ON;

      IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
        THROW 50011, N'Указанный кандидат не найден в базе данных.', 1;

      IF @Status_gotovnosti NOT IN (N'Новый', N'Заказан', N'Готов')
        THROW 50029, N'Указано недопустимое состояние готовности пропуска.', 1;

      IF @Uroven_dostupa IS NULL OR @Uroven_dostupa <= 0
        THROW 50027, N'Уровень доступа пропуска должен быть положительным числом.', 1;

      IF @Nomer_propuska IS NULL OR @Nomer_propuska <= 0
        SELECT @Nomer_propuska = ISNULL(MAX(Nomer_propuska), 500) + 1 FROM Propusk;
      ELSE IF EXISTS (SELECT 1 FROM Propusk
                WHERE Nomer_propuska = @Nomer_propuska
                  AND Nomer_kandidata <> @Nomer_kandidata)
        THROW 50031, N'Указанный номер пропуска принадлежит другому кандидату.', 1;

      BEGIN TRY
        BEGIN TRANSACTION;

        IF EXISTS (SELECT 1 FROM Propusk WHERE Nomer_propuska = @Nomer_propuska)
          UPDATE Propusk
```

```
            SET Status_gotovnosti = @Status_gotovnosti,
              Uroven_dostupa    = @Uroven_dostupa,
              Kod_sotrudnika    = ISNULL(@Kod_sotrudnika, Kod_sotrudnika)
            WHERE Nomer_propuska = @Nomer_propuska;
          ELSE
            INSERT INTO Propusk
              (Nomer_propuska, Nomer_kandidata, Kod_sotrudnika, Status_gotovnosti, Uroven_dostupa)
            VALUES
              (@Nomer_propuska, @Nomer_kandidata, @Kod_sotrudnika, @Status_gotovnosti,
@Uroven_dostupa);

                    DECLARE @Nomer_propuska_Str VARCHAR(40);
                    SET @Nomer_propuska_Str = CONVERT(VARCHAR(40), @Nomer_propuska);
                    DECLARE @Status_gotovnosti_Str VARCHAR(40);
                    SET @Status_gotovnosti_Str = N'Состояние: ' + @Status_gotovnosti;


          EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Оформление пропуска',
            @Nomer_propuska_Str, N'Успешно',
            @Status_gotovnosti_Str;

          COMMIT TRANSACTION;
          RETURN 0;
        END TRY
        BEGIN CATCH
          IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
                    SET @Nomer_propuska_Str = CONVERT(VARCHAR(40), @Nomer_propuska);
                    DECLARE @ErrMsg VARCHAR(200);
                    SET @ErrMsg = ERROR_MESSAGE();

          EXEC usp_Zapis_v_zhurnal @Polzovatel, N'Оформление пропуска',
            @Nomer_propuska_Str, N'Ошибка', @ErrMsg;
          THROW;
        END CATCH
      END
      GO

      IF OBJECT_ID(N'usp_Dostupnye_Procedury', N'P') IS NOT NULL DROP PROCEDURE
usp_Dostupnye_Procedury;
      GO
      CREATE PROCEDURE usp_Dostupnye_Procedury
        @Rol VARCHAR(20)
```

```
AS
BEGIN
  SET NOCOUNT ON;

  IF @Rol NOT IN (N'Менеджер', N'Специалист', N'Администратор')
    THROW 50028, N'Неизвестная роль пользователя, список процедур не сформирован.', 1;

  DECLARE @Zapreshcheno TABLE (Procedura VARCHAR(40));

  IF @Rol = N'Менеджер'
    INSERT INTO @Zapreshcheno (Procedura) VALUES
      (N'usp_Dogovor_Dobavit'), (N'usp_Dogovor_Redaktirovat'),
      (N'usp_Dogovor_Udalit'), (N'usp_Propusk_Sohranit');

  IF @Rol = N'Специалист'
    INSERT INTO @Zapreshcheno (Procedura) VALUES
      (N'usp_Kandidat_Dannye'), (N'usp_Kandidat_Dobavit'),
      (N'usp_Kandidat_Redaktirovat'), (N'usp_Kandidat_Udalit'),
      (N'usp_Zayavka_Udalit'), (N'usp_Soglasie_Sohranit'),
      (N'usp_Zhurnal_Bezopasnosti'), (N'usp_Zhurnal_za_period');

  SELECT p.name AS [Procedura]
  FROM sys.procedures AS p
  WHERE p.name LIKE N'usp[_]%'
    AND p.name NOT IN (SELECT Procedura FROM @Zapreshcheno)
  ORDER BY p.name ASC;
END
GO

IF OBJECT_ID(N'usp_Kandidat_Svyazi', N'P') IS NOT NULL DROP PROCEDURE usp_Kandidat_Svyazi;
GO
CREATE PROCEDURE usp_Kandidat_Svyazi
  @Nomer_kandidata INT
AS
BEGIN
  SET NOCOUNT ON;

  IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Nomer_kandidata = @Nomer_kandidata)
    THROW 50005, N'Кандидат с указанным номером не найден.', 1;

  SELECT Zapis.[Вид записи], Zapis.[Номер], Zapis.[Дополнительно],
```

```
    Zapis.[Препятствует_удалению]
FROM
(
  SELECT 1 AS [Порядок],
      N'Трудовой договор' AS [Вид записи],
      CONVERT(VARCHAR(20), d.Nomer_dogovora) AS [Номер],
      ISNULL(d.Dolzhnost_kandidata, N'') + N' от ' +
        CONVERT(VARCHAR(10), d.Data_sostavleniya, 104) AS [Дополнительно],
      N'Да' AS [Препятствует_удалению]
  FROM Trudovoy_dogovor AS d
  WHERE d.Nomer_kandidata = @Nomer_kandidata

  UNION ALL

  SELECT 2,
      N'Заявка в подразделение',
      CONVERT(VARCHAR(20), z.Nomer_zayavki),
      z.Otdel + N', состояние: ' + z.Status + N', договор ' +
        CONVERT(VARCHAR(20), z.Nomer_dogovora),
      N'Да'
  FROM Zayavka AS z
  WHERE z.Nomer_kandidata = @Nomer_kandidata

  UNION ALL

  SELECT 3,
      N'Согласие на обработку ПДн',
      CONVERT(VARCHAR(20), s.Nomer_soglasiya),
      N'состояние: ' + s.Status + N', от ' +
        CONVERT(VARCHAR(10), s.Data_polucheniya, 104),
      N'Нет'
  FROM Soglasie_na_obrabotku_PDn AS s
  WHERE s.Nomer_kandidata = @Nomer_kandidata

  UNION ALL

  SELECT 4,
      N'Пропуск',
      CONVERT(VARCHAR(20), p.Nomer_propuska),
      N'состояние: ' + p.Status_gotovnosti + N', уровень доступа ' +
        CONVERT(VARCHAR(10), p.Uroven_dostupa),
      N'Нет'
```

```
        FROM Propusk AS p
        WHERE p.Nomer_kandidata = @Nomer_kandidata
    ) AS Zapis
    ORDER BY Zapis.[Порядок], Zapis.[Номер];
END
GO
```

Приложение В

Листинг модуля главной формы Form1.cs

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace ASZI_OK
{

   public partial class Form1 : Form
   {
      public static string Tekushchiy_polzovatel = "";
      public static string Tekushchaya_rol = "";
      public static int Kod_sotrudnika = 0;
      public static string FIO_sotrudnika = "";

      public Form1()
      {
         InitializeComponent();
         dataGridView1.CellEnter += new DataGridViewCellEventHandler(this.dataGridView1_CellEnter);
         NastroitIkonki();
      }

      private void NastroitIkonki()
      {
         OformitKnopku(button1,  Ikony.Dobavit());        // добавить кандидата
         OformitKnopku(button2,  Ikony.Redaktirovat());   // редактировать кандидата
         OformitKnopku(button3,  Ikony.Udalit());         // удалить кандидата
         OformitKnopku(button4,  Ikony.Dobavit());        // добавить заявку
         OformitKnopku(button5,  Ikony.Redaktirovat());   // редактировать заявку
         OformitKnopku(button6,  Ikony.Udalit());         // удалить заявку
         OformitKnopku(button7,  Ikony.Dogovor());        // трудовые договоры
         OformitKnopku(button8,  Ikony.Soglasie());       // согласие на обработку ПДн
         OformitKnopku(button9,  Ikony.Propusk());        // пропуск
         OformitKnopku(button10, Ikony.Poisk());          // поиск кандидата
         OformitKnopku(button11, Ikony.Svodka());         // сводка по заявкам
         OformitKnopku(button12, Ikony.Gotovnost());      // готовность пропусков
         OformitKnopku(button13, Ikony.Zhurnal());        // журнал событий
         OformitKnopku(button14, Ikony.Testirovanie());   // тестирование процедур
```

```
    OformitKnopku(button15, Ikony.Smena());          // смена пользователя
    OformitKnopku(button16, Ikony.Obnovit());        // обновление данных

    menuKandidatDobavit.Image = Ikony.Dobavit();
    menuKandidatRedaktirovat.Image = Ikony.Redaktirovat();
    menuKandidatUdalit.Image = Ikony.Udalit();
    menuKandidatPoisk.Image = Ikony.Poisk();
    menuZayavkaDobavit.Image = Ikony.Dobavit();
    menuZayavkaRedaktirovat.Image = Ikony.Redaktirovat();
    menuZayavkaUdalit.Image = Ikony.Udalit();
    menuDogovory.Image = Ikony.Dogovor();
    menuSoglasie.Image = Ikony.Soglasie();
    menuPropusk.Image = Ikony.Propusk();
    menuOtchetSvodka.Image = Ikony.Svodka();
    menuOtchetPropuski.Image = Ikony.Gotovnost();
    menuOtchetZhurnal.Image = Ikony.Zhurnal();
    menuTestProcedure.Image = Ikony.Testirovanie();
    menuObnovit.Image = Ikony.Obnovit();
    menuSmenaPolzovatelya.Image = Ikony.Smena();
}

private void OformitKnopku(Button knopka, System.Drawing.Bitmap ikonka)
{
    Ikony.Oformit(knopka, ikonka);
}

private void Form1_Shown(object sender, EventArgs e)
{
    try
    {
        FillKandidaty();
        FillZayavki();
        statusLabel1.Text = "Пользователь: " + Tekushchiy_polzovatel + ", роль: " + Tekushchaya_rol;
        OgranicheniyaPoRoli();
        MessageBox.Show("Подключение к базе данных выполнено");
    }
    catch (Exception exep)
    {
        MessageBox.Show("Подключение к базе данных не выполнено! " + exep.Message +
                    "\r\n\r\nПроверьте имя экземпляра СУБД и имя базы данных " +
                    "в строке соединения (поле ConnStr класса RabotaSBazoy).");
    }
```

```
        }

        private void Form1_FormClosing(object sender, FormClosingEventArgs e)
        {
            if (Tekushchiy_polzovatel != "")
            {
                try
                {
                    RabotaSBazoy.ZapisVZhurnal(Tekushchiy_polzovatel, "Выход из системы",
                                    "Form1", "Успешно", null);
                }
                catch
                {
                    // Сбой регистрации события не должен препятствовать
                    // завершению работы приложения
                }
            }
        }

        private void FillKandidaty()
        {
            dataGridView1.DataSource                                                    =
RabotaSBazoy.MyExecuteProcedure("usp_Kandidaty_Spisok").DefaultView;
        }

        private void FillZayavki()
        {
            if (dataGridView1.CurrentRow == null) return;

            int index = dataGridView1.CurrentRow.Index;
            if (index < 0) return;
            if (dataGridView1[0, index].Value == null) return;

            SqlParameter[] p =
            {
                new SqlParameter("@Nomer_kandidata", SqlDbType.Int)
                {
                    Value = Convert.ToInt32(dataGridView1[0, index].Value)
                }
            };

            dataGridView2.DataSource    =    RabotaSBazoy.MyExecuteProcedure("usp_Zayavki_Kandidata",
```

```
p).DefaultView;
            }

            private int NomerVybrannogoKandidata()
            {
                if (dataGridView1.CurrentRow == null) return 0;

                int index = dataGridView1.CurrentRow.Index;
                if (index < 0 || dataGridView1[0, index].Value == null) return 0;

                return Convert.ToInt32(dataGridView1[0, index].Value);
            }

            private bool ZapolnitFormuRedaktirovaniya(Form4 f)
            {
                int nomer_kandidata = NomerVybrannogoKandidata();
                if (nomer_kandidata == 0) return false;

                SqlParameter[] p =
                {
                    new SqlParameter("@Nomer_kandidata", SqlDbType.Int) { Value = nomer_kandidata },
                    new SqlParameter("@Polzovatel", SqlDbType.VarChar, 40) { Value = Tekushchiy_polzovatel }
                };

                DataTable dt = RabotaSBazoy.MyExecuteProcedure("usp_Kandidat_Dannye", p);

                if (dt.Rows.Count == 0)
                {
                    MessageBox.Show("Данные кандидата не найдены в базе данных.");
                    return false;
                }

                DataRow r = dt.Rows[0];

                f.textBox1.Text = Convert.ToString(r["Nomer_kandidata"]);
                f.textBox2.Text = Convert.ToString(r["FIO"]);
                f.textBox3.Text = Convert.ToString(r["Pasport"]);
                f.textBox4.Text = Convert.ToString(r["INN"]);
                f.textBox5.Text = Convert.ToString(r["SNILS"]);
                f.textBox6.Text = Convert.ToString(r["Telefon"]);
                f.textBox7.Text = Convert.ToString(r["Elektronnaya_pochta"]);
```

```
    // Дата приводится к формату, который подсказан в наименовании поля
    f.textBox8.Text = (r["Data_vihoda"] == DBNull.Value)
        ? ""
        : Convert.ToDateTime(r["Data_vihoda"]).ToString("yyyy-MM-dd");

    return true;
}

private void OgranicheniyaPoRoli()
{
    bool polny_dostup = (Tekushchaya_rol == "Менеджер" || Tekushchaya_rol == "Администратор");

    button1.Enabled = polny_dostup;
    button2.Enabled = polny_dostup;
    button3.Enabled = polny_dostup;
    button6.Enabled = polny_dostup;
    button13.Enabled = polny_dostup;

    menuKandidatDobavit.Enabled = polny_dostup;
    menuKandidatRedaktirovat.Enabled = polny_dostup;
    menuKandidatUdalit.Enabled = polny_dostup;
    menuZayavkaUdalit.Enabled = polny_dostup;
    menuOtchetZhurnal.Enabled = polny_dostup;
}

private bool VypolnitProceduru(string imya_procedury, SqlParameter[] p)
{
    try
    {
        RabotaSBazoy.MyExecuteProcedureWithReturn(imya_procedury, p);

        string dopolnitelno = "";

        if (p != null)
        {
            foreach (SqlParameter par in p)
            {
                if (par.Direction == ParameterDirection.Output)
                    dopolnitelno = dopolnitelno + "\r\n" +
                                par.ParameterName.TrimStart('@') + ": " +
                                Convert.ToString(par.Value);
            }
```

```
        }

        MessageBox.Show("Операция выполнена успешно." + dopolnitelno);
        return true;
    }
    catch (SqlException exep)
    {
        MessageBox.Show("Операция не выполнена. " + exep.Message +
                    PoyasnenieKodaOshibki(exep.Number));
        return false;
    }
    catch (Exception exep)
    {
        MessageBox.Show("Операция не выполнена. " + exep.Message);
        return false;
    }
}

private SqlParameter[] SformirovatParametryKandidata(string nomer, string fio, string pasport,
    string inn, string snils, string telefon, string pochta, string data, string polzovatel)
{
    int nomer_kandidata = 0;
    long inn_chislo = 0, snils_chislo = 0, telefon_chislo = 0;
    DateTime data_vihoda;

    if (nomer != null)
    {
        if (!int.TryParse(nomer.Trim(), out nomer_kandidata) || nomer_kandidata <= 0)
        {
            MessageBox.Show("Поле «Номер кандидата» заполнено неверно. " +
                        "Укажите целое положительное число.");
            return null;
        }
    }

    if (fio.Trim() == "")
    {
        MessageBox.Show("Не заполнено поле «ФИО».");
        return null;
    }

    if (inn.Trim() != "" && !long.TryParse(inn.Trim(), out inn_chislo))
```

```
            {
                MessageBox.Show("Поле «ИНН» заполнено неверно. Допускаются только цифры.");
                return null;
            }

            if (snils.Trim() != "" && !long.TryParse(snils.Trim(), out snils_chislo))
            {
                MessageBox.Show("Поле «СНИЛС» заполнено неверно. Допускаются только цифры.");
                return null;
            }

            if (telefon.Trim() != "" && !long.TryParse(telefon.Trim(), out telefon_chislo))
            {
                MessageBox.Show("Поле «Телефон» заполнено неверно. Допускаются только цифры.");
                return null;
            }

            if (pochta.Trim() == "" || pochta.IndexOf('@') < 0)
            {
                MessageBox.Show("Поле «Электронная почта» заполнено неверно. Укажите адрес вида
user@domen.ru.");
                return null;
            }

            if (!DateTime.TryParse(data.Trim(), out data_vihoda))
            {
                MessageBox.Show("Поле «Дата выхода» заполнено неверно. Укажите дату, например 2026-
05-12.");
                return null;
            }

            List<SqlParameter> spisok = new List<SqlParameter>();

            if (nomer != null)
                spisok.Add(new SqlParameter("@Nomer_kandidata", SqlDbType.Int) { Value = nomer_kandidata
});
            else
            {
                SqlParameter pNomer = new SqlParameter("@Nomer_kandidata", SqlDbType.Int);
                pNomer.Direction = ParameterDirection.Output;
                spisok.Add(pNomer);
            }
```

```
            spisok.Add(new SqlParameter("@FIO", SqlDbType.VarChar, 40) { Value = fio.Trim() });
            spisok.Add(new SqlParameter("@Pasport", SqlDbType.VarChar, 150)
            {
               Value = (pasport.Trim() == "") ? (object)DBNull.Value : pasport.Trim()
            });
            spisok.Add(new SqlParameter("@INN", SqlDbType.BigInt)
            {
               Value = (inn.Trim() == "") ? (object)DBNull.Value : inn_chislo
            });
            spisok.Add(new SqlParameter("@SNILS", SqlDbType.BigInt)
            {
               Value = (snils.Trim() == "") ? (object)DBNull.Value : snils_chislo
            });
            spisok.Add(new SqlParameter("@Telefon", SqlDbType.BigInt)
            {
               Value = (telefon.Trim() == "") ? (object)DBNull.Value : telefon_chislo
            });
            spisok.Add(new SqlParameter("@Elektronnaya_pochta", SqlDbType.VarChar, 40) { Value =
pochta.Trim() });
            spisok.Add(new SqlParameter("@Data_vihoda", SqlDbType.DateTime) { Value = data_vihoda });
            spisok.Add(new SqlParameter("@Polzovatel", SqlDbType.VarChar, 40) { Value = polzovatel });

            return spisok.ToArray();
         }

         private void ZapolnitFormuZayavki(Form5 f)
         {
            f.textBox1.Text = DateTime.Today.ToString("yyyy-MM-dd");
            f.textBox3.Text = "1";
            f.textBox2.Text = FIO_sotrudnika + " (код " + Kod_sotrudnika + ")";

            f.comboBox3.DataSource = RabotaSBazoy.MyExecuteProcedure("usp_Kandidaty_Vybor");
            f.comboBox3.DisplayMember = "Naimenovanie";
            f.comboBox3.ValueMember = "Kod";

            f.comboBox5.DataSource = RabotaSBazoy.MyExecuteProcedure("usp_Resursy_Spisok");
            f.comboBox5.DisplayMember = "Naimenovanie";
            f.comboBox5.ValueMember = "Kod";

         }
```

```
        private SqlParameter[] SformirovatParametryZayavki(string data, string otdel, string status,
            int nomer_kandidata, int nomer_dogovora, int kod_resursa, string kolichestvo, string polzovatel)
        {
            int kolichestvo_chislo;
            DateTime data_sozdaniya;

            if (otdel != "ИТ" && otdel != "АХУ" && otdel != "Бухгалтерия")
            {
                MessageBox.Show("Не выбрано подразделение заявки.");
                return null;
            }

            if (status != "Новая" && status != "В работе" && status != "Исполнена")
            {
                MessageBox.Show("Не выбрано состояние заявки.");
                return null;
            }

            if (nomer_kandidata <= 0)
            {
                MessageBox.Show("Не выбран кандидат. Если список пуст, сначала добавьте кандидата.");
                return null;
            }

            if (nomer_dogovora <= 0)
            {
                MessageBox.Show("Не выбран трудовой договор. По кандидату должен быть подготовлен
договор.");
                return null;
            }

            if (kod_resursa <= 0)
            {
                MessageBox.Show("Не выбран запрашиваемый ресурс.");
                return null;
            }

            if (!int.TryParse(kolichestvo.Trim(), out kolichestvo_chislo) || kolichestvo_chislo <= 0)
            {
                MessageBox.Show("Поле «Количество» заполнено неверно. Укажите целое положительное
число.");
                return null;
```

```
        }

        if (!DateTime.TryParse(data.Trim(), out data_sozdaniya))
        {
            MessageBox.Show("Поле «Дата создания» заполнено неверно. Укажите дату, например
2026-05-06.");
            return null;
        }

        SqlParameter pNomer = new SqlParameter("@Nomer_zayavki", SqlDbType.Int);
        pNomer.Direction = ParameterDirection.Output;

        SqlParameter[] p =
        {
            pNomer,
            new SqlParameter("@Otdel", SqlDbType.VarChar, 20) { Value = otdel },
            new SqlParameter("@Data_sozdaniya", SqlDbType.DateTime) { Value = data_sozdaniya },
            new SqlParameter("@Status", SqlDbType.VarChar, 20) { Value = status },
            new SqlParameter("@Nomer_kandidata", SqlDbType.Int) { Value = nomer_kandidata },
            new SqlParameter("@Kod_sotrudnika", SqlDbType.Int) { Value = Kod_sotrudnika },
            new SqlParameter("@Nomer_dogovora", SqlDbType.Int) { Value = nomer_dogovora },
            new SqlParameter("@Kod_resursa", SqlDbType.Int) { Value = kod_resursa },
            new SqlParameter("@Kolichestvo", SqlDbType.Int) { Value = kolichestvo_chislo },
            new SqlParameter("@Polzovatel", SqlDbType.VarChar, 40) { Value = polzovatel }
        };

        return p;
    }

    private void button1_Click(object sender, EventArgs e)
    {
        Form3 f = new Form3();
        SqlParameter[] p = null;

        while (p == null)
        {
            if (f.ShowDialog() != DialogResult.OK) return;

            p = SformirovatParametryKandidata(null, f.textBox1.Text, f.textBox2.Text,
                f.textBox3.Text, f.textBox4.Text, f.textBox5.Text, f.textBox6.Text,
                f.textBox7.Text, Tekushchiy_polzovatel);
        }
```

```
    if (VypolnitProceduru("usp_Kandidat_Dobavit", p))
        FillKandidaty();
}

private void button2_Click(object sender, EventArgs e)
{
    if (dataGridView1.Rows.Count <= 1) return;

    Form4 f = new Form4();
    if (!ZapolnitFormuRedaktirovaniya(f)) return;

    SqlParameter[] p = null;

    while (p == null)
    {
        if (f.ShowDialog() != DialogResult.OK) return;

        p = SformirovatParametryKandidata(f.textBox1.Text, f.textBox2.Text, f.textBox3.Text,
            f.textBox4.Text, f.textBox5.Text, f.textBox6.Text, f.textBox7.Text,
            f.textBox8.Text, Tekushchiy_polzovatel);
    }

    if (VypolnitProceduru("usp_Kandidat_Redaktirovat", p))
        FillKandidaty();
}

private bool EstSvyaziKandidata(int nomer_kandidata)
{
    SqlParameter[] ps =
    {
        new SqlParameter("@Nomer_kandidata", SqlDbType.Int) { Value = nomer_kandidata }
    };

    DataTable svyazi = RabotaSBazoy.MyExecuteProcedure("usp_Kandidat_Svyazi", ps);

    string mezhayut = "";
    string udalyayutsya = "";
    string poryadok = "";

    foreach (DataRow r in svyazi.Rows)
    {
```

```
        string opisanie = "\r\n - " + Convert.ToString(r["Вид записи"]) +
                          " № " + Convert.ToString(r["Номер"]) +
                          " (" + Convert.ToString(r["Дополнительно"]) + ")";

        if (Convert.ToString(r["Препятствует_удалению"]) == "Да")
        {
            mezhayut = mezhayut + opisanie;

            // Подсказка о том, каким окном удаляется мешающая запись
            if (Convert.ToString(r["Вид записи"]).StartsWith("Трудовой"))
            {
                if (poryadok.IndexOf("делопроизводству") < 0)
                    poryadok = poryadok + "\r\n - трудовой договор удаляет специалист " +
                               "по делопроизводству в окне «Трудовые договоры»;";
            }
            else if (poryadok.IndexOf("Удалить заявку") < 0)
                poryadok = poryadok + "\r\n - заявку удаляют в главном окне командой " +
                           "«Удалить заявку»;";
        }
        else
            udalyayutsya = udalyayutsya + opisanie;
    }

    if (mezhayut.Length == 0) return false;

    string tekst = "Удаление кандидата невозможно, за кандидатом закреплены записи:" +
                   mezhayut + "\r\n\r\nПорядок удаления:" + poryadok.TrimEnd(';') + ".";

    // Если текущей роли работа с трудовыми договорами закрыта,
    // приложение предупреждает об этом сразу, а не после отказа
    if (poryadok.IndexOf("делопроизводству") >= 0 && !MozhnoRabotatSDogovorami())
        tekst = tekst + "\r\n\r\nТекущая роль " + Tekushchaya_rol +
                ". Трудовой договор этой роли недоступен, обратитесь к специалисту " +
                "по делопроизводству или к администратору АСЗИ.";

    if (udalyayutsya.Length > 0)
        tekst = tekst + "\r\n\r\nВместе с кандидатом будут удалены:" + udalyayutsya;

    MessageBox.Show(tekst, "Удаление кандидата",
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return true;
}
```

```
public static bool MozhnoRabotatSDogovorami()
{
   return (Tekushchaya_rol == "Специалист" || Tekushchaya_rol == "Администратор");
}

public static string PoyasnenieKodaOshibki(int kod)
{
   if (kod == 229)
      return "\r\n\r\nОперация закрыта на уровне базы данных для учётной записи " +
           "подключения. Выполните раздел 3 скрипта PZ11_01_Diagnostika_" +
           "podklucheniya.sql, чтобы включить учётную запись в прикладную роль, " +
           "которой эта операция разрешена.";

   return "";
}

private void button3_Click(object sender, EventArgs e)
{
   if (dataGridView1.Rows.Count <= 1) return;

   int nomer_kandidata = NomerVybrannogoKandidata();
   if (nomer_kandidata == 0) return;

   try
   {
      // Перечень связанных записей выводится процедурой usp_Kandidat_Svyazi
      if (EstSvyaziKandidata(nomer_kandidata)) return;
   }
   catch (Exception exep)
   {
      MessageBox.Show("Не удалось проверить связанные записи. " + exep.Message);
      return;
   }

   Form7 f = new Form7();
   string fio = Convert.ToString(dataGridView1[1, dataGridView1.CurrentRow.Index].Value);

   f.label2.Text = nomer_kandidata + " - " + fio;

   if (f.ShowDialog() == DialogResult.OK)
   {
```

```
            SqlParameter[] p =
            {
                new SqlParameter("@Nomer_kandidata", SqlDbType.Int) { Value = nomer_kandidata },
                new SqlParameter("@Polzovatel", SqlDbType.VarChar, 40) { Value = Tekushchiy_polzovatel
}
            };

            if (VypolnitProceduru("usp_Kandidat_Udalit", p))
                FillKandidaty();
            else
            {
                try
                {
                    if (!EstSvyaziKandidata(nomer_kandidata))
                        MessageBox.Show("Связанные записи к этому моменту удалены. " +
                                "Повторите удаление кандидата.",
                                "Удаление кандидата",
                                MessageBoxButtons.OK, MessageBoxIcon.Information);
                }
                catch (Exception exep)
                {
                    MessageBox.Show("Не удалось проверить связанные записи. " + exep.Message);
                }

                FillKandidaty();
            }
        }
    }

    // Кнопка «Добавить заявку». Номер заявки присваивает база данных,
    // ресурс и количество указываются в форме
    private void button4_Click(object sender, EventArgs e)
    {
        Form5 f = new Form5();
        SqlParameter[] p = null;

        try
        {
            ZapolnitFormuZayavki(f);
        }
        catch (Exception exep)
        {
```

```
        MessageBox.Show("Не удалось получить справочные данные для формы. " + exep.Message);
        return;
    }

    while (p == null)
    {
        if (f.ShowDialog() != DialogResult.OK) return;

        p = SformirovatParametryZayavki(f.textBox1.Text,
            Convert.ToString(f.comboBox1.SelectedItem),
            Convert.ToString(f.comboBox2.SelectedItem),
            (f.comboBox3.SelectedValue == null) ? 0 : Convert.ToInt32(f.comboBox3.SelectedValue),
            (f.comboBox4.SelectedValue == null) ? 0 : Convert.ToInt32(f.comboBox4.SelectedValue),
            (f.comboBox5.SelectedValue == null) ? 0 : Convert.ToInt32(f.comboBox5.SelectedValue),
            f.textBox3.Text, Tekushchiy_polzovatel);
    }

    if (VypolnitProceduru("usp_Zayavka_Sozdat", p))
        FillZayavki();
}

// Кнопка «Редактировать заявку»
private void button5_Click(object sender, EventArgs e)
{
    if (dataGridView2.CurrentRow == null) return;

    int index = dataGridView2.CurrentRow.Index;

    Form6 f = new Form6();
    f.textBox1.Text = Convert.ToString(dataGridView2[0, index].Value);

    // Текущие значения подразделения и состояния подставляются
    // в выпадающие списки
    string otdel = Convert.ToString(dataGridView2[1, index].Value);
    string status = Convert.ToString(dataGridView2[3, index].Value);

    if (f.comboBox1.Items.Contains(otdel)) f.comboBox1.SelectedItem = otdel;
    if (f.comboBox2.Items.Contains(status)) f.comboBox2.SelectedItem = status;

    if (f.ShowDialog() == DialogResult.OK)
    {
        SqlParameter[] p =
```

```
                {
                    new SqlParameter("@Nomer_zayavki", SqlDbType.Int) { Value = Convert.ToInt32(f.textBox1.Text) },
                    new SqlParameter("@Otdel", SqlDbType.VarChar, 20) { Value = Convert.ToString(f.comboBox1.SelectedItem) },
                    new SqlParameter("@Status", SqlDbType.VarChar, 20) { Value = Convert.ToString(f.comboBox2.SelectedItem) },
                    new SqlParameter("@Polzovatel", SqlDbType.VarChar, 40) { Value = Tekushchiy_polzovatel }
                };

                if (VypolnitProceduru("usp_Zayavka_Redaktirovat", p))
                    FillZayavki();
            }
        }

        // Кнопка «Удалить заявку»
        private void button6_Click(object sender, EventArgs e)
        {
            if (dataGridView2.CurrentRow == null) return;

            int index = dataGridView2.CurrentRow.Index;

            Form7 f = new Form7();
            string nomer_zayavki = Convert.ToString(dataGridView2[0, index].Value);
            string otdel = Convert.ToString(dataGridView2[1, index].Value);

            f.Text = "Удалить заявку";
            f.label2.Text = nomer_zayavki + " - " + otdel;

            if (f.ShowDialog() == DialogResult.OK)
            {
                SqlParameter[] p =
                {
                    new SqlParameter("@Nomer_zayavki", SqlDbType.Int) { Value = Convert.ToInt32(nomer_zayavki) },
                    new SqlParameter("@Polzovatel", SqlDbType.VarChar, 40) { Value = Tekushchiy_polzovatel }
                };

                if (VypolnitProceduru("usp_Zayavka_Udalit", p))
                    FillZayavki();
```

```
    }
}

// Кнопка «Трудовые договоры»
private void button7_Click(object sender, EventArgs e)
{
    int nomer_kandidata = NomerVybrannogoKandidata();
    if (nomer_kandidata == 0) return;

    Form13 f = new Form13();
    f.Nomer_kandidata = nomer_kandidata;
    f.FIO_kandidata = Convert.ToString(dataGridView1[1, dataGridView1.CurrentRow.Index].Value);
    f.label1.Text = "Трудовые договоры кандидата " + nomer_kandidata +
                ", " + f.FIO_kandidata;
    f.ZapolnitSpisok();
    f.ShowDialog();

    ObnovitDannye();
}

// Кнопка «Согласие на обработку ПДн»
private void button8_Click(object sender, EventArgs e)
{
    int nomer_kandidata = NomerVybrannogoKandidata();
    if (nomer_kandidata == 0) return;

    Form15 f = new Form15();
    f.Nomer_kandidata = nomer_kandidata;
    f.FIO_kandidata = Convert.ToString(dataGridView1[1, dataGridView1.CurrentRow.Index].Value);
    f.Zapolnit();
    f.ShowDialog();

    ObnovitDannye();
}

// Кнопка «Пропуск»
private void button9_Click(object sender, EventArgs e)
{
    int nomer_kandidata = NomerVybrannogoKandidata();
    if (nomer_kandidata == 0) return;

    Form16 f = new Form16();
```

```
    f.Nomer_kandidata = nomer_kandidata;
    f.FIO_kandidata = Convert.ToString(dataGridView1[1, dataGridView1.CurrentRow.Index].Value);
    f.Zapolnit();
    f.ShowDialog();

    ObnovitDannye();
}

private void ObnovitDannye()
{
    int nomer_kandidata = NomerVybrannogoKandidata();

    try
    {
        FillKandidaty();

        // После обновления списка восстанавливается выбор строки,
        // чтобы таблица заявок осталась по прежнему кандидату
        if (nomer_kandidata > 0)
        {
            foreach (DataGridViewRow stroka in dataGridView1.Rows)
            {
                if (stroka.Cells[0].Value != null &&
                    Convert.ToInt32(stroka.Cells[0].Value) == nomer_kandidata)
                {
                    stroka.Selected = true;
                    dataGridView1.CurrentCell = stroka.Cells[0];
                    break;
                }
            }
        }

        FillZayavki();
        statusLabel1.Text = "Пользователь: " + Tekushchiy_polzovatel +
                            ", роль: " + Tekushchaya_rol +
                            ". Данные обновлены " + DateTime.Now.ToString("HH:mm:ss");
    }
    catch (Exception exep)
    {
        MessageBox.Show("Не удалось обновить данные. " + exep.Message);
    }
}
```

```
        // Кнопка «Обновить данные»
        private void button16_Click(object sender, EventArgs e)
        {
            ObnovitDannye();
        }

        // Кнопка «Поиск кандидата»
        private void button10_Click(object sender, EventArgs e)
        {
            Form12 f = new Form12();
            f.ShowDialog();
        }

        // Кнопка «Сводка по заявкам»
        private void button11_Click(object sender, EventArgs e)
        {
            Form8 f = new Form8();
            f.dataGridView1.DataSource =
RabotaSBazoy.MyExecuteProcedure("usp_Svodka_Zayavok").DefaultView;
            f.ShowDialog();
        }

        // Кнопка «Готовность пропусков»
        private void button12_Click(object sender, EventArgs e)
        {
            Form9 f = new Form9();
            f.dataGridView1.DataSource =
RabotaSBazoy.MyExecuteProcedure("usp_Propuski_Gotovnost").DefaultView;
            f.ShowDialog();
        }

        // Кнопка «Журнал событий безопасности»
        private void button13_Click(object sender, EventArgs e)
        {
            Form10 f = new Form10();
            f.ShowDialog();
        }

        // Кнопка «Тестировать хранимые процедуры»
        private void button14_Click(object sender, EventArgs e)
        {
```

```
            Form11 f = new Form11();
            f.ShowDialog();
        }

        // Кнопка «Сменить пользователя»
        private void button15_Click(object sender, EventArgs e)
        {
            using (Form2 f = new Form2())
            {
                if (f.ShowDialog() == DialogResult.OK)
                {
                    statusLabel1.Text = "Пользователь: " + Tekushchiy_polzovatel + ", роль: " + Tekushchaya_rol;
                    OgranicheniyaPoRoli();
                }
            }
        }

        // Выбор строки в таблице кандидатов
        private void dataGridView1_CellEnter(object sender, DataGridViewCellEventArgs e)
        {
            try
            {
                FillZayavki();
            }
            catch (Exception exep)
            {
                MessageBox.Show("Не удалось получить заявки кандидата. " + exep.Message);
            }
        }

        // Пункт меню «Сервис» -> «О программе»
        private void MenuOProgramme_Click(object sender, EventArgs e)
        {
            MessageBox.Show("Защищенная автоматизированная информационная система отдела кадров
" +
                            "ООО «Маркетинг с умом». Вариант № 10. Версия 1.0",
                            "О программе", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
}
```

Приложение Г

Листинг класса доступа к базе данных RabotaSBazoy.cs

```
using System;
using System.Data;
using System.Data.SqlClient;

namespace ASZI_OK
{
   class RabotaSBazoy
   {
      public static string ConnStr =
         @"Data Source=localhost;Initial Catalog=Kuznetsov_marketing;Integrated Security=True;Connect
Timeout=10;";

      public static string ConnStrFile =
         @"Data Source=.\SQLEXPRESS;AttachDbFilename=" +
         AppDomain.CurrentDomain.BaseDirectory +
         @"Baza\Kuznetsov_marketing.mdf;Integrated Security=True;User Instance=True";

      public static void MyExecuteNonQuery(string SqlText)
      {
         using (SqlConnection cn = new SqlConnection(ConnStr))
         {
            cn.Open();
            using (SqlCommand cmd = cn.CreateCommand())
            {
               cmd.CommandText = SqlText;
               cmd.ExecuteNonQuery();
            }
         }
      }

      public static DataTable MyExecuteQuery(string SqlText)
      {
         using (SqlConnection cn = new SqlConnection(ConnStr))
         {
            cn.Open();
            SqlDataAdapter da = new SqlDataAdapter(SqlText, cn);
            DataTable dt = new DataTable();
            da.Fill(dt);
            return dt;
```

```
    }
}

public static DataTable MyExecuteProcedure(string Imya_procedury)
{
    using (SqlConnection cn = new SqlConnection(ConnStr))
    {
        cn.Open();
        SqlCommand cmd = new SqlCommand(Imya_procedury, cn);
        cmd.CommandType = CommandType.StoredProcedure;
        SqlDataAdapter da = new SqlDataAdapter(cmd);
        DataTable dt = new DataTable();
        da.Fill(dt);
        return dt;
    }
}

public static DataTable MyExecuteProcedure(string Imya_procedury, SqlParameter[] parametry)
{
    using (SqlConnection cn = new SqlConnection(ConnStr))
    {
        cn.Open();
        SqlCommand cmd = new SqlCommand(Imya_procedury, cn);
        cmd.CommandType = CommandType.StoredProcedure;
        if (parametry != null)
            cmd.Parameters.AddRange(parametry);
        SqlDataAdapter da = new SqlDataAdapter(cmd);
        DataTable dt = new DataTable();
        da.Fill(dt);
        return dt;
    }
}

public static int MyExecuteProcedureWithReturn(string Imya_procedury, SqlParameter[] parametry)
{
    using (SqlConnection cn = new SqlConnection(ConnStr))
    {
        cn.Open();
        SqlCommand cmd = new SqlCommand(Imya_procedury, cn);
        cmd.CommandType = CommandType.StoredProcedure;
        if (parametry != null)
            cmd.Parameters.AddRange(parametry);
```

```
            SqlParameter pKod = new SqlParameter("@Kod_vozvrata", SqlDbType.Int);
            pKod.Direction = ParameterDirection.ReturnValue;
            cmd.Parameters.Add(pKod);

            cmd.ExecuteNonQuery();
            return Convert.ToInt32(pKod.Value);
        }
    }

    public static void ZapisVZhurnal(string polzovatel, string sobytie, string obekt,
                                     string rezultat, string prichina)
    {
        SqlParameter[] p =
        {
            new SqlParameter("@Polzovatel", SqlDbType.VarChar, 40) { Value = polzovatel },
            new SqlParameter("@Sobytie",    SqlDbType.VarChar, 40) { Value = sobytie },
            new SqlParameter("@Obekt",      SqlDbType.VarChar, 40) { Value = obekt },
            new SqlParameter("@Rezultat",   SqlDbType.VarChar, 20) { Value = rezultat },
            new SqlParameter("@Prichina",   SqlDbType.VarChar, 150) { Value = (object)prichina ??
DBNull.Value }
        };
        MyExecuteProcedureWithReturn("usp_Zapis_v_zhurnal", p);
    }
  }
}
```